

практический

TypeScript

TS

Александр Герасимов

Оглавление

 Быстрая навигация по разделам

 Основы TypeScript (01-10)	 Система типов (11-30)
 Лучшие практики (31-49)	 Продвинутые типы (50-64)
 Публикация и API (65-71)	 Производительность и миграция (72-83)
 Справочные материалы	

 Основы TypeScript

1	Как TypeScript связан с JavaScript
2	Как Выбирать нужные опции в TypeScript
3	Генерация кода не зависит от типов
4	Что такое структурная типизация?
5	Как ограничить тип any
6	Как использовать редактор для получения информации о системе типов
7	Типы как множества значений
8	Как выражать принадлежность сущностей к пространству типов или пространству значений
9	Как отдавать предпочтение аннотациям типов, а не утверждениям
10	Как и для чего избегать объектных типов-обертки (String, Number, Boolean, Symbol, BigInt)

 Система типов

11 Проверка лишних свойств

12 Как задать тип всему функциональному выражению

13 Различия между type и interface

14 Readonly в TypeScript

15 Как использовать операции типов

16 Используйте более точные альтернативы для сигнатур индексов

17 Избегайте использования числовых сигнатур индексов

18 Не засорять код лишними аннотациями типов

19 Используйте разные переменные для разных типов

20 Что такое расширение типов в TypeScript

21 Почему лучше создавать объекты целиком

22 Сужение типов

23 Как быть последовательным с использованием псевдонимов

24 Как использовать контекст при выводе типов

25 Эволюционирующие типы

26 Функциональные конструкции и библиотеки для улучшения работы с типами

27 Async-функции вместо обратных вызовов

28 Классы и каррирование для создания новых точек вывода типов

29 Моделирование допустимых состояний (valid states)

30 Будь консервативным в отправке, либеральным в приёме



Лучшие практики

31 Не повторять информацию типа в документации

32 Избегать включения null или undefined в псевдонимы типов

33 Вынесение null на периферию типов

34 Предпочитай объединения интерфейсов (union of interfaces)

35 Использование более узких (строгих) типов вместо `string`

36 Использовать отдельный тип для специальных значений

37 Ограничить использование опциональных свойств

38 Избегайте повторяющихся параметров одного типа

39 Унификация типов, вместо моделирования

40 Лучше неточно, чем недостоверно

41 Имена типов на языке предметной области задачи

42 Типы на основе API и спецификаций, а не данных

43 Использовать максимально узкий диапазон для типов any

44 Более точные варианты any

45 Скрывать небезопасные утверждения типов в хорошо типизированных функциях

46 unknown вместо any для значений с неизвестным типом

47 Типобезопасные решения вместо динамической модификации

48 Избегайте ловушек надежности

49 Отслеживайте зону охвата типов для предотвращения регрессии в типобезопасности

Продвинутые типы

50 Обобщённые (generics) конструкции

51 Избегайте лишних параметров типа

52 Лучше условные типы, чем перегруженные сигнатуры

53 Распределение объединений

54 Шаблонные литеральные типы для моделирования DSL и отношений между строками

55 Как писать тесты для типов

56 Обращайте внимание на отображение типов

57 Отдавайте предпочтение обобщенным типам с хвостовой рекурсией

58 Кодогенерация как альтернатива сложным типам

59 Типы `never` для выполнения проверки полноты

60 Итерация по объектам

61 Типы записей для поддержания синхронизации значений

62 Остаточные параметры и типы кортежа для моделирования функций с переменным числом аргументов

63 Используйте опциональные свойства `never` для моделирования исключающего ИЛИ

64 Маркировки для номинальной типизации

Публикация и API

65 Размещать TypeScript и `@types` в `devDependencies`

66 Проверять совместимость трех версий, задействованных в объявлениях типов

67 Экспортировать все типы, которые появляются в открытом (публичном) API

68 TSDoc для комментариев к API

69 Определять тип `this` в обратных вызовах, если он является частью API

70 Зеркалирование типов для разрыва зависимостей

71 Аугментацию модулей для улучшения типов

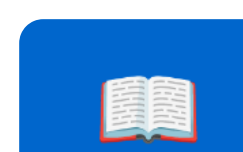
✂ Производительность и миграция

72 Отдавать предпочтение функциональности ECMAScript перед TypeScript

73 Source Map для отладки TypeScript

- 74 Реконструкция типов на стадии выполнения
- 75 Иерархия DOM (Document Object Model)
- 76 Точная модель среды выполнения
- 77 Проверка типов и юнит-тестирование
- 78 Быстродействие компилятора
- 79 Использовать современный JavaScript
- 80 @ts-check и JSDoc в TypeScript
- 81 allowJs для совмещения TypeScript и JavaScript
- 82 Конвертируйте модуль за модулем вверх по графу зависимостей
- 83 Не считайте миграцию завершённой, пока не включите noImplicitAny

Справочные материалы

-  Базовые типы
-  Дженерики
-  Специальные типы

Как использовать навигацию

В PDF-версии вы можете:

- Использовать закладки в боковой панели PDF-ридера
- Кликать по ссылкам в оглавлении для быстрого перехода
- Использовать поиск (Ctrl+F) для нахождения конкретных тем
- Переходить между разделами с помощью навигационных блоков

Предисловие

Данная коллекция статей основана на книге «**Эффективный TypeScript**» автора **Дэна Вандеркама**.

Материалы охватывают основные темы TypeScript:

- **Основы языка** - связь с JavaScript, настройка компилятора, структурная типизация
- **Система типов** - типы как множества, операции с типами, расширение типов
- **Лучшие практики** - избегание `any`, правильное использование аннотаций, создание объектов
- **Продвинутые концепции** - `readonly`, различия между `type` и `interface`, индексные сигнатуры

Каждая статья содержит практические примеры кода с пояснениями и рекомендациями по использованию.

Для кого эта книга

- Разработчики, изучающие TypeScript
- JavaScript-разработчики, переходящие на TypeScript
- Те, кто хочет улучшить понимание системы типов TypeScript
- Разработчики, ищущие практические советы и лучшие практики

Как использовать

Каждая статья самодостаточна и может читаться отдельно. Рекомендуется начать с основ (статьи 01-04) и постепенно переходить к более сложным темам.

Введение

Эта книга — для тех кто уже знаком с JavaScript, короткое и понятное практическое руководство по TypeScript с примерами и объяснениями на русском языке.

Вместо длинных объяснений здесь собраны простые советы, правила и примеры. Они помогут:

- понять, как работает TypeScript;
- писать безопасный и чистый код;
- избегать типичных ошибок;
- правильно использовать типы, интерфейсы, обобщения и другие возможности языка;
- улучшить работу с JavaScript, сделав её предсказуемой и удобной.

Читать можно по порядку или выборочно — каждая часть самодостаточна.

Основана на книге «**Эффективный TypeScript**» автора **Дэна Вандеркама**.

Контакты

Александр Герасимов

- Telegram: [@geralex](#)
 - Git: [geralexgit](#)
-



Основы TypeScript (01-10)

Как TypeScript связан с JavaScript

◆ Что такое JavaScript?

JavaScript (JS) — это **язык программирования**, который выполняется в браузере и на сервере (через Node.js). 📌 Он **динамически типизированный** — ты не указываешь типы, и ошибки могут появиться только во время выполнения.

```
let name = "Alice"; // OK
name = 42;           // JS не ругается, но это может сломать программу
```

◆ Что такое TypeScript?

TypeScript (TS) — это **надстройка над JavaScript**, которая добавляет **типизацию** и другие фишки.

- Это не отдельный язык — это **расширение JS**.
- TypeScript **преобразуется (транспилируется)** в обычный JavaScript.
- Все `.ts` файлы превращаются в `.js`, которые уже и запускаются.



Связь между TypeScript и JavaScript:

JavaScript	TypeScript
Динамически типизирован	Статически типизирован
Ошибки во время выполнения	Ошибки ловятся ещё во время написания кода
Гибкий, но легко ошибиться	Жёсткий, но безопасный
Не требует сборки	Требует компиляции в JavaScript (<code>tsc</code>)
<code>let name = "Alice"</code>	<code>let name: string = "Alice"</code>

Как TypeScript работает?

1. Ты пишешь код на TS:

```
let age: number = "42"; // ❌ Ошибка: нельзя присвоить строку числу
```

2. Компилятор TypeScript (`tsc`) **ловит ошибку до запуска**.

3. После компиляции он превращает твой код в обычный JavaScript:

```
let age = "42"; // обычный JS — ошибок не видно
```

4. Браузер/Node.js работает **только с JavaScript**, TypeScript сам по себе **не исполняется**.

Как представить TypeScript?

Представь, что:

- **JavaScript** — это машина без подушек безопасности.
- **TypeScript** — та же машина, но с подушками, камерой заднего вида, датчиками и предупреждениями.

Он **не мешает тебе ехать**, но **предупредит**, если ты свернул не туда.

Важно запомнить:

- TypeScript = **JavaScript + типы**
 - Всё, что работает в JS — работает в TS.
 - TS **помогает писать более безопасный и понятный код**, особенно в больших проектах.
-

Как Выбирать нужные опции в TypeScript

Выбор **нужных опций в TypeScript** — это как настройка велосипеда под себя: чем лучше настроишь, тем удобнее и безопаснее ехать.

Где вообще настраивается TypeScript?

Все опции указываются в файле `tsconfig.json` — это главный конфиг-файл TypeScript проекта.

Пример:

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "commonjs",
    "strict": true,
    "outDir": "dist",
    "rootDir": "src"
  }
}
```

Как выбирать нужные опции?

Вот **ключевые опции**, которые стоит знать и включать почти всегда:

1. ``strict: true`` — строгий режим

Включает **все строгие проверки**:

```
"strict": true
```

Это как режим "максимальной безопасности" — TypeScript будет следить за каждой мелочью (в хорошем смысле).

2. ``noImplicitAny: true`` — запрет на неявный ``any``

Без этого TypeScript может сказать: "Окей, не знаю что за тип, пусть будет `any`".

С этим флагом он говорит: "Укажи явно, что это за тип, или исправь код."

3. ``strictNullChecks: true`` — не смешивать ``null`` и другие типы

С этим флагом нельзя просто так передавать `null` куда угодно. Помогает избежать ошибок "Cannot read property X of null".

4. `target: "ES6"` (или выше)

Указывает, какой **JavaScript** писать на выходе. ES6 — почти всегда нормальный выбор.

```
"target": "ES6"
```

5. `module: "commonjs"` или `"ESNext"`

Говорит, как собирать модули — зависит от среды:

- `commonjs` — для Node.js
- `ESNext` — для современных браузеров / сборщиков (например, Vite)

6. `rootDir` и `outDir`

Указывают, где лежит исходный код и куда складывать скомпилированный JS:

```
"rootDir": "src",  
"outDir": "dist"
```

Другие полезные:

Опция	Что делает
<code>noUnusedLocals</code>	Ошибка, если есть неиспользуемые переменные
<code>noUnusedParameters</code>	Ошибка, если есть неиспользуемые параметры функции
<code>allowJs</code>	Позволяет подключать <code>.js</code> файлы
<code>checkJs</code>	Проверяет <code>.js</code> файлы как <code>.ts</code>
<code>declaration</code>	Генерирует <code>.d.ts</code> файлы (для библиотек)
<code>esModuleInterop</code>	Делает импорт <code>require()</code> и <code>import</code> совместимыми

Советы по выбору:

- Начинай с `tsc --init` — он создаст базовый `tsconfig.json`.
- Включи `strict: true` — и включатся все строгие опции.

3. Потом можно **дополнительно включать/выключать** опции по ситуации.

Минимальный рекомендуемый `tsconfig.json`:

```
{
  "compilerOptions": {
    "target": "ES6",
    "module": "commonjs",
    "strict": true,
    "rootDir": "src",
    "outDir": "dist",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "skipLibCheck": true
  }
}
```

Генерация кода не зависит от типов

Что это значит?

Когда ты пишешь код на **TypeScript**, ты указываешь **типы** (например: `string`, `number`, интерфейсы, дженерики и т.д.). Но...

👉 **TypeScript удаляет все типы при компиляции.** Они **нужны только во время разработки**, чтобы проверять ошибки, подсказывать автокомплит и т.д.

🧠 **То есть: **типы — это как инструкция для разработчика и компилятора**, но не для машины.**

Пример:

```
function greet(name: string): string {
  return "Hello, " + name;
}
```

➡ После компиляции в JavaScript (`tsc greet.ts`), получится:


```
function greet(name) {  
  return "Hello, " + name;  
}
```

Видишь? 👁️ — **Типы пропали!** `: string` нигде нет. JavaScript работает без них и не знает, что они вообще существовали.

Почему это важно?

- ✓ Код **работает как обычный JavaScript**, без замедления
- ✓ Типы **не влияют** на производительность
- ✓ Ты можешь использовать TypeScript **только для безопасности и удобства**
- ⚠ Но: **в рантайме типы не проверяются**

Что будет, если ты ошибся?

```
function double(x: number): number {  
  return x + x;  
}  
  
double("123"); // ❌ Ошибка на этапе разработки
```

TypeScript **покажет ошибку**, но в скомпилированном JS эта проверка **не останется**.

Если нужна проверка типа в рантайме

Ты должен писать её сам:

```
function safeDouble(x: unknown): number {  
  if (typeof x === "number") {  
    return x + x;  
  } else {  
    throw new Error("Not a number!");  
  }  
}
```


🌟 Итог:

Типы в TypeScript	Во время разработки	Во время выполнения (runtime)
Проверяются	✅ Да	❌ Нет
Сохраняются в коде	❌ Нет	❌ Нет
Влияют на производительность	❌ Нет	❌ Нет

Что такое структурная типизация?

Это одна из **фундаментальных фишек TypeScript**, и понять её важно.

🧱 Что такое ****структурная типизация****?

Структурная типизация (или "типизация по форме") означает:

*TypeScript **сравнивает типы по структуре (содержимому)**, а не по имени.*

📦 Пример:

```
interface User {  
  name: string;  
  age: number;  
}  
  
const person = {  
  name: "Alice",  
  age: 25,  
};  
  
const user: User = person; // ✅ Всё ок
```

👉 Даже если `person` не явно типа `User`, TypeScript смотрит:

- У него есть `name: string` ?
- У него есть `age: number` ?

✅ Да! Значит, `person` подходит под `User`.

В других языках (например, Java, C#) — ****номинативная типизация****

Там объект должен быть **именно типа** `User`, иначе будет ошибка, даже если структура совпадает.

Главное правило структурной типизации:

Если у объекта есть все нужные свойства нужных типов — он подходит.

! Из этого следуют плюсы и минусы:

Плюсы:

- Гибкость: можно не писать лишние типы
- Хорошо работает с plain-объектами (из API, JSON и т.п.)
- Упрощает дженерики

Минусы:

- Легко случайно "подсунуть" лишнее:

```
const weirdUser = {
  name: "Bob",
  age: 30,
  isAdmin: true, // лишнее поле
};

const u: User = weirdUser; //  TypeScript не ругается!
```

➡ TypeScript **не проверяет лишние поля**, если это обычное присваивание.

 Но **если ты создаёшь объект прямо при передаче** — будет ошибка:

```
const u: User = {
  name: "Bob",
  age: 30,
  isAdmin: true, //  Ошибка! Лишнее поле
};
```


Пример из жизни:

```
interface Point2D {  
  x: number;  
  y: number;  
}  
  
function logPoint(p: Point2D) {  
  console.log(`${p.x}, ${p.y}`);  
}  
  
const obj = { x: 1, y: 2, z: 3 };  
  
logPoint(obj); //  Всё работает, потому что x и y есть
```

TypeScript не заботит, что у объекта ещё есть `z`.

Вывод:

Структурная типизация — это:

 Гибко  Просто  Мощно

Но: будь внимателен с лишними свойствами — TypeScript может "пропустить" их.

Как ограничить тип `any`

Тип `any` в TypeScript — это как **выключение всей типовой безопасности**. Полезно временно, но **чем больше `any`, тем больше риска**.

Давай разберёмся, **как ограничить или полностью избежать `any`**.

Почему `any` опасен?

```
let data: any;  
  
data = "hello";  
data.toFixed(); //  Ошибка будет в рантайме, но TypeScript не скажет
```

`any` говорит компилятору: "Я знаю, что делаю, не проверяй меня."

✓ Способы ограничить или запретить `any`

1. **Включи строгий режим**

В `tsconfig.json`:

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

Это включает сразу несколько проверок, включая `noImplicitAny`.

2. **`noImplicitAny: true`** — запрет неявного `any`

```
{
  "compilerOptions": {
    "noImplicitAny": true
  }
}
```

🚫 Это запрещает такие случаи:

```
function greet(name) {
  return "Hello, " + name;
}
// ❌ Ошибка: параметр name не имеет типа
```

✓ Нужно указать:

```
function greet(name: string) {
  return "Hello, " + name;
}
```

3. **`strictFunctionTypes: true`**

Запрещает несовпадающие типы аргументов в функциях. Это тоже помогает избегать `any` в колбэках.

4. **Запретить `any` полностью — через линтер**

Если ты используешь ESLint с TypeScript, можно запретить `any`:

```
// .eslintrc.json
{
  "rules": {
    "@typescript-eslint/no-explicit-any": "error"
  }
}
```

Теперь любой `: any` вызовет ошибку при проверке линтером.

5. **Заменять `any` на более безопасные альтернативы**

Вместо <code>any</code>	Используй
<code>any</code>	<code>unknown</code>
<code>any[]</code>	<code>Array<unknown></code> или <code>unknown[]</code>
<code>function(...): any</code>	Явные типы аргументов и возвращаемого значения

`unknown` — безопасная альтернатива:

```
let value: unknown;

if (typeof value === "string") {
  console.log(value.toUpperCase()); // ✅ безопасно
}
```

`unknown` требует проверки типа перед использованием. Это безопасно, но гибко.

✨ Дополнительно:

- Следи за местами, где TypeScript может подставить `any` сам:
 - неуказанные типы параметров
 - необработанные значения из `JSON.parse()`
 - `catch (err)` блоки — `err` по умолчанию имеет тип `any`

➡ Лучше писать:

```
try {
  // ...
} catch (err: unknown) {
  if (err instanceof Error) {
    console.error(err.message);
  }
}
```

← Вывод:

Чтобы ограничить или запретить `any` :

✅ Включи `strict` и `noImplicitAny` ✅ Используй линтер с правилом `@typescript-eslint/no-explicit-any` ✅ Заменяй `any` на `unknown` или конкретные типы ✅ Будь явным в аннотациях

Как использовать редактор для получения информации о системе типов

TypeScript — это не только язык, но и **умный редакторский опыт**, который помогает тебе **узнавать всё о типах прямо в коде**.

Давай разберём **как использовать редактор (например, VS Code)**, чтобы легко и удобно изучать **типовую информацию**.

🔧 1. ****Наведи мышку на переменную, функцию, тип****

➡ Это покажет **информацию о типе**:

```
const user = { name: "Alice", age: 30 };
```

🖱️ Наведи мышку на `user`, и ты увидишь:

```
const user: {
  name: string;
```



```
age: number;  
}
```

2. ****Cmd (Mac) / Ctrl (Win) + Click**** — перейти к определению

Например:

```
import { Button } from "@/components/ui/button";
```

 Зажми `Ctrl` и кликни на `Button` — редактор откроет его определение, где ты можешь увидеть:

- Исходный интерфейс
- Типы пропсов
- Наследования

3. ****F12**** или ****"Go to Definition"****

То же самое: перейти к определению типа или функции.

4. ****TypeScript: Go to Type Definition****

- `Ctrl + Shift + P` → "Go to Type Definition"
- Это покажет не реализацию, а именно **тип**

5. ****Типовые подсказки при наведении****

Редактор покажет:

- Тип возвращаемого значения
 - Типы аргументов
 - Подробности интерфейса
 - Ошибки типизации (если есть)
-

6. ****Проверка типа через `as const`, `typeof`, `satisfies`****

Редактор может показать точные типы, если использовать:

```
const colors = ["red", "green", "blue"] as const;

type Color = typeof colors[number];
// наведи мышку на Color → покажет: "red" | "green" | "blue"
```

7. ****Проверка вывода типа: `type T = typeof x`****

Иногда удобно ввести:

```
const obj = { x: 1, y: "hi" };

type T = typeof obj;
```

 Наведи на `T`, и редактор покажет, что это:

```
{
  x: number;
  y: string;
}
```

Бонус: ****Inline Type Explorer**** (только в редакторе)

Можешь даже создать такой "типовой sandbox":

```
type MyType = ReturnType<typeof someFunction>;
```

Наведи на `MyType`, и редактор покажет, что именно возвращает `someFunction`.

Что тебе нужно?

Для всего этого достаточно:

- 1. Установить **VS Code**
- 2. Установить **TypeScript (если не встроен)**
- 3. Убедиться, что проект содержит `tsconfig.json`
- 4. Всё — редактор сам "подскажет" тебе всё о типах



Итог: как изучать типы через редактор

Что сделать	Что ты узнаешь
Навести мышку на переменную	Тип этой переменной
Ctrl / Cmd + Клик	Перейти к определению типа
"Go to Type Definition"	Перейти к определению именно типа
Использовать <code>typeof</code> и <code>as const</code>	Вывести точный тип
Посмотреть вывод <code>ReturnType</code>	Что возвращает функция

Типы как множества значений

 **Идея: **тип = множество возможных значений****

В TypeScript **каждый тип можно представить как набор (множество) значений**, которые этому типу "разрешены".



Примеры:

1234

Примитивные типы

```
let a: number;
```

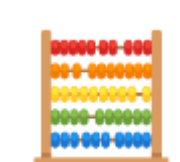
 `number` — это **множество всех возможных чисел**: `...`, `-2`, `-1`, `0`, `1`, `2`, `3.14`, `Infinity`, `NaN` ...


```
let b: boolean;
```

→ `boolean` — это **множество из двух значений**: `{ true, false }`

```
let c: "yes" | "no";
```

→ Это **множество из двух строковых значений**: `{ "yes", "no" }`



Объединения (Union) = объединение множеств

```
type YesNo = "yes" | "no" | "maybe";
```

→ Это **множество из 3 значений**: `{ "yes", "no", "maybe" }`



Пересечения (Intersection) = пересечение множеств

```
type A = { a: number };  
type B = { b: string };  
type AB = A & B;
```

→ `AB` — это **множество объектов**, которые содержат **оба свойства**: `{ a: ..., b: ... }`



Литеральные типы = конкретные значения

```
type Status = "loading" | "success" | "error";
```

Это множество строк **строго ограничено** этими тремя значениями.



Подтипы и расширение

Множества можно сравнивать по включению:


```
type A = "a" | "b";
type B = "a" | "b" | "c"; // B ⊃ A
```

➡ Тип `A` — это **подмножество** `B`. Поэтому значение типа `A` можно присвоить переменной типа `B`, но **не наоборот**.

🤔 Зачем это понимать?

Понимание типов как множеств:

- 💡 Помогает **визуализировать** сложные типы
- 🔒 Помогает **предотвратить ошибки**
- 🧩 Помогает **дизайнить API** (например, `status: "loading" | "done"`)

🧪 Пример с проверкой:

```
type Direction = "left" | "right";

function move(dir: Direction) {
  if (dir === "left" || dir === "right") {
    // Ок
  } else {
    // Никогда не случится — потому что других значений нет
  }
}
```

➡ TypeScript знает: `Direction` — это **множество из 2 значений**, и всё, что вне — ошибка.

✅ Вывод:

Тип	Множество значений	
<code>boolean</code>	<code>{ true, false }</code>	
<code>"yes"</code>	<code>"no"</code>	<code>{ "yes", "no" }</code>
<code>number</code>	Все возможные числа	
<code>never</code>	Пустое множество	
<code>any</code>	Все возможные значения вообще	

unknown	Все значения, но ты ничего о них не знаешь	
---------	--	--

Как выражать принадлежность сущностей к пространству типов или пространству значений

Как выражать **принадлежность сущностей** к:

- 👉 **пространству типов** (типовая область)
- 👉 **пространству значений** (выполняемый код)

📌 Главное различие:

Пространство	Что содержит	Примеры
🧠 Типов	Типы (идеи, формы, интерфейсы)	<code>string</code> , <code>number</code> , <code>User</code>
📄 Значений	Значения (то, что выполняется)	<code>"hello"</code> , <code>42</code> , <code>{name: "A"}</code>

✅ Пространство ****типов****:

Это "мир" описаний, **не существующих во время выполнения**. TypeScript работает с ним **во время компиляции**.

```
type User = { name: string };
```

`User` — это **тип** (принадлежит пространству типов), он не существует в рантайме.

✅ Пространство ****значений****:

Это "мир" реальных значений, **то, что исполняется**.

```
const user = { name: "Alice" };
```


`user` — это **значение**, оно реально в памяти во время выполнения.

🚦 Как выразить принадлежность?

1. **К типам** — через ``:`` или ``extends``

```
let x: string;           // x ∈ string
function fn(a: number)   // a ∈ number
type A = B | C;          // A ⊆ (B ∪ C)
type IsStr<T> = T extends string ? true : false;
```

→ Здесь `:` или `extends` означает **принадлежность типу** или подтипу.

2. **К значениям** — через ``typeof``, ``instanceof``, ``typeof x === "..."``

```
const name = "Bob";      // name ∈ string
const age = 42;          // age ∈ number

typeof name === "string"; // true → name ∈ string
user instanceof UserClass; // true → user ∈ UserClass
```

→ Здесь мы **проверяем принадлежность к пространству значений**.

🎯 Как не путать?

Пример	Пространство	Комментарий	
<code>`type A = string`</code>	<code>number`</code>	Типов	Описание типа, не выполняется
<code>let a: A</code>	Типов + Значений	Значение <code>a</code> , с типом <code>A</code>	
<code>typeof a</code>	Значений	Возвращает <code>"string"</code> в рантайме	
<code>a instanceof B</code>	Значений	Проверка: принадлежит ли объект классу	

T extends U	Типов	Тип <code>T</code> является подмножеством <code>U</code>	
-------------	-------	--	--

Мнемоника:

- `:` — принадлежность к типу
- `extends` — подтип/входит в тип
- `typeof` — узнать **тип значения**
- `instanceof` — принадлежность **объекта** к классу (значение)

Пример из жизни:

```
class Cat {  
  meow() {}  
}  
  
const c = new Cat();  
  
function makeSound(animal: Cat) {  
  animal.meow();  
}
```

- `Cat` — тип и одновременно **конструктор** (принадлежит к обоим мирам)
- `c: Cat` — `c ∈ Cat` по типу
- `c instanceof Cat` — проверка в **мире значений**

Advanced: только в пространстве типов

```
type Keys = keyof User; // Keys ∈ пространство типов
```

Здесь `Keys` — тип, его **не существует** в скомпилированном JS.

Вывод

Как выразить принадлежность

Что это значит

<code>x: T</code>	x принадлежит типу T
<code>T extends U</code>	T — подтип U
<code>typeof x</code>	x принадлежит типу (на уровне значений)
<code>instanceof</code>	x принадлежит классу

Как отдавать предпочтение аннотациям типов, а не утверждениям

В TypeScript есть два способа "сказать" компилятору, какого типа переменная:

 ****Аннотация типов (type annotation)** — правильно**

```
const userName: string = "Alice";
```

Здесь ты **явно указываешь тип**. Это **безопасно**, понятно и проверяется TypeScript.

 ****Утверждение типов (type assertion)** — потенциально опасно**

```
const input = document.getElementById("user") as HTMLInputElement;
```

Ты говоришь: "ДОВЕРЬСЯ мне, это точно HTMLInputElement". TypeScript верит тебе — но **не проверяет**.

Главное отличие:

	Аннотация (✓)	Утверждение (⚠)
<code>const a: string = x</code>	Тип проверяется	
<code>const a = x as string</code>	Тип не проверяется	Можешь "насильно" ошибиться

Почему отдаём предпочтение ****аннотациям****?

✓ 1. ****Аннотации проверяются****

```
const age: number = "30"; // ✗ Ошибка — несовпадение типов
```

⚠ А утверждение может обмануть:

```
const age = "30" as unknown as number; // 😱 Компилятор не жалуется
```

✓ 2. ****Аннотация документирует намерения****

```
function greet(name: string) {  
  console.log("Hi " + name);  
}
```

➡ Любой, кто видит код, сразу понимает: `name` должен быть `string`.

✓ 3. ****Работает с type inference****

TypeScript сам выводит типы, но ты можешь **добавить аннотацию, чтобы усилить проверку**:

```
const user = {  
  name: "Alice",  
  age: 30  
} satisfies { name: string; age: number }; // ✓ проверка
```


🤪 Почему утверждения (``as``) — это костыль?

```
const value = "hello" as number; // ❌ не имеет смысла, но компилятор молчит
```

➡ Ты просишь TypeScript **не проверять** — и он послушно выполняет 🤖

📎 Когда ****можно использовать утверждение (``as``)****?

✅ Только когда ты точно знаешь структуру:

```
const el = document.querySelector("#id") as HTMLDivElement;
```

или:

```
const data = JSON.parse(json) as MyData;
```

➡ Здесь **TypeScript просто не может узнать тип сам**, и ты его подсказываешь.

🏠 Вывод

Лучше делать	А не это
<code>const name: string = "Alice"</code>	<code>const name = "Alice" as string</code>
<code>function fn(x: number)</code>	<code>function fn(x: any) { let y = x as number; }</code>
<code>type-safe annotations</code>	<code>type assertions "на авось"</code>

Как и для чего избегать объектных типов-оберток (`String`, `Number`, `Boolean`, `Symbol`,

BigInt)

Давай разберём что не так с объектными типами-обёртками (`String`, `Number`, `Boolean` и т.д.), почему их лучше избегать и что использовать вместо них.

1. Что такое "объектные обёртки"?

TypeScript (и JavaScript) различает:

Примитив	Объект-обёртка
<code>string</code> (примитив)	<code>String</code> (объект)
<code>number</code>	<code>Number</code>
<code>boolean</code>	<code>Boolean</code>
<code>symbol</code>	<code>Symbol</code>
<code>bigint</code>	<code>BigInt</code>

```
const a: string = "hello";           // примитив
const b: String = new String("hello"); // объект
```

Почему ****НЕ** нужно использовать** обёртки?

1. Это не одно и то же!

```
typeof "hello"           // "string"
typeof new String("hello") // "object"
```

 Они ведут себя **по-разному!**

2. Обёртки ломают строгую типизацию


```
const name: String = new String("Bob");

if (name === "Bob") {
  // ❌ это НЕ выполнится
}
```

➡ Потому что `"Bob"` — примитив, а `new String("Bob")` — объект. Они **не равны** по ссылке.

⚠ 3. Могут привести к странным багам

```
const n: Number = new Number(1);

if (n) {
  // выполняется даже если new Number(0), потому что это объект, а не 0
}
```

➡ Даже `new Number(0)` — **truthy**, потому что это объект. А `0` — falsy. Очень легко ошибиться.

✅ Что использовать вместо них?

Вместо этого 🙅	Используй это 👍
<code>String</code>	<code>string</code>
<code>Number</code>	<code>number</code>
<code>Boolean</code>	<code>boolean</code>
<code>Symbol</code>	<code>symbol</code>
<code>BigInt</code>	<code>bigint</code>

✅ Как писать правильно в TypeScript:

```
let name: string = "Alice";
let age: number = 30;
let isAdmin: boolean = false;
```



```
let id: symbol = Symbol("id");
let big: bigint = 1000n;
```

➡ Это **безопасные примитивные типы**, они работают как ожидается.

📌 Исключение: типы ``Object``, ``Function`` и ``{}``

Они тоже часто **нежелательны** как аннотации. Лучше их избегать или уточнять.

🔍 Пример ошибки:

```
function greet(name: String) {
  console.log("Hello, " + name);
}

greet("Alice"); // ✅ работает
greet(new String("Alice")); // тоже работает, но:
```

- TypeScript не ругнётся
- Но внутри может быть неожиданное поведение при сравнении, сериализации и т.д.

✅ Вывод

Почему избегать обёрток	Что использовать вместо них
Поведение отличается от примитивов	Примитивные типы: <code>string</code> , <code>number</code> и т.д.
Может привести к багам	Простые, предсказуемые типы
Не нужна "лишняя обёртка"	У TypeScript и так есть аннотации

Если хочешь — могу показать, **в каких случаях TypeScript автоматически оборачивает примитив**, например:

```
"hi".toUpperCase(); // работает — TS делает временную обёртку
```

Но ты сам их явно писать **не должен**.



Система типов (11-30)

Проверка лишних свойств

В TypeScript **проверка лишних свойств** (excess property checks) — это **особое правило**, отличное от обычной проверки типов.



Главное различие:

Проверка	Где происходит	Что проверяет
✅ Проверка типов	Всегда при присваивании	Совместимость типов
⚠️ Проверка лишних свойств	Только при литеральных объектах	Запрещает лишние ключи , не описанные в типе



Пример — ****типовая проверка**** (всё нормально)

```
type User = { name: string };

const u1 = { name: "Alice", age: 30 }; // age — лишний

const u2: User = u1; // ✅ ОК — тип совместим (age игнорируется)
```

➡ Почему ОК? Потому что `u1` — **переменная**: **лишние свойства игнорируются**.



Пример — ****проверка лишних свойств****

```
type User = { name: string };

const u: User = { name: "Alice", age: 30 }; // ❌ Ошибка!
```

➡ **!** Здесь объект создаётся **прямо на месте**, и TypeScript говорит:

“У тебя свойство `age`, а его нет в типе `User`.”

Правило:

Лишние свойства проверяются только у объектных литералов, присваиваемых напрямую.

Как обойти (если нужно):

1. Вынести в переменную

```
const temp = { name: "Alice", age: 30 };
const u: User = temp; //  ОК — не проверяется лишнее
```

2. Использовать оператор утверждения:

```
const u: User = {
  name: "Alice",
  age: 30
} as User; //  подавляет проверку, но может скрыть баг
```

3. Указать "дополнительные свойства" в типе:

```
type User = {
  name: string;
  [key: string]: any; // разрешаем любые другие свойства
};
```

Почему TypeScript так делает?

TypeScript специально **строже проверяет объектные литералы**, потому что они, как правило:

- создаются вручную
- могут содержать опечатки (`agge` вместо `age`)

Поэтому компилятор защищает тебя от ошибок.

Мини-сводка:

```
type Config = { host: string };

// ❌ ошибка — лишнее свойство
const cfg: Config = { host: "localhost", port: 3000 };

// ✅ нормально — переменная, не объект напрямую
const tempCfg = { host: "localhost", port: 3000 };
const cfg2: Config = tempCfg;
```

Вывод:

Проверка	Пример	Проверяется ли extraProp?
Прямой литерал	<code>{ a: 1, b: 2 }</code>	✅ Да
Через переменную	<code>const obj = { a: 1, b: 2 };</code>	❌ Нет
С <code>as Type</code>	<code>{ a: 1, b: 2 } as MyType</code>	❌ Нет

Как задать тип всему функциональному выражению

Давай разберём **как задать тип всему функциональному выражению**, а не только его частям. 📌 Это особенно важно при передаче функций как значений.

✅ **Цель: ****типизировать всю функцию целиком****, а не только параметры или ``return``.**

Способы применять типы ко всему функциональному выражению:

✓ 1. ****Через аннотацию переменной****

```
const greet: (name: string) => string = (name) => {  
  return "Hello, " + name;  
};
```

🟢 Здесь тип `(name: string) => string` применяется ко всей функции.

✓ Это **тип функции**, и он применяется к **функциональному выражению**, не к параметрам внутри.

✓ 2. ****Через `type` или `interface`****

```
type Greeter = (name: string) => string;  
  
const greet: Greeter = (name) => "Hi, " + name;
```

➡ Полезно при повторном использовании типа для нескольких функций.

✓ 3. ****Анонимные функции с аннотацией****

При передаче в другие функции:

```
setTimeout(() => console.log("Hi")) as () => void, 1000);
```

⚠ Но лучше объявить переменную с типом — это читаемее.

✓ 4. ****Стрелочная функция в типе функции высшего порядка****


```
function withLogging(fn: (x: number) => number): (x: number) => number {
  return (x) => {
    console.log("Calling with", x);
    return fn(x);
  };
}
```

→ `fn` и возвращаемая функция — **полностью типизированы как выражения**.

⚠ Антипаттерн: тип только у параметров

```
const fn = (x: number) => {
  return x.toString(); // ⚠ Тип возвращаемого значения не указан
};
```

→ Здесь TypeScript **выведет** тип `string`, но иногда **неправильно**, особенно в сложных функциях.

✓ Лучше так:

```
const fn: (x: number) => string = (x) => x.toString();
```

📝 Советы:

Что делать	Зачем
✓ Типизируй через <code>const fn: Type = ...</code>	Явно задаёт всю сигнатуру функции
✓ Используй <code>type</code> или <code>interface</code> для групп	Повторное использование сигнатур
✗ Не полагайся полностью на вывод	Могут появиться ошибки в <code>return</code> , <code>any</code>
✓ Типизируй аргументы и <code>return</code> одновременно	Повышает читаемость и безопасность

✓ Пример: полная типизация


```
type Transformer = (input: string) => number;

const lengthOfTrimmed: Transformer = (input) => input.trim().length;
```

→ Всё выражение соответствует типу `Transformer`.

Различия между `type` и `interface`

♦ Оба `interface` и `type` делают почти одно и то же:

Они создают **свои собственные типы данных**.

Например:

```
interface Person {
  name: string;
  age: number;
}

type Person = {
  name: string;
  age: number;
};
```

Это одно и то же! И `interface`, и `type` создают "чертёж" для объекта.

🔑 Но в чём же разница?

1. ****Расширение (наследование)****

✓ `interface` легко ****дополняется и объединяется****

```
interface A {
  name: string;
}

interface A {
  age: number;
}
```



```
// Теперь A = { name: string; age: number }
```

➡ Это удобно в больших проектах или когда работаешь с библиотеками.

❌ **`type` так ***не умеет*****

```
type A = {  
  name: string;  
};  
  
type A = {  
  age: number;  
}; // ❌ Ошибка — нельзя объявить дважды
```

2. *****Композиция (соединение)*****

`type` — мощнее в комбинировании

```
type A = { name: string };  
type B = { age: number };  
type C = A & B; // { name: string; age: number }
```

➡ Это как "склеить" типы. Удобно!

3. *****Классы*****

Если ты пишешь `class`, то `interface` — твой лучший друг:

```
interface Animal {  
  speak(): void;  
}  
  
class Dog implements Animal {  
  speak() {  
    console.log("Woof");  
  }  
}
```

4. *****Типы не только объектов*****

`type` может описывать **всё, что угодно**, не только объекты:

```
type ID = number | string;
type Callback = () => void;
```

→ `interface` так **не умеет**, он только для объектов и классов.

🧠 Как запомнить:

Если хочешь...	Используй	
Описать объект или класс	interface	
Объединить типы, объединять `	<code>&`</code>	type
Быстро и просто описать что угодно	type	
Использовать в библиотеке, расширять	interface	

Readonly в TypeScript

`readonly` в TypeScript помогает сделать данные **неизменяемыми**, и это мощный инструмент для **предотвращения багов**, особенно в больших проектах.

🔒 Что делает `readonly`?

Добавляя `readonly` к полям или массивам, ты говоришь TypeScript:

“Это значение нельзя изменить после инициализации.”

🧱 Пример: `readonly` в объектах

```
type User = {
  readonly id: number;
  name: string;
};
```



```
const user: User = { id: 1, name: "Alice" };
```

```
user.name = "Bob";    // ✅ можно  
user.id = 2;          // ❌ Ошибка: id — только для чтения
```

➡ Это защищает `id` от случайных изменений.

🧩 Пример: `readonly` для массивов

```
const nums: readonly number[] = [1, 2, 3];
```

```
nums.push(4);          // ❌ Ошибка  
nums[0] = 10;          // ❌ Ошибка
```

Это делает массив **immutable** (нельзя изменить структуру массива).

🧰 Где `readonly` особенно полезен

✅ 1. Константные данные

```
const CONFIG: Readonly<Config> = {  
  apiUrl: "https://...",  
  retryCount: 3  
};
```

➡ Никто не сможет случайно переписать `CONFIG.retryCount = 4`.

✅ 2. Передача аргументов в функции

```
function logUser(user: Readonly<User>) {  
  console.log(user.name);  
  // user.name = "Bob"; // ❌ нельзя изменить  
}
```

➡ TypeScript защитит от ошибок внутри функции.

✓ 3. В Redux, Zustand и других хранилищах

Мутация состояния — частый источник багов. `readonly` заставляет писать **чистые, иммутабельные обновления**.

Readonly утилиты

TypeScript предоставляет встроенные утилиты:

Что	Описание
<code>readonly</code> перед типом поля	Применяется к отдельным свойствам
<code>Readonly<T></code>	Делает все поля типа <code>T</code> только для чтения
<code>ReadonlyArray<T></code>	То же, что <code>readonly T[]</code>

```
type Point = Readonly<{
  x: number;
  y: number;
}>;
```

Почему это важно?

Без `readonly` :

```
function update(p: Point) {
  p.x = 10; // 🐛 возможно, нежелательное поведение
}
```

С `readonly` :

```
function update(p: Readonly<Point>) {
  p.x = 10; // ❌ Ошибка
}
```

➡ Это предотвращает **скрытые мутации**, которые приводят к багам.

 Вывод

Преимущество readonly	Что это даёт
Защищает данные от мутаций	Нет случайных изменений
Повышает надёжность функций	Нет побочных эффектов
Улучшает совместимость с иммутабельной архитектурой (Redux, React, etc.)	Легче отлаживать и тестировать

Сравнение с `const` и `Object.freeze()`

`readonly` , `const` и `Object.freeze()` **все связаны с идеей неизменяемости**, но работают в **разных измерениях**: на уровне **типов** (TypeScript) или **во время выполнения** (JavaScript). Давай разберёмся.

 Кратко: как они работают

Конструкция	Где работает	Что делает
<code>const</code>	JavaScript + TypeScript	Запрещает переприсваивание переменной
<code>readonly</code>	Только TypeScript	Запрещает изменение свойств объекта или массива
<code>Object.freeze()</code>	Только JavaScript	Делает объект глубоко неизменяемым (на рантайме)

 Подробнее:

 1. `const`

```
const user = { name: "Alice" };

user = { name: "Bob" }; // ❌ Ошибка: нельзя переприсвоить
user.name = "Bob";      // ✅ Разрешено — объект всё ещё изменяем
```

 `const` защищает **переменную**, но не **её содержимое**.

✓ 2. `readonly` (TS)

```
type User = {  
  readonly name: string;  
};  
  
const user: User = { name: "Alice" };  
  
user.name = "Bob"; // ✗ Ошибка: name – readonly
```

📌 `readonly` защищает **структуру объекта** (на этапе компиляции), но **не мешает изменению в рантайме**, если обойти проверки.

✓ 3. `Object.freeze()` (JS)

```
const user = Object.freeze({ name: "Alice" });  
  
user.name = "Bob"; // ✗ Ошибка в strict mode (в рантайме)
```

📌 `freeze()` делает объект **иммутабельным во время выполнения**. Но TypeScript всё равно считает его обычным объектом, если ты не добавишь тип `Readonly`.

🔄 Как их сочетать

✓ Самое безопасное — вместе:

```
const user: Readonly<{ name: string }> = Object.freeze({ name: "Alice" });
```

➡ Это:

- `const` → нельзя переопределить
- `Readonly` → нельзя изменить свойства на уровне типов
- `freeze()` → нельзя изменить в рантайме

! В чём подвох?

1. `readonly` — **не работает во время выполнения**

```
const user: Readonly<{ name: string }> = { name: "Alice" };
(user as any).name = "Bob"; // ! это сработает в рантайме!
```

→ TypeScript защищает только **на этапе компиляции**.

2. `Object.freeze()` — **не гарантирует типовую защиту**

```
const user = Object.freeze({ name: "Alice" });
user.name = "Bob"; // ❌ Ошибка в рантайме
// но TypeScript не жалуется, если не указан Readonly тип
```

📌 Итоговая таблица сравнения

Характеристика	const	readonly (TS)	Object.freeze() (JS)
Предотвращает переприсваивание	✓	❌	❌
Предотвращает изменение полей	❌	✓ (на этапе типов)	✓ (в рантайме)
Работает в JS	✓	❌ (удаляется при компиляции)	✓
Проверка на этапе компиляции	✓	✓	❌

💡 Рекомендация

Если ты хочешь **настоящую защиту от изменений**:

```
const config: Readonly<Config> = Object.freeze({
  apiUrl: "https://example.com",
```



```
    retries: 3,  
  });
```

🔒 Это даст и **типовую защиту**, и **защиту в рантайме**.

Как использовать операции типов

В TypeScript ты можешь использовать **операции над типами** (type operations) и **обобщения (дженерики)**, чтобы **не дублировать** одни и те же типы в разных местах. Это помогает:

- 📐 уменьшить повтор кода
- 🧠 повысить читаемость и поддержку
- ✅ избежать расхождений между типами

🔧 Что такое операции над типами?

Это способы **строить новые типы из уже существующих** с помощью встроенных инструментов TypeScript:

Операция	Что делает
Pick<T, K>	Берёт из <code>T</code> только указанные ключи <code>K</code>
Omit<T, K>	Удаляет из <code>T</code> указанные ключи <code>K</code>
Partial<T>	Делает все поля опциональными
Required<T>	Делает все поля обязательными
Readonly<T>	Делает все поля <code>readonly</code>
ReturnType<T>	Получает тип возвращаемого значения функции
Parameters<T>	Получает кортеж типов параметров функции

📦 Пример: избежать повторов с `Pick`

```
type User = {  
  id: number;  
  name: string;  
  email: string;
```



```
    role: string;
};

// Тип для формы регистрации:
type RegisterForm = Pick<User, "name" | "email">;
```

🔄 Вместо того чтобы вручную повторять `{ name: string; email: string }`, ты просто **вытаскиваешь** нужные поля.

🖌️ Пример: `Omit` для исключения

```
type UserWithoutId = Omit<User, "id">;
```

➡ Удобно при создании новых сущностей, где `id` будет сгенерирован автоматически.

🧩 Пример: `Partial` и `Required`

```
type UpdateUser = Partial<User>; // Все поля опциональны
type CompleteUser = Required<User>; // Все поля обязательны
```

📖 Обобщения (дженерики) для повторного использования

```
function getProperty<T, K extends keyof T>(obj: T, key: K): T[K] {
    return obj[key];
}

const user = { id: 1, name: "Alice" };
const name = getProperty(user, "name"); // Тип — string
```

✅ Обобщения позволяют **написать одну функцию**, которая работает с разными типами, **сохраняя типовую безопасность**.

🔗 Комбинирование: обобщения + операции над типами

```
function updateEntity<T, K extends keyof T>(
  entity: T,
  updates: Pick<T, K>
): T {
  return { ...entity, ...updates };
}

const user = { id: 1, name: "Alice", role: "admin" };

const updated = updateEntity(user, { name: "Bob" }); // ✅ тип проверяется
```

➡ Здесь `updates` может содержать **только те поля, которые действительно есть в `user`**.

📌 Вывод

Приём	Для чего использовать
<code>Pick</code> , <code>Omit</code>	Выбрать или убрать поля из типа
<code>Partial</code> , <code>Required</code>	Сделать поля опциональными или обязательными
<code>Readonly</code>	Защитить поля от изменений
Обобщения (<code><T></code>)	Универсальные функции и типы
Комбинации операций	Гибкое построение новых типов из существующих

🎓 Пример для вдохновения

```
type APIResponse<T> = {
  status: number;
  data: T;
};

type User = { id: number; name: string };

const response: APIResponse<User> = {
  status: 200,
```



```
data: { id: 1, name: "Alice" }  
};
```

→ Ты описал `APIResponse` один раз, и можешь применять его к любому типу данных.

Используйте более точные альтернативы для сигнатур индексов

В TypeScript **индексные сигнатуры** (`[key: string]: any`) дают гибкость, но могут быть **слишком расплывчатыми**. Есть **более точные и безопасные альтернативы**, которые позволяют сохранить гибкость, но не терять точность типов.

❌ Проблема с «широкой» индексной сигнатурой

```
type LooseObject = {  
  [key: string]: any;  
};
```

● Проблемы:

-  любой ключ допустим — даже неожиданные
-  теряется информация о **конкретных свойствах**
-  редактор не подсказывает типы значений

Альтернатива 1: Перечисление известных ключей

```
type User = {  
  name: string;  
  age: number;  
};
```

→ Гораздо точнее и безопаснее. Редактор покажет подсказки, ошибки, и автокомплит.

✓ Альтернатива 2: Использовать `Record`

```
type UserMap = Record<string, User>;
```

➡ Это чётко указывает: у объекта ключи — строки, а значения — `User`.

Можно сузить тип ключей:

```
type Roles = "admin" | "user" | "guest";
type RoleConfig = Record<Roles, number>;
```

➡ Только `"admin" | "user" | "guest"` допустимы как ключи.

✓ Альтернатива 3: Использовать объект с шаблонными ключами

```
type EventHandlers = {
  [K in `on${string}`]: () => void;
};
```

➡ Это позволяет указать **форму ключей** (`onClick`, `onHover` и т.д.), но при этом значения строго типизированы.

✓ Альтернатива 4: Типы через Map/Set

Если ключи не строки, а, например, `number | symbol`, или тебе важен порядок:

```
const map = new Map<string, User>();
```

➡ Такой способ может быть точнее и удобнее, чем обычный объект.

⚖ Сравнение подходов

Подход	Пример	Гибкость	Точность	Интеллект
--------	--------	----------	----------	-----------

Индексная сигнатура	<code>[key: string]: any</code>	 Высокая	 Низкая	 Нет
Перечисление свойств	<code>{ name: string; age: number }</code>	 Меньше	 Высокая	 Отлично
<code>Record<K, V></code>	<code>Record<string, User></code>	 Средняя	 Высокая	
Маппинг ключей (<code>[K in ...]</code>)	<code>[K in Keys]: T</code>	 Средняя	 Высокая	

Вывод

 Вместо:

```
type Bad = { [key: string]: any };
```

 Лучше:

```
type UserMap = Record<string, User>;
```

или

```
type Specific = {
  name: string;
  age: number;
};
```

Избегайте использования числовых сигнатур индексов

Числовые индексные сигнатуры (`[index: number]: T`) в TypeScript **выглядят удобно**, но их **использование часто опасно** или **избыточно**, особенно когда есть более точные способы описать массивоподобные структуры.

🧱 Что такое числовая сигнатура индекса?

```
type MyArray = {  
  [index: number]: string;  
};
```

📌 Это говорит TypeScript: объект **может быть доступен по числовому индексу**, и значение всегда `string`.

Но в TypeScript (и JavaScript) ключи ВСЕГДА строки, и `number` превращается в `string`.

👉 Поэтому `[0] === ["0"]` — и это может вести к **неявным багам**.

❌ Почему ****не стоит**** использовать `[index: number]`?

1. 🧱 Не гарантирует, что это настоящий массив

```
const obj: { [index: number]: string } = { 0: "a", 1: "b" }; // Это не Array!  
obj.push("c"); // ❌ Ошибка — у объекта нет методов массива
```

2. 🧠 Сбивает с толку: ожидание «настоящего массива», а приходит обычный объект
3. 🧪 Нет информации о длине, методах, итерации

✅ Альтернатива 1: Использовать `string[]` или `Array`

```
const names: string[] = ["Alice", "Bob"];
```

➡ Это **настоящий массив** — с типами, методами (`map`, `filter`, `length`) и автокомплитом.

✅ Альтернатива 2: Кортежи — для фиксированной длины


```
type Point2D = [number, number];

const point: Point2D = [10, 20];
```

➡ Кортеж — это массив с **известным числом элементов и их типами по позициям**.

✅ Альтернатива 3: Объекты с ключами-числами (если нужны)

Если тебе нужно что-то вроде словаря с числовыми ключами:

```
type NumericKeyed = Record<number, string>;
```

⚠ Но помни, что ключи в JavaScript **всё равно будут превращены в строки**.

🧠 **Совет: если нужна структура ****массивоподобная****, используй массив (``Array``), а не ``[index: number]: T``**

📌 Итого

Нужно описать...	Используй
Обычный массив	<code>T[]</code> или <code>Array<T></code>
Фиксированное количество элементов	Кортеж: <code>[T1, T2, T3]</code>
Объект с числовыми ключами (с осторожностью)	<code>Record<number, T></code>
Не надо	<code>{ [index: number]: T }</code> — избегай!

Не засорять код лишними аннотациями типов

В TypeScript очень важно **не перегружать код избыточными аннотациями типов**, потому что:

- 📐 это делает код длиннее и труднее читаемым;
- 🧠 часто TypeScript и так может **вывести типы сам**;
- ✂ IDE (VS Code и др.) дают подсказки даже без явных аннотаций.

🧠 Правило: ****Аннотируй там, где это нужно****, а не везде

✅ Где ****не нужно**** писать аннотацию — TypeScript сам справится:

Присваивание переменной

```
const name = "Alice"; // TypeScript выводит: string
```

❌ Не нужно писать `const name: string = "Alice"` — это лишнее.

Возвращаемое значение из ``map``, ``filter``, ``reduce`` и т.п.

```
const numbers = [1, 2, 3];  
const doubled = numbers.map(n => n * 2); // number[]
```

❌ Не надо писать `map((n: number): number => ...)`

Функции с очевидным типом возвращаемого значения

```
function greet(name: string) {  
  return `Hello, ${name}`;  
}  
// Тип возвращаемого значения — string (выводится)
```

➡ Не нужно вручную писать `: string` — TypeScript это знает.

✅ Где ****стоит**** писать аннотации

1. Параметры функций (особенно публичных)

```
function greet(name: string) {  
  return `Hi, ${name}`;  
}
```

2. Типы возвращаемых значений в API или библиотеке

Если ты пишешь **публичный API**, лучше писать `: ReturnType` .

```
function getUser(): User { ... }
```

3. Обобщённые функции

```
function identity<T>(value: T): T {  
  return value;  
}
```

4. Сложные объекты

```
const user: User = { id: 1, name: "Alice" };
```

5. Типы параметров в колбэках, если они не выводятся

```
myLib.onEvent((e: CustomEvent) => {  
  // тут нужна аннотация, иначе e: any  
});
```

❌ Пример кода с лишними аннотациями (плохо)

```
const a: number = 5;  
const b: string = "hello";  
const nums: number[] = [1, 2, 3];  
  
function add(a: number, b: number): number {  
  return a + b;  
}
```


✓ Лучше так:

```
const a = 5;
const b = "hello";
const nums = [1, 2, 3];

function add(a: number, b: number) {
  return a + b;
}
```

Совет

- Аннотируй **входы и выходы функций**.
- В остальных местах — **положись на вывод типов** (`type inference`).
- Используй `as const`, `satisfies`, `typeof`, когда нужна точность без явной аннотации.

Используйте разные переменные для разных типов

Почему и как использовать разные переменные для разных типов в TypeScript.

В чём идея?

Не пихай в одну переменную значения разных типов. Вместо этого — **используй отдельные переменные для каждого типа данных**.

Плохой пример — всё в одной переменной

```
let data: string | number;

data = "user123";
data = 42;
```

Здесь переменная `data` используется и как строка, и как число. Это:

-  делает код менее понятным;
-  требует постоянных проверок типа (`typeof data === "string"` и т. д.);

-  приводит к ошибкам и путанице в логике.

✓ Хороший пример — отдельные переменные

```
const userId: string = "user123";
const userAge: number = 42;
```

Теперь:

- ✓ Каждая переменная описывает одно значение;
- ✓ Редактор подскажет точные типы;
- ✓ Минимум ошибок при работе с переменными.

💡 Где это особенно важно?

1. **Обработка разных данных**

```
const rawData: string = '{"name":"Alice"}';

let parsedData: { name: string } | null = null;

try {
  parsedData = JSON.parse(rawData);
} catch {
  // parsedData остаётся null
}
```

✓ Теперь `rawData` всегда `string`, `parsedData` всегда объект или `null`.

2. **Работа с результатами API**

```
const responseText: string = await res.text();

let json: MyType | null = null;
try {
  json = JSON.parse(responseText);
} catch {}
```


Почему не стоит делать так:

```
let result: string | Error;
```

Ты потом должен писать:

```
if (typeof result === "string") {  
  // ...  
} else {  
  // ...  
}
```

А можно просто:

```
const data: string = "ok";  
const error: Error | null = null;
```

Резюме

Вместо	Лучше	
<code>let x: string</code>	<code>number</code>	<code>const id: string</code> , <code>const age: number</code>
<code>let result: boolean</code>	<code>Error</code>	<code>const isValid: boolean</code> , <code>const error: Error</code>
<code>let value: any</code>	Используй конкретные типы и переменные	

 **Правило:** одна переменная — один тип, одна ответственность.

Что такое расширение типов в TypeScript

что такое расширение типов в TypeScript и зачем это нужно.



Что такое "расширение типов"?

Расширение типа — это создание нового типа **на основе другого**, с добавлением или уточнением свойств.

Похожее на **наследование** в объектно-ориентированном программировании.



Пример: расширяем `User`

```
type User = {  
  id: number;  
  name: string;  
};  
  
type Admin = User & {  
  role: "admin";  
};
```



Admin теперь содержит:

```
{  
  id: number;  
  name: string;  
  role: "admin";  
}
```



Два способа расширять типы

1. Через `extends` в интерфейсах

```
interface User {  
  id: number;  
  name: string;  
}  
  
interface Admin extends User {  
  role: "admin";  
}
```



Admin будет иметь все свойства User + role

2. Через `&` (intersections) в `type`

```
type Admin = User & { role: "admin" };
```

Оба способа работают, но `interface` чаще используется, когда ты строишь **иерархию объектов**, а `type` — когда комбинируешь типы (в т.ч. с `union` и utility-типами).



Расширение с изменением типа

Допустим, ты хочешь **заменить** свойство в типе:

```
type User = {
  id: number;
  name: string;
};

type UserWithAge = Omit<User, "name"> & {
  name: string;
  age: number;
};
```

➡ Здесь мы:

1. убрали `name`,
2. добавили новый `name` (например, уточнённый тип),
3. добавили `age`.



Когда это нужно?

- 🔄 **Переиспользование** структуры (DRY)
- 🗝️ Уточнение: `BaseUser` → `AuthenticatedUser` → `Admin`
- 🔄 Создание вариаций: `DraftPost`, `PublishedPost`
- 🛠️ Комбинирование типов (`Record`, `Partial`, `Pick` и т.п.)



Важно помнить

- `type` нельзя расширять `extends`, только `interface` может `extends interface`.
- `interface` можно расширять сколько угодно раз.

- `type` удобнее, когда нужно использовать utility-типы (`Partial` , `Omit` и т.п.).
- `interface` — для описания shape объектов, особенно в библиотеках и классах.

Резюме

Что делаешь	Используй	Пример
Добавить свойства к типу	<code>&</code>	<code>type B = A & { extra }</code>
Расширить интерфейс	<code>extends</code>	<code>interface B extends A { }</code>
Удалить/заменить поле	<code>Omit</code>	<code>Omit<User, "name"> & { name: NewType }</code>

Почему лучше создавать объекты целиком

Что значит "создавать объект целиком"?

Это означает: **создавай объект сразу со всеми его полями**, а не по частям или с пропущенными значениями.

Плохо — создаём объект по частям

```
type User = {
  id: number;
  name: string;
};

let user = {} as User;
user.id = 1;
user.name = "Alice";
```

Почему это плохо:

- TypeScript **не проверит** правильно, есть ли все поля.

- **Возможны ошибки во время выполнения**, например если забыли поле.
- В **рефакторинге** легко что-то пропустить.

✅ Хорошо — создать объект сразу целиком

```
const user: User = {  
  id: 1,  
  name: "Alice",  
};
```

👍 Почему это хорошо:

- TypeScript **сразу проверяет все свойства**.
- Код чище и безопаснее.
- Автокомплит подсказывает нужные поля.
- Ошибка компиляции, если поле пропущено или лишнее.

🧠 Зачем это правило?

1. ✅ **Более надёжная проверка типов**
2. ✅ **Легче читать и понимать структуру**
3. ✅ **Меньше шансов на забытые поля**
4. ✅ **Помогает избежать "type assertion" (`as`) злоупотреблений**

🛡️ А что если объект создаётся динамически?

Если ты **создаёшь объект из переменных**, то всё равно лучше собрать его **в один шаг**, а не присваивать по одному полю.

```
const id = 1;  
const name = "Alice";  
  
const user: User = { id, name };
```

💡 Совет: Не используй `as Type` для "достройки" объекта


```
const user = {} as User;
user.id = 1;
user.name = "Alice";
```

🔴 Это **обходит проверку типов**, что может привести к ошибкам.

✅ Можно использовать вспомогательные функции

```
function makeUser(id: number, name: string): User {
  return { id, name };
}

const user = makeUser(1, "Alice");
```

🔬 Проверка лишних свойств тоже работает только при ****целиком созданном объекте****

```
const user: User = {
  id: 1,
  name: "Alice",
  extra: true, // ❌ Ошибка: лишнее свойство!
};
```

📌 Вывод

Создавай объекты сразу и целиком — это:

- Надёжно,
- Проверяемо,
- Поддерживаемо,
- И безопасно при обновлении кода.

Сужение типов

Сужение типов (type narrowing) — это один из ключевых инструментов TypeScript, который позволяет "умному" коду понимать, **с каким конкретным типом он сейчас работает**, если у тебя есть несколько возможных типов.

Что такое сужение типов?

Сужение типов — это процесс, когда TypeScript **понимает более конкретный тип переменной**, основываясь на проверках (например, `typeof`, `in`, `=== null`, и т. д.).

Пример: объединённый тип

```
function printId(id: string | number) {
  if (typeof id === "string") {
    console.log(id.toUpperCase()); // ✅ id: string
  } else {
    console.log(id.toFixed(2)); // ✅ id: number
  }
}
```

 Здесь TypeScript **сузил тип** `id` до `string` или `number` в каждом блоке.

Способы сужения типов

1. `typeof` — для примитивов

```
function handle(value: string | number | boolean) {
  if (typeof value === "string") {
    // value: string
  } else if (typeof value === "boolean") {
    // value: boolean
  } else {
    // value: number
  }
}
```


2. ****`in` — для объектов с разными полями****

```
type Cat = { meow: () => void };
type Dog = { bark: () => void };

function makeSound(pet: Cat | Dog) {
  if ("meow" in pet) {
    pet.meow(); // ✅ pet: Cat
  } else {
    pet.bark(); // ✅ pet: Dog
  }
}
```

3. ****Сравнение с `null`, `undefined`****

```
function greet(name?: string) {
  if (name !== undefined) {
    console.log("Hello " + name.toUpperCase());
  }
}
```

4. ****`===` / `!==` — для литералов****

```
type Status = "success" | "error";

function show(status: Status) {
  if (status === "success") {
    // status: "success"
  } else {
    // status: "error"
  }
}
```

5. ****Проверка через пользовательский `type guard`****

```
type Fish = { swim: () => void };
type Bird = { fly: () => void };

function isFish(pet: Fish | Bird): pet is Fish {
  return "swim" in pet;
}

function move(pet: Fish | Bird) {
```



```
if (isFish(pet)) {  
  pet.swim(); // ✅ pet: Fish  
} else {  
  pet.fly(); // ✅ pet: Bird  
}  
}
```

⚠️ Без сужения — плохо

```
function handle(id: string | number) {  
  // id.toUpperCase(); // ❌ Ошибка!  
}
```

🧠 Как освоить:

- 📖 Понимай объединённые типы (`|`) и их ограничения.
- 🔧 Упражняйся писать функции, которые принимают `string | number | boolean`, `A | B`, и т. д.
- 🔍 Используй `typeof`, `in`, `===`, `!==`, `instanceof` — и **наблюдай**, как меняется тип в IDE.
- 🔧 Пиши `type guard`-функции (`pet is Dog`) — это мощно!
- 💡 Всегда проверяй переменные перед их использованием, если тип не однозначен.

✅ Сужение = без `any`, без кастов (`as`), без ошибок

TypeScript становится максимально полезным, **когда ты позволяешь ему сузить тип**, а не глушишь проверку.

Как быть последовательным с использованием псевдонимов

Быть последовательным с использованием псевдонимов типов (type aliases) в TypeScript — значит соблюдать ясность, единообразие и читаемость при описании и применении типов в проекте.



Что такое псевдонимы типов?

```
type UserId = string;
type Age = number;
type User = {
  id: UserId;
  age: Age;
};
```

Здесь `UserId` и `Age` — **псевдонимы типов** (type aliases). Это имена, которые ты даёшь существующим типам — примитивам, объектам, объединениям и т.д.



Как быть последовательным — простые правила

1. ****Если начал использовать псевдонимы — используй их везде****

Плохо:

```
type UserId = string;

type User = {
  id: string; // ❌ а где UserId?
};
```

Хорошо:

```
type User = {
  id: UserId; // ✅ понятно, зачем этот тип
};
```

2. ****Даём имена, отражающие смысл****

```
type Email = string;
type IsoDateString = string;
type PriceCents = number;
```


♦ Это делает код **самодокументируемым**. ♦ Даже если в рантайме это просто `string`, IDE показывает нам смысл.

3. ****Избегаем избыточности****

Плохо:

```
type StringAlias = string;
```

❶ Такой псевдоним ничего не даёт. Не создавай типы просто ради типов.

4. ****Отделяй alias от интерфейсов****

Используй `type` для комбинаций, utility-типов, литеральных объединений и т.д.

Используй `interface`, если описываешь структуру объекта, особенно для API, классов или компонентов.

```
interface User {  
  id: UserId;  
  email: Email;  
}
```

5. ****Пиши в одном стиле: либо `type`, либо `interface`, не мешай без причины****

Плохо:

```
type User = { id: string };  
interface Admin extends User {} // ❌ нельзя extends type
```

Хорошо:

```
interface User { id: string }  
interface Admin extends User { role: string }
```

или


```
type User = { id: string };
type Admin = User & { role: string };
```

6. ****Разделяй области: отдельные файлы/модули для типов****

Например:

- types/user.ts
- types/common.ts
- types/api.ts

 Резюме: как быть последовательным

Принцип	Почему важно
✓ Используй псевдонимы осознанно	Чтобы код был читаемым и самодокументированным
✓ Не дублируй string , number без смысла	Это только засоряет код
✓ Однотипный стиль (type или interface)	Избегаешь конфликтов и ошибок
✓ Повторно используй типы	DRY: не дублируй структуру вручную
✓ Пиши понятные имена	Помогает понять, что за данные в типе

Как использовать контекст при выводе типов

 Что такое контекстный вывод типов?

Контекстный вывод — это когда TypeScript автоматически определяет тип переменной или параметра, глядя на окружение, в котором она используется.

Проще говоря: **ты не указываешь тип явно, но TypeScript всё равно его "понимает" благодаря контексту.**

Примеры

✓ Пример 1. Callback в `.map`

```
const names = ["Alice", "Bob", "Charlie"];

const lengths = names.map(name => name.length);
```

Здесь ты **не указываешь**, что `name: string`, но TypeScript **понимает это**, потому что `names: string[]`.

✓ Пример 2. Обработчик события

```
document.addEventListener("click", event => {
  event.preventDefault(); // event: MouseEvent
});
```

TypeScript знает, что `"click" → MouseEvent`, и **сам выводит тип** параметра.

✓ Пример 3. Функции как параметры

```
function withUser(fn: (user: { id: number }) => void) {
  const user = { id: 123 };
  fn(user);
}

withUser(user => {
  console.log(user.id); // user: { id: number }
});
```

Ты не пишешь тип `user`, но он **выводится из параметра** `fn`.

Пример, где контекста нет → нужен явный тип


```
const data = JSON.parse(json); // data: any ❌
```

🔴 Здесь TypeScript **не знает**, что в `data`. Нет контекста.

✅ Лучше явно подсказать:

```
const data: MyType = JSON.parse(json);
```

🧠 Где работает контекстный вывод:

Где используется	Как работает
Функции <code>map</code> , <code>filter</code>	Тип элемента массива известен
События DOM	Тип события выводится из имени (<code>"click"</code>)
Колбэки (<code>setTimeout</code>)	Параметры функции выводятся из сигнатуры
JSX-пропсы (в React)	Типы берутся из компонентов
Передача функций в аргументы	Типы аргументов выводятся из ожидаемой сигнатуры

💡 Советы: как эффективно использовать

✅ Используй функции в контексте

```
const logUser = (user: { id: number }) => {  
  console.log(user.id);  
};
```

Но если ты передаёшь `logUser` в другую функцию — TypeScript поймёт тип `user` **автоматически**.

✅ Не пиши типы, если контекст их даёт

```
const numbers = [1, 2, 3];
```



```
const doubled = numbers.map(n => n * 2); // n: number — автоматически
```

! Не полагайся на вывод, если:

- значения получены из `any` (например, `JSON.parse`, `fetch`)
- ты передаёшь данные между слоями (например, из `API` в `UI`)
- ты пишешь публичный API (тогда лучше явно указывать типы)

 Резюме

Делай	Не делай
✓ Используй вывод в <code>.map</code> , <code>event =></code>	✗ Не полагайся на <code>any</code> без проверки
✓ Позволяй TS сам определить тип	✗ Не пиши лишние аннотации без смысла
✓ Доверяй выводу, если есть контекст	✗ Не используй <code>as</code> без нужды

Эволюционирующие типы

что такое эволюционирующие типы в TypeScript, почему они могут вести себя неожиданно, и как с ними работать правильно

 Что такое *эволюционирующий тип*?

*Эволюционирующий тип — это тип переменной, который **ещё не определён явно**, и TypeScript **выводит его "на ходу"** по мере использования.*

 Пример:

```
let value; // Тип пока не задан → value: any

value = 42;           // value: number
value = "hello";      // ! всё ещё допустимо, т.к. был any
```


🔴 Здесь `value` — **эволюционирующий тип**. Без указания типа TypeScript **не может его стабилизировать**.

🧩 Другой пример: массив

```
let items = [];           // ? items: any[]
items.push("hello");      // → items: string[]
items.push(42);           // 🔴 Опа! Теперь items: (string | number)[]
```

👉 Массив **меняет тип** по мере добавления новых значений — это тоже эволюция типа.

! Почему это опасно?

1. TypeScript сначала допускает любое — `any` или `unknown`
2. Тип может стать неожиданно объединённым: `string | number | null | ...`
3. Не ловит ошибки на этапе добавления, пока не стало поздно

✅ Как правильно:

1. **Сразу задавай явный тип**

```
let value: string;
value = "hello";
```

```
let items: string[] = [];
items.push("world");
```

🛡 Это делает тип стабильным и предотвращает нежелательную "эволюцию".

2. **Используй `const` вместо `let`, если значение не изменяется**

```
const greeting = "hello"; // greeting: "hello" (литеральный тип)
```


Это полезно для строгой типизации и избежания случайных изменений.

3. **Для массивов указывай тип, даже если он пустой**

```
let ids: number[] = []; // Чётко: сюда можно только number
```

4. **Не полагайся на `let x;` без типа**

```
let data; // ❌ data: any
```

Лучше так:

```
let data: string | null = null; // ✅ явно
```



Типичная ловушка

```
let config = {};  
config.apiUrl = "https://..."; // ❌ Ошибка: свойство 'apiUrl' не существует
```

Почему? Потому что `{}` выводится как **пустой объект** (`{}`), а не `Record<string, unknown>`. Тип не успел "эволюционировать".

✅ Лучше:

```
let config: { apiUrl?: string } = {};  
config.apiUrl = "https://...";
```



Резюме

Делай

Не делай

✓ Задавай типы явно	✗ Не пиши <code>let x;</code> без типа
✓ Определи тип массива сразу	✗ Не полагайся на <code>[]</code> без типа
✓ Используй <code>const</code> , если можешь	✗ Не полагайся на автоматический вывод при сложных значениях
✓ Контролируй структуру объектов	✗ Не доверяй пустым объектам без типов

Функциональные конструкции и библиотеки для улучшения работы с типами

Использование **функциональных конструкций** и специализированных **библиотек** помогает не только писать чище и понятнее, но и значительно усиливает **безопасность типов** в TypeScript.

🧠 Идея: типы и функции должны быть *связаны*

Во **функциональном стиле** ты строишь программы из **маленьких чистых функций** с явными типами, и TypeScript может **максимально использовать вывод и проверки типов**.

✓ Что значит "функциональные конструкции"?

Это:

- `map`, `filter`, `reduce`
- `compose`, `pipe`
- `Option`, `Either`, `Result`
- `pattern matching`
- неизменяемые данные (`readonly`)
- функции высшего порядка

📦 Примеры и как они улучшают типы

1. **Функции высшего порядка с типами**


```
function mapArray<T, U>(arr: T[], fn: (item: T) => U): U[] {  
  return arr.map(fn);  
}
```

🧠 TypeScript выведет `U` автоматически.

2. ****Функции-комбинаторы (`pipe`, `compose`)****

С библиотекой `fp-ts` или `effect`, или даже с `lodash/fp`:

```
import { pipe } from 'fp-ts/function';  
  
const result = pipe(  
  [1, 2, 3],  
  arr => arr.map(x => x * 2),  
  arr => arr.filter(x => x > 2)  
);
```

✅ Каждая функция типизирована, результат выводится безопасно.

3. ****Типизированные контейнеры: `Option`, `Either`****

```
import { Option, some, none, isSome } from 'fp-ts/Option';  
  
const maybeName: Option<string> = some("Alice");  
  
if (isSome(maybeName)) {  
  console.log(maybeName.value.toUpperCase()); // ✅ TS знает: string  
}
```

📌 Альтернатива `string | null | undefined`, но с полной безопасностью.

4. ****Pattern Matching (например, `ts-pattern`)****

```
import { match } from 'ts-pattern';  
  
type Shape =  
  | { kind: 'circle'; radius: number }  
  | { kind: 'square'; size: number };  
  
const area = (shape: Shape) =>  
  match(shape)  
    .with({ kind: 'circle' }, s => Math.PI * s.radius ** 2)
```



```
.with({ kind: 'square' }, s => s.size ** 2)
.exhaustive(); // ✅ Проверка на полный разбор вариантов
```

🔒 Ты больше не забудешь про ни один вариант `Shape`.

📖 Полезные библиотеки

Библиотека	Что даёт
<code>fp-ts</code>	Алгебраические типы (<code>Option</code> , <code>Either</code> , <code>Task</code> , <code>IO</code> , ...)
<code>ts-pattern</code>	Безопасный <code>pattern matching</code> с проверкой <code>exhaustiveness</code>
<code>zod</code> / <code>io-ts</code>	Проверка данных и построение типов из них
<code>effect</code>	Мощная функциональная экосистема с типовой безопасностью
<code>remeda</code>	Типобезопасная замена <code>lodash</code>

🎯 Как улучшает работу с типами?

- 🔒 Меньше `any`, `as`, и других "затычек"
- 🔗 Композируемость и повторное использование
- ✅ Автоматический вывод типов между шагами
- 🧩 Явные альтернативы `null`, `throw`, `try/catch`

💡 Советы

- 🧠 Используй `Option` вместо `string | null`
- 🚫 Не пиши `if (x)` — пиши `match(x)` или `isSome(x)`
- 🧱 Композиция функций вместо вложенных вызовов
- 🚫 Избегай мутации — это ломает вывод типов
- 🔍 Используй библиотеки, где типы — часть контракта

Async-функции вместо обратных вызовов

Использование `async` / `await` вместо обратных вызовов (callback'ов) — это способ сделать **асинхронный код в TypeScript** более читаемым, безопасным и удобным для работы с

типами.

Разберёмся **по-простому и с примерами**:

Что такое "обратный вызов" (callback)?

Это функция, которую ты передаёшь как аргумент, чтобы она была вызвана позже:

```
function fetchData(callback: (data: string) => void) {
  setTimeout(() => {
    callback("hello");
  }, 1000);
}
```

Но такой код:

- тяжело читать (особенно вложенный)
- сложно обрабатывать ошибки (`try/catch` не работает)
- хуже типизируется

Как использовать `async/await` вместо этого

```
function fetchData(): Promise<string> {
  return new Promise(resolve => {
    setTimeout(() => {
      resolve("hello");
    }, 1000);
  });
}

async function run() {
  const data = await fetchData(); //  простой, понятный код
  console.log(data);
}
```

Как это улучшает работу с типами

TypeScript **понимает тип возвращаемого значения** из `Promise<T>`, и ты получаешь **автоматическую проверку типов**:

```
async function getUser(): Promise<{ id: number; name: string }> {
  return { id: 1, name: "Alice" };
}
```



```
}

async function showUser() {
  const user = await getUser(); // user: { id: number; name: string }
  console.log(user.name);       // ✅ всё типизировано
}
```

🧱 Как переделать код с callback → `async`

Было:

```
function loadUser(callback: (err: Error | null, user?: string) => void) {
  // ...
}
```

Стало:

```
function loadUser(): Promise<string> {
  return new Promise((resolve, reject) => {
    // ...
  });
}

const user = await loadUser();
```



Обработка ошибок

С callback'ами:

```
loadUser((err, user) => {
  if (err) {
    console.error(err);
    return;
  }
  console.log(user);
});
```

С `async/await` :


```
try {
  const user = await loadUser();
  console.log(user);
} catch (err) {
  console.error("Ошибка:", err);
}
```

✔ Можно использовать `try/catch`, как в обычном синхронном коде.

Резюме

Было (Callback)	Стало (async/await)
Много вложенных функций	Последовательный, читаемый код
Ошибки — через <code>err</code> в параметре	Ошибки — через <code>try/catch</code>
Нет вывода типов	Тип <code>Promise<T></code> → автоматический вывод
Сложно обрабатывать цепочки	Просто вызывать <code>await</code> в любом месте

Советы

- Всегда возвращай `Promise<T>` из `async`-функций
- Оборачивай старые `callback`-функции через `new Promise(...)`
- Используй `await` только внутри `async`-функций
- Обработывай ошибки через `try/catch` или `.catch(...)`

Классы и каррирование для создания новых точек вывода типов

Как **классы** и **каррирование** могут помочь тебе **создавать новые точки вывода типов** в TypeScript.

Что такое "точка вывода типов"?

Это место, где **TypeScript** может **автоматически вывести типы** из контекста.

Например:

```
const result = fn("hello"); // ← точка вывода: TypeScript знает тип аргумента и резул
```

Теперь давай посмотрим, как **классы** и **каррирование** помогают создавать такие точки.

1. Классы как точка вывода

Классы часто используются как "типовые фабрики", особенно если ты хранишь `T` в поле и используешь его в методах.

```
class Box<T> {
  constructor(private value: T) {}

  get(): T {
    return this.value;
  }

  map<U>(fn: (value: T) => U): Box<U> {
    return new Box(fn(this.value));
  }
}

const b = new Box("hello"); // ← TypeScript выводит T = string
const len = b.map(s => s.length); // U = number
```

✓ Здесь `Box<T>` становится **новой точкой вывода**: как только ты передал `T`, остальные методы используют его.

2. Каррирование как точка вывода

Каррирование — это преобразование функции с несколькими аргументами в цепочку вызовов по одному аргументу:

```
function add(a: number): (b: number) => number {
  return b => a + b;
}

const add2 = add(2); // ← TypeScript выводит: add2: (b: number) => number
add2(3); // = 5
```

Каррированные функции создают **точки вывода на каждом шаге**:

- `T` выводится на первом вызове,
- `U` — на втором и т. д.

Пример: создать трансформатор с классом и каррированием

```
class Transformer<T> {
  constructor(private value: T) {}

  map = <U>(fn: (v: T) => U): Transformer<U> => {
    return new Transformer(fn(this.value));
  }

  get = (): T => this.value;
}

// Использование:
const result = new Transformer("hello")
  .map(s => s.length)           // T = string → U = number
  .map(n => n * 2)              // T = number → U = number
  .get();                      // number
```

✓ Типы выводятся на каждом `.map()` — это новые **точки вывода типов**.

Комбинация: каррированная функция-конструктор с классом

```
function createBox<T>(value: T) {
  return new Box(value);
}

const strBox = createBox("test"); // T = string
const numBox = strBox.map(s => s.length); // U = number
```

Зачем это нужно?

- **Улучшение вывода типов** — не нужно вручную указывать `<T>` или `<U>`
- **Меньше дублирования** — ты передаёшь тип один раз, а потом он "течёт" по цепочке
- **Создание DSL / fluent API** — удобно для библиотек и конфигов

Резюме

Что	Как помогает
Класс	Хранит <code>T</code> , создаёт методы, где <code>T</code> используется или заменяется
Метод <code>.map</code>	Создаёт новую точку вывода через <code>fn: (T) => U</code>
Каррирование	Делит функцию на этапы с выводом на каждом этапе
Фабрики	Помогают автоматически выводить <code><T></code> при создании

Моделирование допустимых состояний (valid states)

Моделирование допустимых состояний (valid states): идея в том, чтобы **невозможные состояния были невозможны** с точки зрения типов.

Цель:

Сделать так, чтобы код, представляющий недопустимое состояние, просто не компилировался.

Принципы:

- Используй объединения (`|`) и литеральные типы для различения вариантов.
- Разделяй состояния явным образом (discriminated unions).
- Избегай лишних `null | undefined` без нужды.
- Делай невозможное недостижимым на уровне типов.

Пример: загрузка данных

 Неправильно (можно совместить несовместимое):


```
type DataState = {
  loading: boolean;
  data?: string;
  error?: string;
};

// Можем получить: { loading: true, error: "fail" } — что нелогично!
```

✅ Правильно (моделируем допустимые состояния явно):

```
type Loading = { status: "loading" };
type Success = { status: "success"; data: string };
type Error = { status: "error"; error: string };

type DataState = Loading | Success | Error;

// Теперь статус строго ограничен:
const state: DataState = { status: "success", data: "hello" };
```

🧠 Теперь TypeScript заставит тебя обрабатывать все случаи:

```
function render(state: DataState) {
  switch (state.status) {
    case "loading":
      return "Загрузка...";
    case "success":
      return `Данные: ${state.data}`;
    case "error":
      return `Ошибка: ${state.error}`;
  }
}
```

🧱 Пример: формы

```
type ValidForm = { valid: true; values: { name: string } };
type InvalidForm = { valid: false; errors: string[] };

type FormState = ValidForm | InvalidForm;

function handleForm(f: FormState) {
  if (f.valid) {
    f.values.name; // ✅ TypeScript знает: valid → есть values
  } else {
    f.errors;      // ✅ valid: false → есть errors
  }
}
```



```
}  
}
```

📌 Техника: \[**Tagged unions** / Discriminated unions**]

Используй:

```
{ kind: "... " } или { type: "... " } или { status: "... " }
```

чтобы TypeScript мог **сужать тип** внутри `if` или `switch`.

✨ Плюсы подхода

Безопасность	Поведение
Нельзя создать недопустимое состояние	Типы защищают от ошибок
Легче обрабатывать логические ветви	Все варианты учтены явно
Улучшается автодополнение	IntelliSense помогает
Поведение чётко по модели	Прочитать и понять — легко

! Не допускай таких состояний:

```
type User = {  
  loggedIn: boolean;  
  token?: string;  
};
```

Можно получить: `{ loggedIn: false, token: "abc" }` — бессмыслица.

✅ Лучше:

```
type LoggedOut = { loggedIn: false };  
type LoggedIn = { loggedIn: true; token: string };
```



```
type User = LoggedOut | LoggedIn;
```

Подсказка: типы = множества допустимых значений

Если представить тип как **множество возможных объектов**, то хорошая модель — это **точно заданное подмножество**: только допустимые конфигурации.

Резюме

 Всегда описывай состояние так, чтобы недопустимое просто не могло существовать:

- Используй объединения (`|`) и литеральные теги (`status` , `type` , `kind`).
- Разделяй состояния, не смешивай несовместимые поля.
- Пусть TypeScript проверяет корректность вместо тебя.

Будь консервативным в отправке, либеральным в приёме

Идея — "будь консервативным в отправке, либеральным в приёме" — пришла из проектирования сетевых протоколов, но в контексте TypeScript она означает:

Принцип:

Когда ты что-то отправляешь (например, вызываешь функцию, возвращаешь объект, экспортируешь тип) — делай это строго. А когда принимаешь — допускай большее разнообразие.

На практике в TypeScript:

1. ****Либеральный приём: принимать больше вариантов****

Когда ты **принимаешь значения**, позволяй использовать разные, более широкие типы:

```
function greet(person: { name: string }) {  
  console.log(`Hello, ${person.name}`);  
}
```

```
// Ты можешь передать объект с лишними полями — и это ок:  
greet({ name: "Alice", age: 30 }); // ✓
```

2. ****Консервативная передача: возвращать минимально достаточное****

Когда ты **возвращаешь объект** — старайся возвращать только нужное, а не больше.

```
function createUser(name: string): { name: string } {  
  return { name }; // 🔒 не добавляем лишние поля  
}
```

Так потребитель не будет случайно полагаться на лишние поля, которые мы могли бы удалить в будущем.



Почему это важно?

- 💥 Если ты начнёшь **возвращать больше, чем нужно**, потребители могут начать полагаться на лишние детали → потом больно менять.
- ✚ Если ты **принимаешь только минимально нужное**, у вызывающего больше свободы.



Пример хорошего баланса

```
// Либерально: принимаем любой объект с name  
function sayHi(p: { name: string }) {  
  console.log("Hi, " + p.name);  
}  
  
// Консервативно: возвращаем объект с ровно одним полем  
function createName(name: string): { name: string } {  
  return { name };  
}
```


Что не делать

Не возвращай «тяжёлые» объекты:

```
function getUser() {  
  return {  
    id: 1,  
    name: "Alice",  
    password: "123", // Лишнее!  
    internalFlag: true, // Лишнее!  
  };  
}
```

Потребитель может начать полагаться на `password`, что небезопасно.

Пример с типами:

```
type MinimalUser = { name: string };  
type FullUser = { name: string; age: number; admin: boolean };  
  
// Мы принимаем минимальное:  
function welcome(user: MinimalUser) {  
  console.log("Welcome, " + user.name);  
}  
  
// И возвращаем только минимальное:  
function createUser(name: string): MinimalUser {  
  return { name };  
}  
  
// Но использовать можно и более полный объект:  
const u: FullUser = { name: "Alice", age: 30, admin: true };  
welcome(u); //  ОК
```

Итоговое правило:

Действие	Стратегия
 Принимаешь	Тип широкий
 Отдаёшь	Тип узкий, строгий



Лучшие практики (31-49)

Не повторять информацию типа в документации



Принцип

Документация не должна дублировать то, что уже выражено в типах.

Потому что:

- Типы **уже** рассказывают о структуре, параметрах и возвратах.
- Повторение типа в комментарии = риск **расхождения** документации и кода.



Пример плохой практики

```
/**
 * Функция возвращает сумму двух чисел
 * @param a Число
 * @param b Число
 * @returns Число
 */
function sum(a: number, b: number): number {
  return a + b;
}
```

Зачем тут писать `Число`, если `number` уже видно?



Лучше так:

```
/**
 * Складывает два числа.
 * Возвращает их сумму.
 */
```



```
function sum(a: number, b: number): number {  
  return a + b;  
}
```

Комментарий **добавляет смысл**, а не повторяет тип.

Когда писать документацию?

Пиши документацию, когда нужно объяснить:

- **Назначение функции** или переменной.
- **Побочные эффекты** (например, "отправляет запрос", "изменяет состояние").
- **Ограничения** ("строка не должна быть пустой", "если X — true, то Y обязано быть передано").
- **Примеры использования**.

Правило: "Объясняй поведение, не структуру"

Комментируй	Не комментируй
Что делает функция	Что такое <code>number</code> , <code>string</code> и т.п.
Почему так, а не иначе	Полностью повторённые типы
Как использовать или вызывать	Названия переменных

Вместо комментариев — используйте мощные типы

Пример:

```
type HexColor = `#${string}`;  
  
/**  
 * Устанавливает цвет фона  
 */  
function setBackground(color: HexColor) { ... }
```

✓ Комментарий объясняет **намерение**, тип — **формат**.

🌟 Используй JSDoc там, где это нужно:

```
/**
 * @param user Пользователь, которому отправим письмо
 * @param urgent Если true, отправка будет немедленной
 */
function sendEmail(user: User, urgent = false): void { ... }
```

Это полезно, **если пояснение неочевидно из типов**.

🧠 Вывод

📌 **Не дублируй типовую информацию в комментариях.** Пусть комментарии объясняют поведение, намерение и контекст.

💡 Чем выразительнее у тебя типы, тем **меньше нужна документация**, и наоборот — документация должна **дополнять**, а не **повторять** код.

Избегать включения `null` или `undefined` в псевдонимы типов

Избегать включения `null` или `undefined` в псевдонимы типов — это про **ясность**, **безопасность** и **предсказуемость типов**.

🔍 Что это значит?

В TypeScript ты можешь определить тип-псевдоним (type alias), который включает `null` или `undefined`:

```
type MaybeString = string | null | undefined;
```

Но такое объединение **приводит к потенциальным ошибкам**, особенно если ты **не всегда проверяешь значения**.

❌ Почему это плохо?

1. Заставляет проверять везде:

```
function greet(name: MaybeString) {  
  console.log("Hello, " + name.toUpperCase()); // ✨ ошибка, если name === null  
}
```

2. **Расширяет область действия "пустых" значений** — теперь `null` и `undefined` могут "утечь" в другие части кода.

3. **Снижает надёжность**: невозможно понять, когда `null` допустим, а когда нет.

✅ Что делать вместо этого

1. Разделяй "есть значение" и "нет значения"

```
type Some<T> = { kind: "some"; value: T };  
type None = { kind: "none" };  
type Option<T> = Some<T> | None;  
  
function getName(): Option<string> {  
  return Math.random() > 0.5 ? { kind: "some", value: "Alice" } : { kind: "none" };  
}
```

Теперь TypeScript заставит тебя явно обрабатывать оба варианта — `some` и `none`.

2. Избегай включения ``null | undefined`` в сам тип

Плохо:

```
type User = {  
  name: string | undefined;  
};
```

Лучше:

```
type User = {  
  name?: string; // поле может отсутствовать, но это не то же самое, что `undefined`  
};
```


3. Будь явным в том, что `null` или `undefined` — часть логики

Если `null` несёт **реальный смысл**, отрази это через структуру, а не тип:

```
type WithError<T> = { kind: "error"; message: string } | { kind: "ok"; value: T };
```

Пример из жизни

❌ Плохо:

```
type Response = string | null | undefined;

function printResponse(r: Response) {
  if (r) console.log(r.toUpperCase()); // всё ещё не безопасно
}
```

✅ Лучше:

```
type Response = { kind: "ok"; value: string } | { kind: "none" };

function printResponse(r: Response) {
  if (r.kind === "ok") console.log(r.value.toUpperCase());
}
```

Резюме: как избегать `null` / `undefined` в псевдонимах

✅ Используй `Option<T>` или `Result<T, E>`-подобные паттерны ✅ Разделяй состояния явно (через `kind`, `tag`, `status`) ✅ Не добавляй `null | undefined` к типу без крайней необходимости ✅ Используй `?` для полей, а не `string | undefined` ✅ Используй строгий режим: `strictNullChecks: true`

Вынесение `null` на периферию типов

В TypeScript принцип "**выноси `null` и `undefined` на периферию типов**" помогает делать типы **безопаснее, чётче и проще в использовании**.

Что значит "на периферию"?

Это значит:

Пусть `null` и `undefined` появляются только в местах, где ты явно их обрабатываешь, а не внутри сложных или вложенных структур.

Плохо: ``null`/`undefined`` внутри сложных типов

```
type User = {
  name: string;
  email: string | null;
  profile: {
    age: number | undefined;
  };
};
```

Тут тебе придётся везде проверять `email` и `age`, что усложняет использование.

Хорошо: оборачивай всё в явно различимые формы

Вынеси `null` наружу:

```
type Profile = {
  age: number;
};

type User = {
  name: string;
  email: string;
  profile: Profile;
};

// И возвращай User | null при отсутствии
function getUser(): User | null {
  ...
}
```


Теперь ты всегда работаешь с **полноценным User**, или **не работаешь вообще**. Проверку делаешь один раз:

```
const user = getUser();
if (user) {
  user.email.toUpperCase(); // безопасно
}
```

Подход: моделируй отсутствие целиком, а не кусками

Плохо:

```
type Comment = {
  authorName: string | null;
  content: string | null;
};
```

Лучше:

```
type Comment =
  | { kind: "present"; authorName: string; content: string }
  | { kind: "deleted" }; // или { kind: "missing" }
```

"Периферия" — это:

- ✓ входные параметры функций
- ✓ возвращаемые значения
- ✓ поля, отмеченные `?` — но только если это необходимо

Примеры на практике

Внутри структуры:

```
type Config = {
  retries: number | undefined;
```



```
};
```

→ Надо проверять `config.retries` каждый раз.

✓ На уровне возвращаемого значения:

```
type Config = {
  retries: number;
};

function loadConfig(): Config | undefined { ... }

const config = loadConfig();
if (config) {
  config.retries; // безопасно
}
```

🧠 Зачем это делать?

Причина	Объяснение	
🔧 Типы становятся чище	Без `null`	undefined` внутри вложенностей
🧠 Меньше когнитивной нагрузки	Не нужно везде помнить про проверки	
🛡️ Выше безопасность	Ошибки ловятся на этапе компиляции	
🔄 Ясные границы	Где начинается и заканчивается "пустота"	

💡 Совет

Используй `null | undefined` **только**:

- когда значение **целиком отсутствует**
- и ты **сразу проверяешь его после получения**

🧩 Хочешь пример "до и после"?


```
// ❌ До: сложный тип с неопределённостью внутри
type Task = {
  title: string;
  assignee?: {
    name: string;
    email: string | null;
  };
};

// ✅ После: отсутствие вынесено наружу
type Assignee = { name: string; email: string };
type Task = {
  title: string;
  assignee: Assignee | null;
};
```

✅ Резюме: как выносить `null` на периферию

- Не пидай `null` / `undefined` внутрь сложных типов
- Делай `T | null` или `T | undefined` — **вокруг всего объекта**, не внутри
- Проверь один раз, а не много
- Используй `Option<T>`, `Result<T, E>` или `Tagged unions` при необходимости
- Включи `strictNullChecks` — он поможет

Предпочитай объединения интерфейсов (union of interfaces)

🧩 Принцип:

Предпочитай объединения интерфейсов (union of interfaces) ❌ вместо:
интерфейс с полями-объединениями (`interface with unioned properties`)

Это делает код **более читаемым**, типы — **более точными**, а поведение — **предсказуемым**.

🤔 В чём разница?

❌ Интерфейс с объединением в поле:

```
interface Animal {
  kind: "cat" | "dog";
  sound: string;
}
```

TypeScript **не понимает**, что `"cat"` и `"dog"` имеют **разные свойства** — приходится проверять вручную.

✅ Объединение интерфейсов:

```
interface Cat {
  kind: "cat";
  sound: "meow";
  purrs: boolean;
}

interface Dog {
  kind: "dog";
  sound: "woof";
  barks: boolean;
}

type Animal = Cat | Dog;
```

Теперь TypeScript **понимает различия**, и можно использовать **type narrowing**:

```
function speak(animal: Animal) {
  if (animal.kind === "cat") {
    animal.purrs; // ✅ доступно
  } else {
    animal.barks; // ✅ доступно
  }
}
```

🧠 Почему это лучше?

Интерфейс с объединением	Объединение интерфейсов		
<code>kind: "cat"</code>	<code>"dog"</code>	<code>Cat</code>	<code>Dog</code>
Свойства общие	Свойства могут отличаться		

TypeScript не может уточнить	TypeScript делает type narrowing		
Мутный тип	Чёткие границы поведения		

Запомни:

- ❌ Плохо:

```
interface Shape {
  kind: "circle" | "square";
  radius?: number;
  sideLength?: number;
}
```

- ✅ Лучше:

```
interface Circle {
  kind: "circle";
  radius: number;
}

interface Square {
  kind: "square";
  sideLength: number;
}

type Shape = Circle | Square;
```

Как перейти с «интерфейса с объединениями» на объединения интерфейсов?

- Выдели значение-дискриминатор (`kind` , `type` , `status`)
- Создай **отдельный интерфейс для каждого варианта**
- Объедини их в `type Union = A | B | C`
- Используй **type narrowing** с `switch` или `if (kind === "...")`

Пример: API ответ

- ❌ Плохо:


```
interface ApiResponse {
  status: "ok" | "error";
  data?: User;
  error?: string;
}
```

✔ **Лучше:**

```
interface OkResponse {
  status: "ok";
  data: User;
}

interface ErrorResponse {
  status: "error";
  error: string;
}

type ApiResponse = OkResponse | ErrorResponse;
```

Теперь ты можешь писать:

```
function handleResponse(res: ApiResponse) {
  if (res.status === "ok") {
    res.data.id; // ✔ доступно
  } else {
    console.error(res.error); // ✔ доступно
  }
}
```

Резюме

Делай	Не делай		
<code>`type A = B`</code>	<code>C`</code>	<code>`interface A { x: B`</code>	<code>C`</code>
Разделяй поведение по типу	Не смешивай варианты в одном поле		
Используй <code>kind</code> - дискриминатор	Не прячь различия в <code>optional</code> - полях		

Использование более узких (строгих) типов вместо `string`

Использование **более узких (строгих) типов вместо** `string` — это один из самых мощных способов сделать TypeScript-код надёжнее и самодокументируемым.

Почему просто `string` — не всегда хорошо?

`string` охватывает **любой текст** — это слишком широкое множество значений:

```
function setRole(role: string) {  
  // какой-нибудь role вроде "cat" тоже пройдёт  
}
```

Ошибки типа `setRole("adminn")` — не поймаются на этапе компиляции.

Как сделать лучше?

1. Используй ****литеральные типы****:

```
type Role = "admin" | "user" | "guest";  
  
function setRole(role: Role) {  
  // теперь только допустимые значения  
}
```

Теперь `setRole("superuser")` выдаст ошибку!

2. Используй ****брендированные строки**** (Branded types)

Если тебе нужно передавать **разные строки по смыслу**, но по структуре они одинаковые:

```
type Email = string & { __brand: "email" };  
type UserID = string & { __brand: "user-id" };  
  
function sendEmail(to: Email) { ... }  
function findUser(id: UserID) { ... }
```


Теперь ты **не сможешь перепутать** `Email` и `UserID`, даже если это строки.

3. 🧩 Используй ****шаблонные строковые типы****

```
type HexColor = `#${string}`;

const color: HexColor = "#ffcc00"; // ✅
const bad: HexColor = "blue";      // ❌ ошибка
```

4. 📁 Используй ****enum**** (если значения фиксированы и нужны числа)

```
enum Status {
  Pending = "pending",
  Approved = "approved",
  Rejected = "rejected"
}

function handleStatus(status: Status) {
  ...
}
```

Enum даёт те же гарантии, что и литеральный тип, но с другим синтаксисом.

📌 Пример до и после

❌ До (широкий `string`)

```
function startAction(action: string) { ... }
startAction("launch"); // никто не знает, правильно ли
```

✅ После (литеральный тип)

```
type Action = "launch" | "stop" | "pause";
function startAction(action: Action) { ... }
```




Резюме: как заменить `string`

Вместо...	Используй...	Для чего?	
<code>string</code>	Литеральный тип (`"foo"`)	"bar"`)	Ограниченные значения
<code>string</code>	Шаблонные строки	Валидные форматы (например, цвета, URL)	
<code>string</code>	Брендированные типы	Разделение по смыслу (email, slug)	
<code>string</code>	<code>enum</code>	Перечисления с авто/ручным значением	

Использовать отдельный тип для специальных значений

Использование **отдельных типов для специальных значений** — это мощный приём, который помогает избежать путаницы и ошибок, особенно когда одно и то же по структуре значение (`string`, `number`, и т.д.) может означать **разные вещи в контексте**.



Что значит "специальные значения"?

Это такие значения, которые:

- имеют особое **значение в логике** программы,
- **отличаются по смыслу** от других значений такого же типа,
- требуют **особого обращения** (например, `unknown`, `loading`, `invalid`, `notFound`, и т.д.).



Плохо: использовать обычные значения

```
function getUsername(id: string): string {
  const user = db.find(id);
  return user ? user.name : "unknown"; // ! "unknown" — специальное значение, но всё
}
```


Тут `"unknown"` — это не имя, а флаг. Но TypeScript не может это понять.

✓ Лучше: использовать отдельный тип

```
type UserName = string;
type UnknownUser = { kind: "unknown" };

type UserResult = UserName | UnknownUser;

function getUsername(id: string): UserResult {
  const user = db.find(id);
  return user ? user.name : { kind: "unknown" };
}
```

Теперь ты **явно** моделируешь "специальное" состояние, и TypeScript заставляет тебя его обрабатывать.

🔧 Ещё примеры

📄 Пример 1: Специальное значение в результатах

Плохо:

```
function findProduct(id: string): Product | null {}
```

Лучше:

```
type NotFound = { kind: "not-found" };
type ProductResult = Product | NotFound;

function findProduct(id: string): ProductResult {}
```

💬 Пример 2: Статус загрузки

```
type Loading = { kind: "loading" };
type Success<T> = { kind: "success"; data: T };
type Failure = { kind: "error"; message: string };
```



```
type LoadResult<T> = Loading | Success<T> | Failure;
```

Теперь ты можешь писать:

```
function render(result: LoadResult<User>) {
  switch (result.kind) {
    case "loading":
      return "⌚";
    case "success":
      return result.data.name;
    case "error":
      return "Error: " + result.message;
  }
}
```

Зачем это делать?

Причина	Объяснение
✔ Типовая безопасность	TypeScript заставит обрабатывать особые случаи
🧠 Явное намерение	Тип говорит сам за себя: <code>NotFound</code> , <code>Empty</code> , <code>Pending</code>
📦 Отделение данных от флагов	Не смешиваются "значения" и "сигналы"
⚠ Меньше магических строк	Нет <code>"unknown"</code> , <code>"none"</code> , <code>"n/a"</code> в <code>string</code>

Используй, когда...

- Значение особенное и **не должно обрабатываться как обычное**
- Оно требует **особой логики**
- Ты хочешь **отделить "невалидное" от "валидного"**

Резюме

Вместо...	Используй...		
<code>string = "n/a"</code>	<code>type NotAvailable = { kind: "n/a" }</code>		

<code>number = -1</code>	<code>type InvalidID = { kind: "invalid" }</code>		
<code>`null`</code>	<code>string`</code>	<code>`string`</code>	<code>{ kind: "none" }`</code>
<code>`"error"`</code>	<code>string`</code>	<code>`{ kind: "error" }`</code>	<code>{ kind: "ok", value: string }`</code>

Ограничить использование опциональных свойств

опциональные свойства (`?`) кажутся удобными, но при чрезмерном или неправильном использовании могут привести к неясному или ошибочному коду.

🔍 Что значит "ограничить использование опциональных свойств"?

Это значит:

- Не ставить `?` «на всякий случай»
- Строго контролировать, когда свойство может **действительно отсутствовать**
- Избегать опциональности, когда она **мешает безопасности типов** или **затрудняет логику**

! Почему чрезмерное использование вредно?

1. 🔍 **Непредсказуемо**: приходится каждый раз проверять на `undefined`
2. ⚠️ **Усложняет поведение**: логика зависит от наличия/отсутствия поля
3. ☹️ **Трудно отлаживать**: почему поле отсутствует — баг или фича?

✅ Как ограничивать опциональность?

1. **Предпочитай объединения типов с ``kind`` вместо ``?``**

❌ Плохо:


```
interface User {  
  name: string;  
  email?: string;  
}
```

✓ **Лучше:**

```
interface UserWithEmail {  
  name: string;  
  email: string;  
}  
  
interface UserWithoutEmail {  
  name: string;  
}  
  
type User = UserWithEmail | UserWithoutEmail;
```

2. ****Уточняй типы при построении****

Вместо:

```
type Product = {  
  name: string;  
  discount?: number;  
}
```

Сделай:

```
type DiscountedProduct = { name: string; discount: number };  
type FullPriceProduct = { name: string };  
  
type Product = DiscountedProduct | FullPriceProduct;
```

3. ****Или — используй `null`/`undefined` как значение, не как отсутствие****

```
interface User {  
  bio: string | null; // явно: поле всегда есть, но может быть пустым
```



```
}
```

Теперь не нужно проверять `if ("bio" in user)` — `bio` **всегда существует**.

4. ****Строго описывай этапы создания объекта****

Например, если объект строится поэтапно:

```
type PartialUser = {  
  name?: string;  
  email?: string;  
}  
  
type CompleteUser = {  
  name: string;  
  email: string;  
}
```

А потом можно использовать **type guards**:

```
function isCompleteUser(u: PartialUser): u is CompleteUser {  
  return !!u.name && !!u.email;  
}
```

5. ****Не делай поля опциональными, если они "появляются позже"****

Вместо:

```
interface Order {  
  id: string;  
  status?: "pending" | "paid"; // ❌ статус должен быть всегда  
}
```

Сделай:

```
type OrderPending = { id: string; status: "pending" };  
type OrderPaid = { id: string; status: "paid" };  
type Order = OrderPending | OrderPaid;
```


Резюме

Делай	Не делай
Используй объединения вместо <code>?</code>	Не ставь <code>?</code> «про запас»
Делай <code>null</code> значением, не отсутствием	Не смешивай <code>undefined</code> и <code>null</code>
Используй <code>kind</code> -дискриминаторы	Не надевай одну маску на два состояния

Избегайте повторяющихся параметров одного типа

Повторяющиеся параметры одного типа могут приводить к:

- 💣 ошибкам (легко перепутать порядок),
- 🤔 нечитаемому коду (непонятно, что за что отвечает),
- 🧠 усложнению поддержки (особенно если типы одинаковые: `string`, `number`, и т.д.).

❌ Пример плохой практики

```
function createUser(name: string, email: string, phone: string) {  
  // какой аргумент что означает?  
}
```

Все параметры — `string`, и легко перепутать их порядок:

```
createUser("user@example.com", "John", "+123456"); // 💣 ошибка, но компилятор не руг
```

✅ Как избежать?

1. 📦 Объединяй параметры в объект с полями

```
function createUser(params: { name: string; email: string; phone: string }) {  
  const { name, email, phone } = params;
```



```
}
```

Преимущества:

- Явные имена
- Нет зависимости от порядка
- IDE помогает с автодополнением

Вызов:

```
createUser({ name: "John", email: "user@example.com", phone: "+123456" });
```

2. 🧩 Используй именованные типы

```
type UserInput = {  
  name: string;  
  email: string;  
  phone: string;  
};  
  
function createUser(input: UserInput) { ... }
```

Так можно переиспользовать тип и документировать намерения.

3. 🔍 Используй ****tuple only if order matters and types are distinct****

Если всё-таки нужно позиционное API (например, в библиотеках или низкоуровневых функциях), то лучше использовать **tuple с разными типами**, чтобы избежать путаницы:

```
function setBounds(bounds: [number, number]) {  
  const [min, max] = bounds;  
}
```

Но если все элементы одного типа — предпочтительнее объект.

4. 🧠 Типы с брендингом (если всё-таки нужно оставить позиционность)


```
type UserName = string & { __type: "UserName" };
type Email = string & { __type: "Email" };

function register(name: UserName, email: Email) { ... }
```

Теперь ты не передашь `email` вместо `name` , даже если они — просто строки.

Резюме

Делай	Не делай
Используй объекты с полями	Не передавай 2+ одинаковых <code>string</code>
Вводи <code>type</code> / <code>interface</code>	Не полагайся на порядок аргументов
Рассматривай брендинг, если надо	Не жертвуй читаемостью ради краткости

Унификация типов, вместо моделирования

Стремиться к унификации типов, а не к моделированию различий — это принцип, который помогает писать проще, надёжнее и легче масштабируемый код в TypeScript.

Что это значит?

Вместо того чтобы **создавать много почти одинаковых типов**, которые немного отличаются, старайся найти **общее ядро** — и описать различия в данных, а не в типах.

Плохо: избыточное моделирование различий

```
type Circle = {
  kind: "circle";
  radius: number;
};

type Square = {
  kind: "square";
  size: number;
};
```



```
type Triangle = {  
  kind: "triangle";  
  base: number;  
  height: number;  
};
```

Тут 3 типа, у каждого свои свойства. Это может привести к дублированию, путанице и сложной логике обработки.

✓ Лучше: ****унифицированная форма****

Найди **общую структуру**, даже если не все поля всегда используются:

```
type Shape = {  
  kind: "circle" | "square" | "triangle";  
  radius?: number;  
  size?: number;  
  base?: number;  
  height?: number;  
};
```

Если нужно точнее — лучше сделать дискриминирующее объединение, **но с унифицированной обработкой**:

```
type Shape =  
  | { kind: "circle"; radius: number }  
  | { kind: "square"; size: number }  
  | { kind: "triangle"; base: number; height: number };
```

А ещё лучше:

```
interface ShapeBase {  
  color: string;  
  position: { x: number; y: number };  
}  
  
type Shape =  
  | (ShapeBase & { kind: "circle"; radius: number })  
  | (ShapeBase & { kind: "square"; size: number })  
  | (ShapeBase & { kind: "triangle"; base: number; height: number });
```

Так ты **унифицируешь общее**, а различия изолируешь.



Пример из API

Плохо:

```
type Admin = { name: string; isAdmin: true; rights: string[] };
type RegularUser = { name: string; isAdmin: false; profile: Profile };
```

Лучше:

```
type User = {
  name: string;
  isAdmin: boolean;
  rights?: string[];
  profile?: Profile;
};
```

Или даже:

```
type UserBase = { name: string };
type Admin = UserBase & { role: "admin"; rights: string[] };
type Regular = UserBase & { role: "user"; profile: Profile };
type User = Admin | Regular;
```

Теперь логика обработки `User` становится предсказуемой.



Когда уместна унификация:

Подход	Когда применять
Унификация через общее ядро	Разные сущности имеют 70–80% совпадающих полей
Объединения с дискриминатором	Различия важны, но нужно единое API
Расширение интерфейсов	Подтипы можно выразить через базовый тип
Массивы и функции	Нужен общий тип-параметр для коллекций/обработки



Принцип

** Типы — это средство для описания структуры, не для отражения различий в реальности.*

Резюме

Плохо	Лучше
Много почти одинаковых типов	Один обобщённый тип
Повторяющиеся поля в разных типах	Вынеси общее в базовый интерфейс
Сложная логика из-за разной формы данных	Унификация формы — меньше проверок

Лучше неточно, чем недостоверно

Принцип "**лучше неточно, чем недостоверно**" — один из самых важных в работе с типами в TypeScript. Он помогает писать надёжный и предсказуемый код.

Что это значит?

✓ **Лучше сказать TypeScript-у "я не уверен до конца", чем сказать "я уверен", но ошибаться.**

Проще говоря:

Пусть тип будет шире, чем нужно, чем уже, чем есть на самом деле.

! Недостоверный тип: опасно

```
function getUser(): { name: string } {  
  return JSON.parse(localStorage.getItem("user")!);  
}
```

Здесь мы утверждаем, что у результата **точно** есть `name: string`. Но:

- `localStorage.getItem` может вернуть `null`
- `JSON.parse` может вернуть что угодно
- объект может быть `{}`

💣 Это **недостоверный тип** — TypeScript считает, что у объекта точно есть `name`, а на деле — может и нет.

✅ Неточный, но безопасный тип

```
function getUser(): unknown {
  const raw = localStorage.getItem("user");
  return raw ? JSON.parse(raw) : null;
}
```

Здесь мы признаём: результат неизвестен. TypeScript **заставит** нас провести проверку:

```
const user = getUser();

if (user && typeof user === "object" && "name" in user) {
  console.log((user as any).name);
}
```

Или ещё лучше — делаем runtime-проверку и преобразование:

```
function isUser(obj: unknown): obj is { name: string } {
  return typeof obj === "object" && obj !== null && "name" in obj;
}
```

🎯 Когда использовать более ****неточный**** тип

Ситуация	Делай так		
Данные приходят из внешнего источника	Используй <code>unknown</code> или <code>any</code> + проверку		
Работа с <code>JSON.parse</code> , <code>fetch</code> , <code>eval</code> , и т.п.	Не доверяй структуре без валидации		

Точно не знаешь структуру заранее	Скажи <code>Partial<T></code> , <code>Record<string, unknown></code> и т.д.		
Возвращаем неустойчивые значения	Делай результат <code>`T</code>	<code>undefined</code> или <code>T</code>	<code>null`</code>

Пример: API-запрос

Недостоверно:

```
async function fetchUser(): Promise<User> {
  const res = await fetch("/api/user");
  return await res.json(); // может вернуть что угодно!
}
```

Безопаснее:

```
async function fetchUser(): Promise<unknown> {
  const res = await fetch("/api/user");
  return await res.json();
}
```

Затем:

```
function isUser(obj: unknown): obj is User { ... }

const data = await fetchUser();
if (isUser(data)) {
  showUser(data);
} else {
  showError("Invalid response");
}
```

Идея в одном предложении

 **Тип не должен говорить больше, чем ты можешь гарантировать на 100%.**

Резюме

Недостоверно	Неточно, но безопасно
Утверждать <code>T</code> , не проверив	Начать с <code>unknown</code> , <code>any</code> , <code>Partial<T></code>
Уверенность без оснований	Валидация и runtime-проверки
Ошибки при обращении к несуществующим свойствам	Type narrowing и guards

Имена типов на языке предметной области задачи

Назначение имён типам на **языке предметной области (domain language)** — это важная практика для понятного, сопровождаемого и масштабируемого кода.

Что это значит?

Не называй типы технически (типа `Data`, `Item`, `Info`, `Props`) — называй их так, как говорят эксперты в твоей области.

Такой код становится самодокументируемым: читающий понимает, что происходит, **без копания в деталях**.

Пример

Плохо:

```
type Data = {
  id: string;
  name: string;
};

function process(data: Data): void { ... }
```

Что за `Data`? Про что? Что делает `process`?

Хорошо (на языке предметной области):


```
type Customer = {
  id: string;
  name: string;
};

function onboardCustomer(customer: Customer): void { ... }
```

Теперь ясно: ты работаешь с клиентами и «онбордишь» их.

🔧 Как подбирать имена

Вместо...	Лучше использовать...
Info / Data	UserProfile , InvoiceDetails
Item / Entry	Product , Transaction , Post
Params / Props	CreateOrderInput , EmailFormData
Result / Output	SearchResults , AuthToken

📦 Пример из интернет-магазина

Плохо:

```
type Item = {
  title: string;
  cost: number;
};
```

Хорошо:

```
type Product = {
  title: string;
  price: number;
};
```

Или если в заказе:


```
type OrderItem = {  
  productId: string;  
  quantity: number;  
};
```

Зачем это нужно

-  Код понятен как документация
-  Проще искать и переиспользовать
-  Легче масштабировать при развитии продукта
-  Разработчики «думают» в терминах бизнеса

Полезный приём: "задать вопрос"

Если ты можешь сказать: «**Я сейчас работаю с ___**», — это имя и есть хороший кандидат для типа.

Примеры:

- «Я работаю с **CustomerOrder**» — 
- «Я работаю с **Data**» — 

Резюме

Делай	Избегай
Используй термины бизнеса	Абстрактных имён (<code>Data</code> , <code>Info</code>)
Думай в терминах задачи	Называть по структуре (<code>Props</code>)
Названия = понятные сущности	Технических сокращений

Типы на основе API и спецификаций, а не данных

В TypeScript (и вообще в typed-разработке) **нужно генерировать типы на основе спецификаций API, а не на основе данных**, потому что данные — **частные случаи**, а спецификация — **источник истины**.

Что значит "генерировать типы по спецификации"?

Вместо того чтобы писать типы руками, глядя на JSON-ответ, лучше:

- Использовать спецификацию API (например, OpenAPI, GraphQL, gRPC)
 - Генерировать из неё TypeScript-типы автоматически
 - Или хотя бы — основывать ручные типы на **формальной схеме**, а не на случайных примерах
-

Плохо: типы по наблюдаемым данным

```
// получаем JSON
const response = {
  id: 1,
  name: "Alice",
  age: 30
};

// и думаем "ага", значит тип:
type User = {
  id: number;
  name: string;
  age: number;
};
```

▲ Проблема: завтра API может добавить поле `email`, поменять тип `age`, или вообще вернуть ошибку — а ты об этом не знаешь.

Хорошо: типы из спецификации

Например, API имеет [OpenAPI \(Swagger\)](#) описание:

```
components:
  schemas:
    User:
      type: object
      required: [id, name]
      properties:
```



```
id:
  type: integer
name:
  type: string
age:
  type: integer
```

→ Это можно **автоматически** превратить в TypeScript-тип:

```
type User = {
  id: number;
  name: string;
  age?: number;
};
```

🔧 Инструменты для генерации типов

API/Протокол	Генерация типов	Инструменты
REST / OpenAPI	<code>.ts</code> -типы по Swagger/JSON	<code>openapi-typescript</code> , <code>swagger-typescript-api</code>
GraphQL	Автоматически по схеме	<code>graphql-codegen</code>
gRPC / Protobuf	Из <code>.proto</code> → <code>.ts</code>	<code>ts-proto</code>
JSON Schema	Типы из схемы	<code>json-schema-to-typescript</code>

🧠 Почему так важно?

Генерация по спецификации	Ручное написание по данным
✅ Актуально при изменениях	❌ Устаревает быстро
✅ Проверяет весь контракт	❌ Покрывает только пример
✅ Позволяет договариваться с backend	❌ Ломается при изменениях API
✅ Помогает в автотестах и документации	❌ Хрупко, особенно при рефакторинге

🔄 Если спецификации нет?

- Требуй её 😊
- Или используй **runtime-валидацию** с помощью `zod`, `io-ts`, `valibot`, `yup`
- Пример:

```
import { z } from "zod";

const UserSchema = z.object({
  id: z.number(),
  name: z.string(),
  age: z.number().optional(),
});

type User = z.infer<typeof UserSchema>;
```

Теперь у тебя и **валидатор**, и **тип** — из одного источника 🔥

Резюме

Делай	Избегай
Генерируй типы из OpenAPI / GraphQL / JSON Schema	Не строй типы только по JSON-примерам
Используй инструменты типа <code>openapi-typescript</code>	Не верь, что данные всегда такие, как сейчас
Валидация + типизация из одной схемы	Дублирование схемы и типов вручную

Использовать максимально узкий диапазон для типов `any`

В TypeScript тип `any` стоит использовать **только в самом узком контексте**, если без него никак — и всегда стремиться **ограничить его распространение** по коду.

Почему `any` опасен?

`any` отключает проверку типов:


```
const value: any = getSomething();
value();           // ок, даже если это не функция
value.notAProp();  // ок, TypeScript не ругается
```

 Это нарушает цель TypeScript — предотвращать ошибки до запуска.

✓ Как использовать `any` максимально безопасно

1. **Ограничь область действия `any` одной переменной**

Плохо:

```
function handle(data: any) {
  const user: User = data; // небезопасно: передаём any дальше
  processUser(user);
}
```

Хорошо:

```
function handle(data: any) {
  if (isUser(data)) {
    processUser(data);
  }
}
```

2. **Сужай `any` как можно скорее**

```
function handleResponse(raw: any) {
  const user = parseUser(raw); // parseUser: (x: any) => User | null
  if (user) showUser(user);
}
```

✓ Тип `any` только внутри `handleResponse`, а в остальном коде — строго.

3. **Не распространяй `any` в другие типы**

```
type Response = {
  user: any; // ✗ плохо: any теперь "заражает" всё
```



```
};
```

Лучше:

```
type Response = {  
  user: unknown; // ✅ безопаснее  
};
```

И потом:

```
if (isUser(response.user)) {  
  doSomething(response.user);  
}
```

4. ****Используй `unknown` вместо `any`****

- `unknown` — безопасный аналог `any`
- Но требует проверок перед использованием

```
function handle(x: unknown) {  
  if (typeof x === "string") {  
    console.log(x.toUpperCase());  
  }  
}
```

5. ****Ставь `any` только в точках, где *другого выхода нет*****

Например:

- Работа с `window` или сторонней библиотекой без типов
- Динамический JSON (но и тут лучше `unknown`)
- Переходный этап миграции JS → TS

6. ****Оборачивай `any`-объекты в валидаторы****

```
import { z } from "zod";  
  
const UserSchema = z.object({  
  id: z.number(),
```



```
    name: z.string(),
  });

function parseUser(input: any): User | null {
  const result = UserSchema.safeParse(input);
  return result.success ? result.data : null;
}
```

Теперь даже если `input: any`, ты гарантируешь тип после проверки.

🧠 Резюме: как ограничивать ``any``

Правило	Делай это
⚠️ Минимизируй зону действия <code>any</code>	Используй только в одной переменной
🔒 Не передавай <code>any</code> другим функциям/типам	Сначала сужай, потом вызывай
🛡️ Используй <code>unknown</code> вместо <code>any</code>	Он безопаснее
🔧 Проводи runtime-проверку (<code>typeof</code> , <code>zod</code>)	Для безопасного извлечения данных

Более точные варианты `any`

Когда хочется написать `any`, но **нельзя терять безопасность типов**, — лучше использовать **более точные альтернативы**, которые ограничивают поведение, **но остаются гибкими**.

🛡️ Почему не ``any``?

- `any` отключает проверку типов полностью
- Типы перестают проверяться вообще — это как выключить TypeScript

✅ Альтернативы ``any`` — с пояснениями

Альтернатива	Когда использовать	Пример применения
<code>unknown</code>	Когда точно не знаешь тип, но планируешь его проверить	<pre>function handle(x: unknown) { if (typeof x === 'string') ... }</pre>

<code>Record<string, unknown></code>	Когда нужен объект с произвольными ключами и неизвестными значениями	<code>function parse(obj: Record<string, unknown>) { ... }</code>
<code>Partial<T></code>	Когда нужен объект, у которого часть полей может отсутствовать	<code>updateUser(data: Partial<User>)</code>
<code>T null</code> / <code>T undefined</code>	Когда значение может быть отсутствующим	<code>name?: string</code> или <code>name: string undefined</code>
<code>object</code>	Когда известно, что это не примитив , но структура не важна	<code>function doSomething(val: object) { ... }</code>
<code>Function</code>	Когда значение — функция , но сигнатура неизвестна	<code>let cb: Function = () => {}</code> (лучше избегать)

❌ Чего избегать

Тип	Почему плохо
<code>any</code>	Отключает проверки типов — теряется смысл TypeScript
<code>Function</code> (часто)	Не даёт проверок аргументов или возвращаемого значения
<code>object</code> (без уточнений)	Не указывает структуру — почти как <code>any</code>

🔍 Примеры

1. `unknown` вместо `any`

```
function handle(data: unknown) {
  if (typeof data === "string") {
    console.log(data.toUpperCase());
  }
}
```

 `unknown` требует проверок, в отличие от `any`

2. `Record` вместо `any`


```
function handleUser(user: Record<string, unknown>) {  
  if (typeof user.name === "string") {  
    console.log("User:", user.name);  
  }  
}
```

✅ Позволяет работать с объектами, не теряя проверки ключей

3. `Partial` — частично известная структура

```
type User = {  
  name: string;  
  age: number;  
};  
  
function updateUser(user: Partial<User>) {  
  if (user.age !== undefined) {  
    console.log(user.age + 1);  
  }  
}
```

4. Уточнение сигнатуры функции

Плохо:

```
const callback: any = () => {};
```

Хорошо:

```
type Callback = (event: MouseEvent) => void;  
const callback: Callback = (event) => { ... };
```

5. Обёртка с `zod` (или другим валидатором)

```
import { z } from 'zod';  
  
const UserSchema = z.object({
```



```
    id: z.number(),
    name: z.string()
  });

function parseUser(input: unknown): { id: number; name: string } | null {
  const parsed = UserSchema.safeParse(input);
  return parsed.success ? parsed.data : null;
}
```

Резюме

Не делай	Вместо этого
<code>const x: any</code>	<code>const x: unknown</code> , <code>Record<string, unknown></code>
<code>arg: any</code>	Проверяй аргументы: <code>unknown</code> + <code>typeof</code>
<code>Function</code> , <code>object</code>	Определи сигнатуру и структуру
Не проверяй тип <code>any</code>	Добавь runtime-проверку

Скрывать небезопасные утверждения типов в хорошо типизированных функциях

Чтобы **скрывать небезопасные утверждения типов** (type assertions), нужно **локализовать "опасные" участки кода** и **изолировать их в функции с хорошей типизацией**. Так TypeScript не теряет уверенность в остальной программе.

Цель

Спрятать `as any`, `as T`, `!`, `unknown as Something` — в одном месте, с минимальной зоной риска.

Как это делать правильно

1. **Упаковывай небезопасность в маленькие, строго типизированные функции**

● Плохо:

```
const user = response as any;
console.log(user.name);
```

● Хорошо:

```
function parseUser(raw: unknown): User | null {
  if (typeof raw === "object" && raw !== null && "name" in raw) {
    return { name: String((raw as any).name) }; // <-- локализовано
  }
  return null;
}

const user = parseUser(response);
if (user) {
  console.log(user.name); // ✅ теперь безопасно
}
```

📦 Всё сомнительное — **внутри** `parseUser`, остальной код работает с **строгими типами**

2. **Пиши обёртки с валидаторами**

С библиотеками вроде `zod`, `io-ts`, `valibot`, `runtypes`:

```
import { z } from 'zod';

const UserSchema = z.object({
  name: z.string(),
  age: z.number(),
});

function safeParseUser(data: unknown): User | null {
  const result = UserSchema.safeParse(data);
  return result.success ? result.data : null;
}
```

Здесь утверждение типа (`as`) делает библиотека, а не ты ✨

3. ****Скрывай `as` внутри утилит****

Пример утилиты для получения свойства:

```
function getProp<T extends object, K extends keyof T>(obj: T, key: K): T[K] {  
  return obj[key];  
}
```

Даже если внутри что-то типа `as`, пользователь этого не видит — только строгий тип.

4. ****Сужай `any` и `unknown` до строго определённых типов****

● Плохо:

```
function doSomething(x: any) {  
  const user = x as User;  
  ...  
}
```

● Лучше:

```
function isUser(x: unknown): x is User {  
  return typeof x === "object" && x !== null && "name" in x;  
}  
  
function doSomething(x: unknown) {  
  if (isUser(x)) {  
    console.log(x.name);  
  }  
}
```

5. ****Документируй риск, если избежать `as` невозможно****

```
// ВНИМАНИЕ: здесь мы утверждаем тип, доверяя API  
const user = response as User;
```


Это позволяет позже легко найти все "небезопасные" места.

🧠 Резюме

Советы	Почему важно
✅ Оборачивай <code>as</code> в безопасные функции	Изолируешь риск
✅ Проверяй структуру перед приведением типа (<code>typeof</code> , <code>in</code>)	TypeScript начинает "доверять" данным
✅ Используй библиотеки-валидаторы (<code>zod</code> , <code>io-ts</code>)	Упрощает проверку и заменяет <code>as</code>
✅ Добавляй типы на входе, не на выходе	Так меньше риска "навязать" неправильный тип
❌ Не делай <code>const user = something as User</code> без проверки	Это потенциальный баг

unknown вместо any для значений с неизвестным типом

Отличный и важный вопрос: как использовать `unknown` вместо `any` — и при этом сохранять безопасность типов.

🤔 Почему `unknown` лучше, чем `any`?

<code>any</code>	<code>unknown</code>
Отключает все проверки типов	Требует явной проверки перед использованием
Может привести к ошибкам в рантайме	Защищает от некорректного использования
"Дырка" в системе типов	Тип-безопасный "чёрный ящик"

✅ Как использовать `unknown` правильно?

1. Объявляй переменные или аргументы как `unknown`, если не знаешь тип заранее

```
function handleInput(input: unknown) {  
  // теперь ты обязан проверить тип  
}
```

2. Используй проверки типов перед использованием

```
function handleInput(input: unknown) {  
  if (typeof input === 'string') {  
    console.log(input.toUpperCase()); // ✅ безопасно  
  }  
  
  if (Array.isArray(input)) {  
    console.log(input.length); // ✅ безопасно  
  }  
}
```

3. Используй пользовательские функции-утверждения (type guards)

```
type User = { name: string };  
  
function isUser(obj: unknown): obj is User {  
  return typeof obj === 'object' && obj !== null && 'name' in obj;  
}  
  
function process(x: unknown) {  
  if (isUser(x)) {  
    console.log(x.name); // ✅ x точно User  
  }  
}
```

4. Используй библиотеки для валидации (`zod`, `io-ts`, `valibot` и т.д.)

```
import { z } from 'zod';  
  
const UserSchema = z.object({ name: z.string() });
```



```
function parse(data: unknown) {
  const result = UserSchema.safeParse(data);
  if (result.success) {
    const user = result.data;
    console.log(user.name); // ✅ безопасно
  }
}
```

5. Никогда не делай так:

```
function bad(x: unknown) {
  console.log((x as any).foo); // ❌ смысл `unknown` теряется
}
```

🧠 Резюме

Правило	Зачем
✅ Используй <code>unknown</code> , если не знаешь тип	Это безопасная альтернатива <code>any</code>
✅ Проверяй тип перед использованием	TypeScript гарантирует безопасность
✅ Используй type guards и валидацию	Для строгого контроля
❌ Не «ломай» <code>unknown</code> через <code>as any</code>	Это теряет смысл и приводит к ошибкам

Типобезопасные решения вместо динамической модификации

Динамическая модификация объектов в JavaScript (через `obj[prop] = value`, `delete obj[key]`, `Object.assign`, и т.п.) удобна, но **в TypeScript может нарушать типобезопасность**.

Вместо этого — лучше использовать **типобезопасные решения**, которые:

- работают в рамках системы типов TypeScript;
- сохраняют автодополнение, проверки и читаемость;
- минимизируют ошибки на этапе компиляции.

❌ Примеры динамической модификации (плохо в TypeScript)

```
const user: any = {};  
user['name'] = 'Alice';           // 🔴 небезопасно  
delete user['age'];               // 🔴 структура "плавающая"
```

✅ Альтернативы — типобезопасные подходы

1. **Явное создание объектов (сразу целиком)**

```
interface User {  
  name: string;  
  age: number;  
}  
  
const user: User = {  
  name: 'Alice',  
  age: 30,  
}; // ✅ строго соответствует типу
```

2. **Чистые функции для изменения объектов (без мутаций)**

```
function withName(user: User, name: string): User {  
  return { ...user, name };  
}  
  
const updated = withName(user, 'Bob'); // ✅ тип сохранён
```

3. **Mapped types + Record для предсказуемых структур**

```
type UserRoles = 'admin' | 'editor' | 'viewer';  
  
const accessMap: Record<UserRoles, boolean> = {  
  admin: true,  
  editor: true,  
  viewer: false,  
}; // ✅ только допустимые ключи
```


4. **Функции обновления с типами ключей**

```
function updateField<T, K extends keyof T>(obj: T, key: K, value: T[K]): T {
  return { ...obj, [key]: value };
}

const newUser = updateField(user, 'name', 'Charlie'); // ✅ тип "name" = string
```

5. **Удаление свойств — через утилиты или строгие обёртки**

```
type Without<T, K extends keyof T> = Omit<T, K>;

function withoutKey<T, K extends keyof T>(obj: T, key: K): Without<T, K> {
  const { [key]: _, ...rest } = obj;
  return rest;
}

const userWithoutAge = withoutKey(user, 'age'); // ✅ безопасно
```

Резюме

Динамика JavaScript	Типобезопасная альтернатива
<code>obj[prop] = value</code>	<code>updateField(obj, key, value)</code> с <code>keyof</code>
<code>delete obj[key]</code>	<code>Omit</code> + обёртка
<code>Object.assign(obj, x)</code>	<code>...spread</code> с типами
<code>obj['anyKey']</code>	<code>Record<Key, Value></code>
<code>any</code> -поля для расширения	<code>Partial<T></code> , <code>Record<K, V></code>

Избегайте ловушек надёжности

«Избегать ловушек надёжности» в TypeScript — это значит **не создавать видимость безопасности типов там, где её на самом деле нет.**

🕳️ Что такое "ловушки надёжности"?

Это ситуации, когда **код компилируется без ошибок**, но **типовая система ничего не гарантирует**, либо гарантирует не то, что мы ожидаем.

🔥 Примеры ловушек и как их избегать

1. ****Использование `any`****

🔥 Типовая система отключена

```
function parse(input: any): User {  
  return input; // 🔴 нет гарантии, что это User  
}
```

✅ Используй `unknown` + валидацию:

```
function parse(input: unknown): User {  
  if (isUser(input)) return input;  
  throw new Error("Invalid user");  
}
```

2. ****Тип `{}` (пустой объект)****

🔥 Слишком широкий тип, почти как `any`

```
function log(obj: {}) {  
  obj.toString(); // 🔴 не всегда безопасно  
}
```

✅ Лучше использовать `object` или конкретные типы:

```
function log(obj: object) { ... }
```

3. ****Объектные типы-обёртки (`String`, `Number`, `Boolean`)****

🔥 Они не то же самое, что `string`, `number`, `boolean`


```
const x: String = new String('hello'); // 🚫 ловушка
```

✅ Используй простые типы:

```
const x: string = 'hello';
```

4. ****Избыточная уверенность через `!` (non-null assertion)****

```
function getName(user?: User) {  
  return user!.name; // 🚫 упадёт, если user === undefined  
}
```

✅ Явная проверка:

```
function getName(user?: User) {  
  if (!user) return 'Unknown';  
  return user.name;  
}
```

5. ****Слепое утверждение типов (`as T`) без проверки****

```
const result = input as SomeType; // 🚫 ты утверждаешь, но не проверяешь
```

✅ Лучше проверять и использовать **type guards** или **валидацию**.

6. ****Опциональные поля без проверки****

```
type User = { name?: string };  
  
function greet(user: User) {  
  return user.name.toUpperCase(); // 🚫 ошибка в рантайме  
}
```

✅ Проверяй:


```
if (user.name) {
  return user.name.toUpperCase();
}
```

✓ Общие советы, чтобы не попасть в ловушку

Совет	Зачем		
🔍 Проверяй входящие данные (через guards, <code>typeof</code> , библиотеки)	Не доверяй внешним значениям вслепую		
🚫 Избегай <code>any</code> , <code>as</code> , <code>!</code> без причины	Это лазейки, которые обходят систему типов		
✓ Предпочитай <code>unknown</code> вместо <code>any</code>	<code>unknown</code> требует проверки перед использованием		
🧩 Определи, что <code>null</code> и <code>undefined</code> — тоже значения	Добавляй их явно: <code>`string`</code>	<code>null</code> , <code>number</code>	<code>undefined`</code>
🔒 Всегда моделируй все возможные состояния	Используй <code>union</code> , <code>never</code> , <code>exhaustive switch</code>		

Отслеживайте зону охвата типов для предотвращения регрессии в типобезопасности

Как отслеживать "зону охвата типов" в TypeScript, чтобы предотвратить регрессии типобезопасности — это про то, где и насколько хорошо система типов защищает код, и как не ослаблять её со временем.

💡 Что такое "зона охвата типов"?

Это участки кода, **покрытые системой типов TypeScript** — где данные и поведение описаны типами настолько точно, что большинство ошибок ловятся на этапе

КОМПИЛЯЦИИ.

● Хорошо типизированная зона — это когда:

- все функции имеют аннотации типов;
- нет `any`, `as`, `!`;
- есть проверки для `null`, `undefined`;
- используется `strict` режим.

● Плохо типизированная зона:

- не указаны типы;
- много `any`;
- типы "протекают";
- silent-casting и приведения через `as`.

✓ Как отслеживать и поддерживать типовую зону охвата?

1. **Включи строгий режим (`strict`)**

`tsconfig.json` :

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

Включает:

- `noImplicitAny`
- `strictNullChecks`
- `strictFunctionTypes`
- и др.

🔒 Это расширяет охват типами и не даёт писать код без явно определённого поведения.

2. ****Избегай `any` и замени их на `unknown`, `never`, `T`, `Partial`, и т.д.****

Используй [`noImplicitAny`], [`noImplicitThis`] → TypeScript будет требовать, чтобы **все** переменные, параметры и возвращаемые значения были строго определены.

3. ****Запрещай `any` через `tsconfig`****

```
{
  "compilerOptions": {
    "noImplicitAny": true
  }
}
```

Или в ESLint:

```
"@typescript-eslint/no-explicit-any": "error"
```

4. ****Интегрируй линтер (ESLint + TypeScript)****

С правилами, отслеживающими:

- `any`
- неиспользуемые типы
- слабые типы (`{}` | `object`)
- небезопасные утверждения

5. ****Проверяй границы зон: где типы «теряются»****

💡 Например:

```
function handleData(data: any) { ... } // 🔴 потеря зоны охвата

function handleData(data: unknown) { // ✅ тип сохраняется
  if (isData(data)) {
    // тип снова точен
  }
}
```

→ Каждая функция — либо усиливает типовую защиту, либо «дырявит» её.

6. ****Пиши больше *type-safe* утилит вместо дублирования****

```
function update<T, K extends keyof T>(obj: T, key: K, value: T[K]): T {
  return { ...obj, [key]: value };
}
```

→ Это переносит логику внутрь **строго типизированной зоны**.

7. ****Автоматизируй проверку охвата****

- Используй `ts-prune` для выявления неиспользуемых типов
- ESLint + `@typescript-eslint` для контроля за `any` и `unknown`
- Добавляй `// @ts-expect-error` вместо `// @ts-ignore`, чтобы явно показать отклонение от типовой зоны (и не забыть про него)

 Резюме: как удерживать границы типобезопасности

Приём	Зачем
✔ <code>strict: true</code> в tsconfig	Усиливает зону типобезопасности
✔ <code>noImplicitAny</code> , <code>noUncheckedIndexedAccess</code>	Не позволяет писать "скользкий" код
✔ ESLint с правилами типизации	Автоматически ловит ослабление типов
✔ <code>unknown</code> вместо <code>any</code>	Сохраняет обязанность проверять
✔ Границы между типами — через <code>type guards</code>	Предотвращает «утечку» слабых типов
✔ <code>@ts-expect-error</code> вместо <code>@ts-ignore</code>	Явно отслеживает места отклонения от типовой модели



Продвинутые типы (50-64)

Обобщённые (generics) конструкции



Используйте обобщённые (generics) конструкции, когда тип может быть разным — но правила работы с ним остаются одинаковыми.



Простой пример

Вместо того чтобы писать одну и ту же функцию для каждого конкретного типа:

```
function identityNumber(x: number): number {  
  return x;  
}  
  
function identityString(x: string): string {  
  return x;  
}
```

✓ Обобщаем:

```
function identity<T>(x: T): T {  
  return x;  
}
```

Теперь функция работает **одинаково для всех типов**, и TypeScript выводит тип `T`:

```
identity(123);    // T = number  
identity('hi');  // T = string
```



Где рассматривать обобщения в практике

Вместо этого	Лучше использовать обобщения
Повторять код для <code>number</code> , <code>string</code>	<code>function fn<T>(value: T): T</code>
Приводить к <code>any</code> , потом обратно	Обобщения с <code>T</code> , <code>K extends keyof T</code> и т.д.
Жестко указывать <code>Array<User></code> и <code>Array<Post></code>	Использовать <code>Array<T></code> или <code>T[]</code>
Создавать дублирующие интерфейсы	Использовать обобщённые <code>interface</code> или <code>type</code>

Примеры «обобщённого мышления» в типах

1. Обобщённая функция фильтра

```
function filter<T>(items: T[], predicate: (item: T) => boolean): T[] {
  return items.filter(predicate);
}
```

Работает для любого массива с любым типом.

2. Обобщённый тип ответа API

```
type ApiResponse<T> = {
  status: number;
  data: T;
};

const userResponse: ApiResponse<User> = {
  status: 200,
  data: { id: 1, name: 'Alice' },
};
```

3. Обобщённый компонент в React

```
function List<T>({ items, render }: { items: T[]; render: (item: T) => JSX.Element })
  return <ul>{items.map(render)}</ul>;
}
```


✓ Зачем рассматривать обобщения?

- 📦 Избавляют от повторяющегося кода
- 🗝️ Сохраняют типобезопасность
- 🧩 Универсальные, гибкие и хорошо сочетаются с другими типами
- 🧠 Заставляют думать о **структуре и правилах**, а не о конкретных значениях

Избегайте лишних параметров типа

«Как избегать лишних параметров типа (generic parameters)» — это про то, как **не** усложнять типы без необходимости и удерживать код читаемым и точным.

📌 В чём проблема с лишними тип-параметрами?

Иногда разработчики добавляют тип-параметры (`<T>` , `<U>` , `<K>` , и т.п.) там, где:

- их можно **вывести автоматически** (инференс),
- или они **не нужны вовсе**,
- или они **ничего не ограничивают**.

Это делает код:

- ✗ трудным для чтения,
- ✗ подверженным ошибкам,
- ✗ сложным для повторного использования.

✗ Пример лишнего параметра

```
function toArray<T>(value: T): T[] {  
  return [value];  
}  
  
const result = toArray<number>(42); // ✗ явно указали, но это лишнее
```

✓ Лучше:

```
function toArray<T>(value: T): T[] {  
  return [value];  
}
```



```
const result = toArray(42); // ✓ T выводится как number автоматически
```

🔧 Как распознать и избежать лишнего параметра

✅ Правило 1: Убирай параметр, если его можно вывести

```
function wrap<T>(value: T): { value: T } {  
  return { value };  
}  
  
// Вызов:  
wrap("hello"); // T = string — выводится сам
```

Если ты **не используешь** `T` в сигнатуре функции, значит он лишний.

✅ Правило 2: Если параметр никогда не ограничивается — он не нужен

```
function log<T, U>(a: T, b: U): void {  
  console.log(a, b);  
}
```

🔥 `U` ни на что не влияет — можно использовать `unknown`, если точная связь не нужна, или объединить тип.

✅ Правило 3: Используй ``extends`` и ``keyof`` для ограничения обобщения

```
function getValue<T, K extends keyof T>(obj: T, key: K): T[K] {  
  return obj[key];  
}
```

→ Здесь оба параметра нужны: `K` зависит от `T`.

✅ **Правило 4: Заменяй одиночные `T` на конкретные типы, если они не должны быть обобщёнными**

```
function double<T>(x: T): T { // ❌
  return x + x; // ошибка — T может быть строкой, числом, и т.д.
}
```

✅ **Лучше:**

```
function double(x: number): number {
  return x + x;
}
```

✅ **Правило 5: Используй TypeScript-инференс максимально**

TypeScript почти всегда может **выводить тип-параметры сам**, особенно:

- из аргументов функции,
- из возвращаемых значений,
- из контекста.

👉 Убери `<T>` — если всё и так работает.

🧠 **Кратко**

Лишний приём	Вместо этого
<code>function f<T>(x: T): T</code> с простыми типами	<code>function f(x: number): number</code>
Явно передавать <code><T></code>	Позволь TypeScript вывести <code>T</code> сам
Обобщение без <code>extends</code> или <code>keyof</code>	Используй ограничение, или не обобщай
Слишком много <code><T, U, V></code>	Упростить до одного параметра или конкретных

Лучше условные типы, чем перегруженные сигнатуры

Используйте **одну обобщённую функцию с условными типами**, вместо нескольких перегрузок (overloads), когда это возможно.

Что такое перегруженные сигнатуры?

Это когда у функции несколько «объявлений» типов:

```
function parse(input: string): string[];
function parse(input: number): number[];
function parse(input: string | number): any[] {
  if (typeof input === 'string') return input.split('');
  return [input];
}
```

 Минусы:

- код сложнее поддерживать;
- сигнатуры дублируются;
- невозможно передать тип результата другим функциям.

Вместо этого — используем ****условный тип**** (`T extends X ? A : B`):`

```
function parse<T extends string | number>(input: T): T extends string ? string[] : number[] {
  if (typeof input === 'string') {
    return input.split('') as any;
  }
  return [input] as any;
}
```

Теперь:

```
const a = parse('abc'); // a: string[]
const b = parse(123);   // b: number[]
```

 Один универсальный интерфейс, и результат зависит от входа.

🧠 Почему это лучше?

Перегрузка	Условный тип
📄 Нужно писать много сигнатур	🧠 Логика вся в одном обобщённом выражении
❌ Тип результата нельзя легко вынести	✅ Тип результата можно использовать отдельно
⚠️ Сложнее поддерживать и расширять	✅ Гибко адаптируется к новому поведению

📌 Пример: вместо перегрузок

❌ Так не надо:

```
function getLength(x: string): number;
function getLength(x: any[]): number;
function getLength(x: string | any[]): number {
  return x.length;
}
```

✅ Лучше так:

```
function getLength<T extends string | any[]>(x: T): number {
  return x.length;
}
```

💡 Когда перегрузки всё же полезны?

- Когда **внутреннее поведение функции сильно отличается**;
- Когда **нужна чёткая сигнатура для автокомплита**;
- Когда **невозможно выразить тип через условный** `extends` .

Пример: `Array.prototype.map` , `fetch` и т.п.

Распределение объединений

Когда ты используешь **условные типы** (`extends ? :`) вместе с **объединениями** (`|`), важно понимать поведение TypeScript по умолчанию, которое называется **распределение объединений** (*distributive conditional types*).

Что такое ****распределение объединений****?

Когда ты пишешь условный тип так:

```
type Result<T> = T extends string ? 'str' : 'other';
```

...и передаёшь **объединение** в `T`:

```
type R = Result<string | number>;
```

Ты можешь ожидать, что `R` станет `'other'`, но на самом деле:

TypeScript распределяет объединение:

```
Result<string | number>  
→ Result<string> | Result<number>  
→ 'str' | 'other'
```

Как ****управлять распределением****?

1. ****Распределение включено по умолчанию****

Такой тип **распределяет**:

```
type Foo<T> = T extends string ? 'str' : 'other';  
  
type R = Foo<string | number>; // 'str' | 'other'
```

2. ****Отключить распределение вручную**** — оберни `T` в кортеж `[T]`


```
type Foo<T> = [T] extends [string] ? 'str' : 'other';

type R1 = Foo<string | number>; // 'other' !
```

 `[T]` не распределяется, поэтому условие применяется к **всему объединению**, а не каждому элементу.

Когда нужно включать или отключать?

 **Распределять** — когда нужно ****отдельно обработать каждый тип****:

```
type ToArray<T> = T extends any ? T[] : never;

type A = ToArray<string | number>; // string[] | number[]
```

 **Не распределять** — когда нужно ****принять только точный тип****:

```
type IsString<T> = [T] extends [string] ? true : false;

type A = IsString<string | number>; // false 
```

Резюме

Что нужно	Как сделать
 Распределять объединение	<code>T extends X ? A : B</code> (по умолчанию)
 Отключить распределение	<code>[T] extends [X] ? A : B</code>
 Обработать каждый вариант по-своему	Использовать <code>T extends any ? ... : ...</code>
 Проверить, весь ли тип входит в условие	Обернуть <code>T</code> в <code>[T]</code>

Шаблонные литеральные типы для моделирования DSL и отношений между строками

Шаблонные литеральные типы в TypeScript

(Template Literal Types)

Это фича, которая позволяет **создавать новые строковые типы, комбинируя другие строковые типы или литералы**.

Зачем они нужны?

С их помощью ты можешь:

- моделировать **мини-языки (DSL)** прямо в типах,
 - проверять **отношения между строками**,
 - **автоматически строить строки** типа `"GET:/users"` или `"POST:/users/:id"`,
 - создавать **более точные API-контракты**.
-

Синтаксис

```
type Hello<T extends string> = `Hello, ${T}`;
```

```
type Greet = Hello<'World'>; // "Hello, World"
```

Примеры использования

1. **Моделирование API DSL**


```
type Method = 'GET' | 'POST';
type Route = '/users' | '/users/:id';

type APIEndpoint = `${Method}:${Route}`;

// Теперь доступны:
type A = APIEndpoint;
// "GET:/users" | "GET:/users/:id" | "POST:/users" | "POST:/users/:id"
```

✓ 2. **Маппинг из имени в ключ (CSS, HTML, конфиги, и т.п.)**

```
type Color = 'primary' | 'secondary';

type CSSVar<T extends string> = `--color-${T}`;

type Vars = CSSVar<Color>; // "--color-primary" | "--color-secondary"
```

✓ 3. **Типизация строк-команд в DSL**

```
type Command = `run ${'build' | 'test'} --env=${'dev' | 'prod'}`;

// Примеры:
const c1: Command = "run test --env=dev"; // ✓
const c2: Command = "run deploy --env=dev"; // ✗ ошибка
```

✓ 4. **Отношения между типами строк**

```
type Entity = 'user' | 'product';
type Id = `${Entity}_id`; // "user_id" | "product_id"
```

```
type MapToLabel<T extends Entity> = `label_${T}`;
// "label_user" | "label_product"
```

✓ 5. **Ключи объектов по шаблону**


```
type Fields = 'name' | 'age';
type FormData = {
  [K in Fields as `form_${K}`]: string
};

// Result:
type FormData = {
  form_name: string;
  form_age: string;
}
```



Советы

Что хочешь сделать	Как сделать
 Построить строку из других типов	Используй шаблонные типы
 Проверить соответствие строк шаблону	Используй <code>extends \ prefix_\${string}`</code>
 Генерировать ключи или DSL-команды	Комбинируй <code>in</code> + <code>as</code> + шаблонные строки
 Моделировать безопасные идентификаторы	Например, <code>id:\${string}</code> или <code>\${T}_id</code>



Проверка формата строки

```
type IsId<T> = T extends `${string}_id` ? true : false;

type A = IsId<'user_id'>; // true
type B = IsId<'user'>;    // false
```



Пример: генерация сообщений ошибок

```
type Field = 'email' | 'password';

type ErrorMsg<T extends Field> = `Invalid field: ${T}`;

const msg: ErrorMsg<'email'> = 'Invalid field: email'; // ✓
```


Когда это особенно полезно

- при создании **конфигов**,
- при работе с **i18n ключами** (`home.title`, `auth.login.error`, ...),
- для строгой типизации **endpoint-строк**, событий, команд, GraphQL-полей, ...

Как писать тесты для типов

Писать **тесты для типов в TypeScript** — значит **проверять, как работает система типов**, не запуская код. Это особенно полезно в больших проектах или при создании библиотек, где важно, чтобы типы работали как задумано.

Способы тестирования типов

✓ 1. ****С помощью утилит компилятора (`@ts-expect-error`, `@ts-ignore`, `satisfies`, и др.)****

Это самый простой и нативный способ.

✓ **Пример: проверить, что тип сужается корректно**

```
type User = { name: string };

const validUser: User = { name: 'Alice' }; // ✓ OK

// @ts-expect-error: missing property "name"
const invalidUser: User = {}; // ✗ Должна быть ошибка
```

- `@ts-expect-error` — говорит компилятору: **ожидается ошибка здесь**
- Если ошибка *не произойдёт* — компилятор пожалуется

✓ 2. ****С помощью `satisfies`****

`const x satisfies T` проверяет, что **тип** `x` **подходит под** `T`, но не сужает его.

```
const command = {
  type: 'start',
  payload: { verbose: true }
} satisfies { type: string, payload: object };
```


Используется для **подтверждения соответствия типу**, но с сохранением точной структуры `command`.

✅ 3. ****Использовать библиотеки для типовых тестов****

🔧 Библиотека [``tsd``](https://github.com/SamVerschueren/tsd)

- Позволяет писать *assertions* на типы в виде отдельных файлов
- Устанавливается через `npm install --save-dev tsd`
- Тесты хранятся в `test-d` директории

```
// example.test-d.ts
import { expectType, expectError } from 'tsd';

declare const result: number;

expectType<number>(result); // ✅
expectError<string>(result); // ❌ Ошибка, т.к. result — это number
```

🔧 Альтернатива: [``type-plus``](https://github.com/unional/type-plus)

Предоставляет утилиты вроде `Equals<T, U>`, `IsAny<T>` и т.д.

🧠 Самодельные проверки через условные типы

Ты можешь написать собственный "тестовый" тип:

```
type AssertEqual<T, Expected> =
  (<G>() => G extends T ? 1 : 2) extends
    (<G>() => G extends Expected ? 1 : 2) ? true : false;

type Test1 = AssertEqual<string, string>; // true
type Test2 = AssertEqual<string, number>; // false
```

Этот трюк позволяет **сравнивать типы на равенство** (а не только совместимость).

🔧 Где писать типовые тесты?

- Отдельный файл: `types.test.ts` или `*.test-d.ts`

- Используй `tsd` или `tsc` с `@ts-expect-error`
- В документации или рядом с исходным кодом типов (комментарии и тест-кейсы)

Резюме

Метод	Когда использовать
<code>@ts-expect-error</code>	Проверить, что тип вызывает ошибку
<code>satisfies</code>	Убедиться, что объект подходит типу
<code>tsd</code>	Формальные тесты типов в CI
<code>AssertEqual</code> , <code>Equals<T,U></code>	Внутренние проверки типов (библиотеки)

Обращайте внимание на отображение типов

Фраза "**Обращайте внимание на отображение типов**" (или "*mind type mapping*") означает:

 Следите за тем, ****как вы преобразуете и создаёте новые типы**** на основе других — особенно с помощью ****mapped types**** (отображаемых типов).

Что такое "отображение типов"?

Это способ **создавать новые типы**, проходясь по ключам существующего типа и трансформируя их.

```
type Original = { name: string; age: number };

type ReadonlyVersion = {
  readonly [K in keyof Original]: Original[K];
};
```


Мы взяли ключи `original`, и для каждого ключа скопировали тип — но сделали `readonly`.

🎯 Зачем ****обращать внимание****?

1. ✅ Чтобы избежать **случайного изменения ключей**
2. ✅ Чтобы **точно контролировать модификацию** свойств (добавить `?`, убрать `readonly`, и т.п.)
3. ✅ Чтобы понимать, как **генерация типов влияет на смысл**

📌 Примеры, где это важно

✅ 1. Добавляем модификаторы аккуратно

```
type Optional<T> = {  
  [K in keyof T]?: T[K]; // все поля становятся опциональными  
};
```

✅ 2. Убираем модификаторы

```
type RemoveReadOnly<T> = {  
  -readonly [K in keyof T]: T[K];  
};
```

Обрати внимание на `-readonly` — **убирает модификатор**, как `-?` убирает `?`

✅ 3. Генерируем новые ключи

```
type PrefixKeys<T> = {  
  [K in keyof T as `form_${string & K}`]: T[K];  
};
```



```
type Form = PrefixKeys<{ name: string; age: number }>;  
// => { form_name: string; form_age: number }
```

⚠️ Опасность: не потеряй информацию

```
type X = { name: string, age?: number };  
  
type Oops<T> = {  
  [K in keyof T]: T[K];  
};  
  
type R = Oops<X>; // 🚫 `age` по-прежнему опционален, но если ты вручную перезаписыва
```

🧠 Вывод

Когда ты "**обращаешь внимание на отображение типов**", это значит:

- 💡 Осознавай, **как ты модифицируешь ключи и значения**
- 🔒 Не забывай про модификаторы (`?`, `readonly`) — они **не переносятся по умолчанию**, если не указать явно
- ⚙️ Используй `as` для трансформации ключей
- 🛒 Пользуйся встроенными типами (`Partial`, `Required`, `Readonly`, `Pick`, `Omit`, `Record`) — они сделаны с учётом этих нюансов

Отдавайте предпочтение обобщенным типам с хвостовой рекурсией

Фраза «**Отдавайте предпочтение обобщённым типам с хвостовой рекурсией**» относится к **типам в TypeScript**, особенно в сложных типовых вычислениях — когда ты обрабатываешь **списки, строки, структуры и т.п. с помощью условных и рекурсивных типов**.

📖 Что это значит?

Когда ты создаёшь **рекурсивный условный тип** (например, обрабатываешь массив поэлементно), лучше писать его **в виде обобщения с хвостовой рекурсией**, потому что:

- ✅ Это **понятнее и управляемее**

-  TypeScript **проще вычисляет** такие типы
-  Упрощается **отладка и тестирование**

Пример: рекурсивный тип БЕЗ хвостовой рекурсии

```
type Reverse<T extends any[]> =  
  T extends [infer Head, ...infer Tail]  
    ? [...Reverse<Tail>, Head]  
    : [];
```

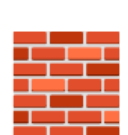
- Это **рекурсивный тип**, который переворачивает массив.
- Но `Reverse<Tail>` **встраивается внутрь нового массива** — TypeScript иногда не может вывести такой тип (или превышает глубину рекурсии).

Пример: тот же тип С ХВОСТОВОЙ РЕКУРСИЕЙ (Tail Recursion)

```
type Reverse<T extends any[], Acc extends any[] = []> =  
  T extends [infer Head, ...infer Tail]  
    ? Reverse<Tail, [Head, ...Acc]>  
    : Acc;
```

- Мы **накапливаем результат** в `Acc`
- Это хвостовая рекурсия: **вызов** `Reverse` **в последней позиции**
- Такой стиль более **предсказуем и стабилен**

Почему это важно?

-  **Хвостовая рекурсия** делает тип «ближе к итеративному» стилю
-  TypeScript может **лучше оптимизировать и обрабатывать** такие типы
-  Это **лучше масштабируется**, особенно с длинными структурами

Когда использовать?

- При **переборе списков типов**

- При создании **DeepPartial**, `Flatten`, `Zip`, `Chunk`, `Tail`, `Drop<N>` и др.
- При **строковых манипуляциях** (`Split`, `Join`, `Trim`)

✓ Пример — `Join<['a', 'b', 'c'], '->`

Без хвостовой рекурсии:

```
type Join<T extends string[], Sep extends string> =
  T extends [infer Head extends string, ...infer Rest extends string[]]
    ? `${Head}${Rest extends [] ? '' : Sep}${Join<Rest, Sep>}`
    : '';
```

С хвостовой рекурсией:

```
type Join<T extends string[], Sep extends string, Acc extends string = ''> =
  T extends [infer Head extends string, ...infer Rest extends string[]]
    ? Join<Rest, Sep, Acc extends '' ? Head : `${Acc}${Sep}${Head}`>
    : Acc;
```

Такой тип **гораздо устойчивее** при длинных массивах строк.

🧠 Вывод

"Предпочитайте обобщённые типы с хвостовой рекурсией" — это рекомендация писать типы, которые обрабатывают структуры **через аккумулирующий параметр** (`Acc`), чтобы:

- Упростить логику
- Избежать переполнения типа
- Повысить читаемость и контроль

Кодогенерация как альтернатива сложным типам

Фраза «**кодогенерация как альтернатива сложным типам**» означает:

Вместо написания **очень сложных, рекурсивных или вычисляемых типов в TypeScript**, иногда **лучше сгенерировать нужные типы автоматически во время сборки** — с помощью инструментов или скриптов.

Почему это важно

TypeScript — мощный, но **типовая система не Turing-полная** и имеет **ограничения по глубине рекурсии, вычислений, и выводов типов**.

👉 Если тип становится слишком сложным:

- 💣 Компилятор может «захлебнуться»
- 🧠 Типы становятся нечитаемыми и неотлаживаемыми
- 🐢 Производительность IDE падает

Примеры ситуаций, когда стоит подумать о ****кодогенерации****

- 🔗 **Большие API-объекты** (например, OpenAPI, GraphQL)
- 🌿 **Вложенные структуры с глубокой трансформацией** (`DeepReadonly` , `DeepMerge` , `Flatten`)
- 🏷️ **Маппинг строк или ключей** (`toSnakeCase` , `FormField<Name>`)
- 🧬 **Связанные сущности**, где типы должны зависеть от данных

✅ Альтернативы: инструменты кодогенерации

Инструмент / подход	Что делает
<code>openapi-typescript</code>	Генерирует типы по OpenAPI спецификации
<code>graphql-codegen</code>	Генерирует типы из GraphQL схем
<code>zod-to-ts</code> / <code>io-ts</code>	Генерируют TS-тип по схеме валидации
🔧 Скрипт на Node.js	Пишете сами: парсите данные → генерируете <code>.d.ts</code> файл

Пример: собственная генерация типов

Допустим, у нас есть JSON-описание форм:

```
[
  { "name": "email", "type": "string" },
  { "name": "age", "type": "number" }
]
```

Можно написать генератор, который создаст:

```
export type FormFields = {
  email: string;
  age: number;
};
```

И использовать `FormFields` в коде, **без глубоких условных типов**.

Преимущества кодогенерации

- ✓ Убирает логику из типов — **типовая система остаётся простой**
- ✓ Улучшает **производительность и стабильность IDE**
- ✓ Код легче поддерживать
- ✓ Вы можете использовать полноценные языки (`JS` , `TS` , `Python`) для описания логики

Недостатки

- Требуется **инфраструктура** (CI, скрипты)
- Кодогенерация — ещё один слой, который нужно поддерживать
- Нужно следить, чтобы **сгенерированные типы были актуальными**

Когда ****точно**** лучше использовать кодогенерацию:

- Когда у вас много однотипных API/форм/полей
- Когда тип зависит от **внешних данных** (спецификаций, конфигов, БД)
- Когда вы **не можете выразить** нужный тип средствами TypeScript

✓ Заключение

Кодогенерация — это **инженерский компромисс**: вместо усложнения типов, **переносим вычисления на *compile time*** и работаем с простыми, сгенерированными типами.

Типы = результат кода → а не вычисление в типах.

Типы `never` для выполнения проверки полноты

Тип `never` в TypeScript можно использовать как **инструмент для проверки полноты (exhaustiveness check)** — то есть, чтобы убедиться, что **все возможные случаи** обработаны. Это особенно полезно при работе с **дискретными объединениями (discriminated unions)** и `switch` или `if`-проверками.

🔍 Что значит "проверка полноты"?

Это когда вы гарантируете, что **все возможные значения** переменной учтены — и **ни один не пропущен**.

📌 Пример: дискретное объединение

```
type Shape =  
  | { kind: "circle"; radius: number }  
  | { kind: "square"; side: number }  
  | { kind: "rectangle"; width: number; height: number };
```

✓ Проверка с помощью ``never``

```
function area(shape: Shape): number {  
  switch (shape.kind) {
```



```
case "circle":
    return Math.PI * shape.radius ** 2;

case "square":
    return shape.side ** 2;

case "rectangle":
    return shape.width * shape.height;

default:
    // 📌 здесь `shape` должен быть типа `never`
    const _exhaustiveCheck: never = shape;
    return _exhaustiveCheck;
}
```

🔒 Как это работает:

- Если вы добавите в `Shape` **новый вид фигуры**, но **забудете про него в `switch`**, TypeScript покажет ошибку:

```
"Type '{ kind: "triangle", ... }' is not assignable to type 'never'"
```

✅ То же самое с `if/else`

```
function render(shape: Shape) {
    if (shape.kind === "circle") {
        return "Circle";
    } else if (shape.kind === "square") {
        return "Square";
    } else if (shape.kind === "rectangle") {
        return "Rectangle";
    } else {
        const _never: never = shape; // 🚫 если появился неучтённый вариант
        return _never;
    }
}
```

🎯 Зачем это нужно?

- Гарантирует, что **вы не забыли** обрабатывать новые случаи
- Устраняет **логические дыры**
- Улучшает **типовую надёжность и читаемость**



Частая ошибка: `default` без `never`

Если в `switch` просто написать `default: return ...`, то TypeScript **не сможет проверить полноту** — лучше в `default` явно **вынести** `never`-проверку.



Функция для проверки

Можно выделить утилиту:

```
function assertNever(x: never): never {
  throw new Error("Unexpected value: " + x);
}
```

И использовать:

```
default:
  return assertNever(shape);
```



Вывод

Тип `never` — это **сигнал для TypeScript**, что **значение невозможно получить**. Если вы где-то **всё-таки его получили** — значит, **что-то пошло не так**, и **компилятор вас предупредит**.

Итерация по объектам

Чтобы **итерировать по объектам в TypeScript**, нужно учитывать два аспекта:

1. **Как итерировать** — это можно делать стандартными способами JavaScript.
2. **Как правильно описать типы** — чтобы TypeScript понимал, с чем вы работаете и не ругался.

✂ Базовые способы итерации


```
const user = {
  name: "Alice",
  age: 30,
  active: true,
};
```

1. `for...in` — по ключам

```
for (const key in user) {
  console.log(key);           // "name", "age", "active"
  console.log(user[key]);     // TS: ⚠ Error, т.к. key — string
}
```

👉 Нужно уточнить тип:

```
for (const key in user) {
  const value = user[key as keyof typeof user]; // ✅
  console.log(value);
}
```

2. `Object.entries`

```
for (const [key, value] of Object.entries(user)) {
  console.log(key, value);
}
```

♦ По умолчанию `value` будет типа `any`, поэтому можно уточнить:

```
const entries = Object.entries(user) as [keyof typeof user, (typeof user)[keyof typeof user]];

for (const [key, value] of entries) {
  console.log(key, value);
}
```

3. `Object.keys`


```
Object.keys(user).forEach((key) => {  
    const value = user[key as keyof typeof user];  
    console.log(key, value);  
});
```

Объект с известной структурой

Если у вас, например:

```
type Product = {  
    id: number;  
    title: string;  
    price: number;  
};  
  
const product: Product = {  
    id: 1,  
    title: "Phone",  
    price: 999,  
};
```

Вы можете безопасно итерироваться по:

```
(Object.keys(product) as Array<keyof Product>).forEach((key) => {  
    const value = product[key];  
    console.log(`${key}: ${value}`);  
});
```

Итерирование по объекту с динамическими ключами

```
const stats: Record<string, number> = {  
    views: 100,  
    likes: 10,  
    shares: 5,  
};  
  
for (const key in stats) {  
    const value = stats[key]; //  value: number  
}
```


🎯 Полезные типы и утилиты

- `keyof T` — тип, представляющий ключи объекта
- `(typeof obj)[keyof typeof obj]` — тип значений

✅ Рекомендации

Что делать	Почему
Используйте <code>keyof typeof obj</code>	чтобы сохранить типовую безопасность
Не оставляйте <code>key: string</code>	TypeScript не сможет проверить значения
Приводите <code>Object.keys()</code> и <code>entries</code>	к ожидаемым типам (<code>as const</code> , <code>as keyof</code>)

Типы записей для поддержания синхронизации значений

Типы **записей** (`Record<Keys, Type>`) в TypeScript позволяют строго сопоставлять **набор ключей** с **типами значений**, и это очень удобно, когда нужно **синхронизировать данные между структурами** — например:

- ключи `enum` ↔ обработчики
- названия полей формы ↔ их значения
- имена компонентов ↔ настройки

📌 Синтаксис `Record`

```
Record<K, T> // объект, где ключи типа K, значения типа T
```

Пример:

```
type Status = "success" | "error" | "loading";

const messages: Record<Status, string> = {
  success: "Загрузка завершена",
```



```
error: "Произошла ошибка",
loading: "Загрузка...",
};
```

❌ Попробуйте забыть ключ или добавить лишний — TypeScript это отловит.

🧠 Применение: синхронизация значений

1. 💬 Сопоставление ключей и значений

```
const labels = {
  name: "Имя",
  age: "Возраст",
  email: "Эл. почта",
} as const;

type Field = keyof typeof labels; // "name" | "age" | "email"

const formData: Record<Field, string> = {
  name: "",
  age: "",
  email: "",
};
```

✅ `formData` теперь синхронизирован с `labels`: если вы уберёте или добавите ключ в `labels`, TypeScript заставит вас обновить `formData`.

2. ⚙️ Перечисление `enum` → значения

```
enum Route {
  Home = "home",
  Profile = "profile",
  Settings = "settings",
}

type RouteHandler = () => void;

const handlers: Record<Route, RouteHandler> = {
  home: () => {},
  profile: () => {},
  settings: () => {},
};
```


✓ Без `Record` TypeScript не проверил бы полноту.

3. ✓ Связывание типов ключей и значений

```
type Roles = "admin" | "user" | "guest";

type RolePermissions = {
  canEdit: boolean;
  canDelete: boolean;
};

const permissions: Record<Roles, RolePermissions> = {
  admin: { canEdit: true, canDelete: true },
  user: { canEdit: true, canDelete: false },
  guest: { canEdit: false, canDelete: false },
};
```

👉 Это позволяет синхронизировать роли и их поведение в UI/логике.

4. 📄 Форма или конфиг как типизированная таблица

```
type FieldName = "username" | "password" | "email";

const fieldPlaceholders: Record<FieldName, string> = {
  username: "Введите имя",
  password: "Введите пароль",
  email: "Введите почту",
};
```

Если вы добавите новый `FieldName` — TypeScript попросит вас добавить его и в `Record`.

💡 Почему `Record` помогает держать значения синхронизированными?

- ✓ **Централизует ключи** — меньше дублирования
- ✓ **Поддерживает exhaustiveness** (проверку полноты)
- ✓ **Устраняет "магию строк"** (всё привязано к типам)
- ✓ **Гибко трансформируется** (можно использовать `keyof`, `Pick`, `Omit` и т.д.)

Заключение

Используйте `Record<K, T>`:

- Когда у вас **фиксированный набор ключей**
- Когда хотите, чтобы все эти ключи **имели соответствующие значения**
- Когда нужно **автоматически проверять синхронизацию**

Остаточные параметры и типы кортежа для моделирования функций с переменным числом аргументов

В TypeScript можно **типизировать функции с переменным числом аргументов** с помощью:

-  **Остаточных параметров** (`...args`)
-  **Кортежей** (tuple types)

Это позволяет **гибко моделировать сигнатуру функций**, где количество и типы аргументов зависят от контекста — и при этом сохраняется **типобезопасность**.

Пример 1: Простая функция с переменным числом аргументов

```
function logAll(...args: string[]) {
  args.forEach(arg => console.log(arg));
}
```

 Это работает, но тут все аргументы должны быть `string`.

Пример 2: Типизация кортежей

```
function print(a: number, b: string, c: boolean) {}

type Args = [number, string, boolean];
```



```
function callWithArgs(...args: Args) {  
  print(...args); // ✅ типы строго соблюдены  
}
```

Здесь `Args` — **tuple type**, задающий точные типы и порядок аргументов.

🎯 Пример 3: Дженерик с кортежем

```
function callFn<T extends any[], R>(  
  fn: (...args: T) => R,  
  ...args: T  
): R {  
  return fn(...args);  
}  
  
// ✅ Пример использования:  
const result = callFn((a: number, b: string) => a + b.length, 10, "hello");
```

Здесь `T` — это **обобщённый кортеж** аргументов, и TypeScript **выводит** точные типы.

🔄 Пример 4: Перегрузка через условный кортеж

```
type Overload =  
  | [string]  
  | [string, number]  
  | [string, number, boolean];  
  
function process(...args: Overload) {  
  // обрабатываем по длине кортежа  
  if (args.length === 1) {  
    const [a] = args;  
    // a: string  
  } else if (args.length === 2) {  
    const [a, b] = args;  
    // a: string, b: number  
  } else {  
    const [a, b, c] = args;  
    // a: string, b: number, c: boolean  
  }  
}
```


Тип `Overload` описывает все допустимые комбинации аргументов.

🧠 Пример 5: Частичное применение (каррирование)

```
type BinaryFn = (a: number, b: number) => number;

function curry<T extends any[], R>(fn: (...args: T) => R) {
  return (...args: T) => fn(...args);
}

const add: BinaryFn = (a, b) => a + b;

const curriedAdd = curry(add); // типы сохраняются: (a: number, b: number) => number
```

✅ Когда использовать

Задача	Решение
Переменное число аргументов	<code>...args: T[]</code>
Жёстко заданный порядок и типы	Кортеж: <code>[number, string]</code>
Повторное использование сигнатур	Объявить <code>type Args = [...]</code>
Типобезопасный вызов обобщённой функции	<code>T extends any[]</code> + <code>...args: T</code>

📌 Вывод

Остаточные параметры и кортежи:

- Позволяют строго типизировать **позиционные аргументы**
- Обеспечивают **гибкость и безопасность** для сложных сигнатур
- Отлично подходят для **обобщений, каррирования, перегрузок**

Используйте опциональные свойства `never` для моделирования исключающего ИЛИ

В TypeScript можно использовать **опциональные свойства с типом `never`** для моделирования **исключающего ИЛИ (XOR)** — то есть когда объект должен соответствовать **только одному** из возможных вариантов.

Что такое исключающее ИЛИ (XOR)?

XOR означает:

✅ **Либо** структура A **либо** структура B, ❌ **но не обе одновременно.**

Почему обычный `A | B` не всегда работает?

```
type A = { foo: string };
type B = { bar: number };

type AorB = A | B;

const value: AorB = { foo: "x", bar: 123 }; // ❌ допустимо! А мы хотим запретить
```

`A | B` позволяет объединить оба свойства — это **не настоящий XOR**.

✅ Решение: опциональные свойства `never`

Мы используем `never` в качестве **запретного типа**, а `?` — чтобы избежать обязательности:

XOR через утилиту:

```
type Without<T, U> = { [K in Exclude<keyof T, keyof U>]?: never };
```



```
type XOR<T, U> = (T & Without<U, T>) | (U & Without<T, U>);
```

Пример:

```
type WithEmail = { email: string };
type WithPhone = { phone: string };

type Contact = XOR<WithEmail, WithPhone>;
```

Теперь:

```
const a: Contact = { email: "test@example.com" }; // ✓
const b: Contact = { phone: "123456" };           // ✓
const c: Contact = {};                             // ✗ ни одно не указано
const d: Contact = { email: "test", phone: "123" }; // ✗ оба указаны!
```

📖 Как это работает?

```
type Without<U, T> = {
  [K in Exclude<keyof U, keyof T>]?: never;
};
```

- Without<WithPhone, WithEmail> = { phone?: never }
- WithEmail & { phone?: never } означает: 👉 email **обязательно**, ✗ phone **запрещено**

🎯 Когда использовать?

- При валидации конфигураций (file **или** url)
- При построении компонентов с разными режимами
- Для API, где нужно передать один из возможных вариантов

📌 Вывод

- Используйте never + ? для создания **взаимоисключающих** структур
- Это обеспечивает **настоящий XOR**

- Делает ваш код **строго типизированным** и **предсказуемым**

Маркировки для номинальной типизации

Маркировки (**brand types**) позволяют в TypeScript **моделировать номинальную типизацию**, несмотря на то, что язык использует **структурную типизацию** по умолчанию.

Что такое номинальная типизация?

Номинальная типизация означает:

*Два типа считаются разными, даже если их структура совпадает, **если у них разные имена**.*

TypeScript по умолчанию так **не делает**:

```
type UserId = string;
type OrderId = string;

function getUser(id: UserId) {}

const orderId: OrderId = "abc";
getUser(orderId); // ❌ Ожидали ошибку, но это сработает — типы одинаковые (оба string)
```

Решение: маркировка через ``& { __brand: '...' }``

```
type UserId = string & { __brand: 'UserId' };
type OrderId = string & { __brand: 'OrderId' };
```

Теперь TypeScript отличает типы:

```
declare function getUser(id: UserId): void;
declare function getOrder(id: OrderId): void;

const userId = "u123" as UserId;
const orderId = "o456" as OrderId;
```



```
getUser(userId);    //   
getUser(orderId);  //  Ошибка: тип несовместим
```

Как создать безопасно?

Лучше не использовать `as` напрямую в продакшене — создаём фабрики:

```
function createUserId(id: string): UserId {  
    return id as UserId;  
}
```

Где применяют?

Применение	Пример
ID разных сущностей	<code>UserId</code> , <code>OrderId</code>
Разные единицы	<code>Meters</code> , <code>Pixels</code> , <code>Seconds</code>
Скрытые ключи	<code>PrivateKey</code> , <code>PublicKey</code>
Безопасные обёртки	<code>SafeHTML</code> , <code>UnsafeHTML</code>

Почему это работает?

- Тип `string & { __brand: 'X' }` — **расширение** базового типа
- Это **невозможно создать случайно**, пока не используешь `as`
- TypeScript видит их как **несовместимые** типы, даже если они оба `string`

Вывод

 **Брендовые типы:**

- Используются для **номинальной типизации**
- Позволяют **различать одинаковые по структуре типы**
- Улучшают **безопасность**, особенно при работе с ID, единицами и ключами



Публикация и API (65-71)

Размещать TypeScript и @types в devDependencies

Размещение `typescript` и `@types/...` в `devDependencies` — это стандартная и правильная практика в большинстве TypeScript-проектов.



Краткий ответ

Помести `typescript` и `@types/...` в `devDependencies`, потому что они **нужны только при разработке и сборке**, а не при работе приложения в продакшене.



Что такое `devDependencies`?

В `package.json`:

```
"devDependencies": {
  "typescript": "^5.3.2",
  "@types/node": "^20.0.0",
  "@types/express": "^4.17.17"
}
```

- `devDependencies` — это пакеты, **не нужные в рантайме**, а только при **разработке, сборке, тестировании**.
- Они **не попадают в финальный продакшен-бандл**, если вы его создаёте, скажем, через webpack, esbuild, Vite или `tsc`.



Почему `typescript` — это devDependency?

Потому что:

- TypeScript-компилятор (`tsc`) нужен **только чтобы скомпилировать TS → JS**

- В продакшен-приложении TypeScript-код **уже скомпилирован**
- TypeScript **не нужен во время запуска приложения** — например, на сервере или в браузере



Почему `@types/...` — это devDependency?

- Пакеты `@types/...` — это **только типы**, без кода
- Они помогают редактору и компилятору, но в рантайме ничего не делают
- Например, `@types/express` помогает при написании кода с `express`, но в JS-файлы не попадает



Почему ****не стоит**** помещать в `dependencies`

- Иначе типы и компилятор будут устанавливаться **на продакшен-сервер**, где они не нужны
- Это **увеличивает размер** `node_modules`
- Увеличивает время установки зависимостей и сборки Docker-образов
- Может вызвать **лишние конфликты версий**



Исключения

Когда можно положить в `dependencies`?

Пишете **CLI-инструмент**, который компилирует TS во время выполнения

Используете `ts-node` **в production** (редко, но бывает)

Создаёте **библиотеку**, и вам нужно предоставить `@types/...` для потребителей



Вывод

Пакет	Куда?	Почему
<code>typescript</code>	<code>devDependencies</code>	Нужен только на этапе сборки
<code>@types/...</code>	<code>devDependencies</code>	Нужны только для компиляции и разработки
<code>express</code>	<code>dependencies</code>	Используется в рантайме

Проверять совместимость трех версий, задействованных в объявлениях типов

Фраза «Проверять совместимость трёх версий, задействованных в объявлениях типов» относится к важной практике в TypeScript-проектах, особенно когда используются сторонние библиотеки с типами из `@types`.

🧩 Что за «три версии»?

Когда вы используете библиотеку с типами, фактически участвуют **три компонента**:

1. **Библиотека** — основной npm-пакет (например, `express`)
2. **Типы библиотеки** — декларации из `@types/...` (например, `@types/express`)
3. **Версия TypeScript** — ваша установленная версия `typescript`

🎯 Почему это важно?

Каждая из этих версий может **по-своему эволюционировать**:

- У `express` может измениться API
- `@types/express` может обновиться для новой версии `express`
- В новых версиях `TypeScript` могут появиться изменения в поведении системы типов

⚠️ Несовместимость между этими тремя — частый источник **странных ошибок типов**.

📌 Пример проблемы

```
npm install express@4
npm install @types/express@5
```

В этом случае:

- Вы используете `express@4`
- Но установленные типы — для `express@5`

- В результате: TypeScript может **ошибочно подсвечивать** методы, которых нет, или жаловаться на несовпадения

✓ Как правильно проверять совместимость

1. 📦 Убедитесь, что версия `@types/...` подходит для библиотеки

В `package.json` типа (`@types/express`) обычно указана поддерживаемая версия через `dependencies`.

Например:

```
"dependencies": {
  "express": "^4.17.0"
}
```

Это означает, что типы подходят для `express@4.17.x`.

🔍 Можно посмотреть это через `npm info @types/express peerDependencies`.

2. 🛠 Убедитесь, что `@types/...` совместимы с вашей версией TypeScript

Типы в `@types/...` могут использовать новые возможности языка, несовместимые со старыми версиями TypeScript.

Например:

- `@types/react@18` может требовать `typescript >= 4.5`
- А у вас установлен `typescript@3.9` — и тогда типы начнут ломаться

3. 🛠 Заблокируйте версии в `package.json`

Для надёжности:

```
"dependencies": {
  "express": "4.18.2"
},
"devDependencies": {
  "@types/express": "4.17.21",
```



```
"typescript": "5.3.3"
}
```

Совет: настройка `package-lock.json` или `pnpm-lock.yaml`

- Убедитесь, что lock-файл фиксирует согласованные версии
- Используйте `npm ci` или `pnpm install --frozen-lockfile` для воспроизводимости

Вывод

«Проверять совместимость трёх версий» означает:

-  Убедитесь, что `@types/...` подходят к используемой версии библиотеки
-  Убедитесь, что они совместимы с вашей версией TypeScript
-  Зафиксируйте все три версии для стабильности и предсказуемости

Экспортировать все типы, которые появляются в открытом (публичном) API

Экспортировать все типы, которые появляются в открытом (публичном) API, Отличный и важный вопрос для построения **хорошо типизированных публичных интерфейсов!**

Цель

Экспортировать все типы, которые появляются в открытом (публичном) API, чтобы:

-  Потребители библиотеки могли использовать эти типы напрямую
-  IDE могла давать автокомплит и подсказки
-  Предотвратить дублирование и ошибки в использовании интерфейса

Что значит «открытый API»?

Обычно это:

- **Публичные функции и классы**, экспортируемые из библиотеки/модуля
- **Аргументы, возвращаемые значения, опции** — всё, что **видно снаружи**
- **События, контракты данных, объекты конфигурации**

Пример: библиотека или модуль

```
// math.ts
export function add(a: number, b: number): number {
  return a + b;
}
```

Если ты используешь сложный тип:

```
export interface AddOptions {
  a: number;
  b: number;
  precision?: number;
}

export function addWithOptions(opts: AddOptions): number {
  return parseFloat((opts.a + opts.b).toFixed(opts.precision ?? 2));
}
```

Надо экспортировать и `AddOptions`!

Так пользователь сможет делать:

```
import { addWithOptions, AddOptions } from 'your-lib';

const opts: AddOptions = { a: 1, b: 2, precision: 3 };
addWithOptions(opts);
```

Почему это важно

-  **Плохо**: типы определены, но не экспортированы → пользователь не может до них добраться
 -  **Хорошо**: типы экспортируются → переиспользуемость и надёжность
-

Практика: `index.ts` как точка входа

Если проект модульный:

```
// src/index.ts
export * from './math';
export * from './geometry';
```

И типы из `math.ts`, `geometry.ts` **должны быть экспортированы**, если они:

- используются в сигнатурах
- или представляют значимые сущности

Автоматизация (по желанию)

С помощью утилит вроде [ts-prune](#) можно находить:

- типы, **используемые, но не экспортированные**
- или **экспортированные, но не используемые**

При создании библиотеки (npm-пакета)

Важно:

- Все **public API-типы** должны быть в **export'ax**
- Указывай `"types"` в `package.json`:

```
{
  "main": "dist/index.js",
  "types": "dist/index.d.ts"
}
```

Повторим правило

Если тип участвует в API — он должен быть экспортирован.

✓ Чеклист

- ☒ Типы аргументов публичных функций — экспортированы
- ☒ Типы возвращаемых значений — экспортированы
- ☒ Типы конфигураций, событий, ошибок — экспортированы
- ☒ Псевдонимы и интерфейсы, используемые вне модуля — экспортированы

TSDoc для комментариев к API

TSDoc — это **спецификация комментирования API в TypeScript**, похожая на JSDoc, но с улучшенной типовой поддержкой и строгими правилами для генерации документации.

🔧 Что такое TSDoc?

TSDoc — это стандартизированный формат комментариев `/** ... */`, который:

- 💬 документирует **функции, интерфейсы, типы, классы**
- 🛠️ используется инструментами вроде `API Extractor`, `TypeDoc`, IDE
- 📦 помогает **генерировать документацию** и улучшает **автодополнение**

📖 Базовый синтаксис

```
/**
 * Вычисляет сумму двух чисел.
 *
 * @param a Первое число.
 * @param b Второе число.
 * @returns Сумма чисел.
 */
export function add(a: number, b: number): number {
  return a + b;
}
```

🧠 Куда писать комментарии?

К:

- `export function ...`

- `export interface ...`
- `export type ...`
- `export class ...`
- `export const ...` (если это функция или объект API)

🔧 Основные теги TSDoc

Тег	Описание
<code>@param</code>	Пояснение параметра функции
<code>@returns</code> / <code>@return</code>	Описание возвращаемого значения
<code>@remarks</code>	Дополнительная информация
<code>@example</code>	Пример использования
<code>@deprecated</code>	Указывает, что устарело
<code>@defaultValue</code>	Значение по умолчанию (для пропов)
<code>@see</code>	Ссылка на другие части API
<code>@throws</code>	Описание возможной ошибки
<code>@internal</code>	Скрыть от публичной документации

📦 Пример для интерфейса

```
/**
 * Параметры для функции отправки письма.
 */
export interface SendEmailOptions {
  /**
   * Кому отправляется письмо (email).
   */
  to: string;

  /**
   * Тема письма.
   */
  subject: string;

  /**
   * Тело письма (может быть HTML).
   */
}
```



```
body: string;
}
```

Пример с `@example`

```
/**
 * Форматирует имя пользователя.
 *
 * @param name Имя.
 * @returns Имя с заглавной буквы.
 *
 * @example
 * ```ts
 * formatName('анна'); // "Анна"
 * ```
 */
export function formatName(name: string): string {
  return name.charAt(0).toUpperCase() + name.slice(1);
}
```

Важно

- Комментарии **должны идти сразу перед сущностью**, к которой они относятся
- Для `@param`, `@returns` — параметры должны **совпадать по имени** с кодом
- Работает только с `/** ... */`, а не `//`

Инструменты, совместимые с TSDoc

Инструмент	Назначение
TypeDoc	Генерация HTML-документации
API Extractor	Создание <code>.d.ts</code> и API отчётов
VSCode / WebStorm	Показывают документацию в IDE

Структура хорошей документации


```
/**
 * Подключается к серверу.
 *
 * @param url URL сервера.
 * @param token Токен авторизации.
 * @returns Объект клиента API.
 *
 * @example
 * ```ts
 * const client = connect('https://api.com', 'my-token');
 * ```
 */
export function connect(url: string, token: string): ApiClient {
  // ...
}
```

✓ Резюме

- Используйте `/** ... */` перед экспортируемыми сущностями
- Следуйте TSDoc-формату с тегами: `@param`, `@returns`, `@example`, `@remarks`
- Интегрируйте с инструментами документации
- Обеспечьте авто-дополнение и документацию для пользователей API

Определять тип `this` в обратных вызовах, если он является частью API

В TypeScript очень важно **явно указывать тип `this`**, особенно если ты создаёшь **API с обратными вызовами (callbacks)**, где контекст `this` должен быть строго определён.

🔧 Как указать тип `this`` в функциях

TypeScript позволяет **указать `this` как первый параметр** функции (специальный синтаксис).

✓ Пример:

```
interface Button {
  text: string;

  /**
   * Регистрирует обработчик нажатия.
   * @param callback Обратный вызов с `this`, указывающим на кнопку.
   */
}
```



```
*/
onClick(this: Button, callback: () => void): void;
}
```

Теперь в реализации:

```
const button: Button = {
  text: "OK",
  onClick(this: Button, callback: () => void) {
    callback.call(this); // `this` гарантированно Button
  },
};
```

И когда пользователь пишет:

```
button.onClick(function () {
  console.log(this.text); // `this` — это Button!
});
```

➡ TypeScript **понимает тип** `this` **внутри функции**, если ты используешь `function`, а не стрелку!

❌ Почему нельзя использовать стрелочные функции?

Стрелочные функции **не имеют собственного** `this`. Пример:

```
button.onClick(() => {
  console.log(this.text); // ❌ 'this' здесь — внешний контекст, не Button
});
```

⚙️ Сигнатура с `this` в типе функции

Можно также описать тип обратного вызова отдельно:

```
type ClickHandler = (this: Button) => void;

interface Button {
```



```
onClick(handler: ClickHandler): void;
}
```

Куда это нужно?

- Обработчики DOM-like событий
- API, где пользователь предоставляет функции обратного вызова
- Fluent API и библиотеки с кастомным контекстом

Бонус: использование `bind`

Поскольку TypeScript знает, что `this: Button`, ты получаешь ошибку, если `this` не привязан:

```
button.onClick(function () {
  console.log(this.text); //  ОК
});

button.onClick(() => {
  console.log(this.text); //  Ошибка: this не Button
});
```

Вывод

- Используй `this: Type` как **первый неименованный параметр** в функции или методе
- Это работает только для обычных (`function`) функций, не для стрелочных
- Позволяет TypeScript гарантировать корректность контекста
- Очень важно для создания **типобезопасных API с callback'ами**

Зеркалирование типов для разрыва зависимостей

Зеркалирование типов — это продвинутая техника в TypeScript, которая используется для **разрыва зависимостей между модулями**, особенно при проектировании больших систем и библиотек. Она помогает сделать систему **менее связанной (decoupled)**, сохранив при этом типовую безопасность.



Что значит "зеркалировать типы"?

Зеркалирование типов — это когда:

- Один модуль **экспортирует только типовую форму** (без деталей реализации)
- Другой модуль **создаёт свои типы, отзеркаливая** или **переиспользуя** структуру внешних данных без прямого импорта реализации

Это особенно важно, когда:

- Нужно избежать циклических зависимостей
- Реализация "тяжелая", но нужен только тип
- Интерфейс создаётся до (или отдельно от) реализации



Как это выглядит на практике?



Цель: отделить типы от реализации

```
// types.ts — определяем интерфейс
export interface User {
  id: string;
  name: string;
}
```

```
// db.ts — реализация, может быть "тяжелая"
import type { User } from './types';

export function getUser(id: string): Promise<User> {
  // ...
}
```

```
// api.ts — хотим использовать тип User, но НЕ тянуть зависимости
import type { User } from './types';

type APIResponse = {
  data: User;
};
```



Мы "зеркалим" тип `User` в `api.ts`, **не импортируя** реализацию `db.ts`, а только **форму (тип)**.



Трюк: зеркалировать через `typeof`

Если у нас есть структура, но не хотим напрямую на ней зависеть:

```
// model.ts
export const model = {
  id: 0,
  name: "",
  age: 30,
};
```

В другом файле:

```
import type { model } from './model';

type Mirror = typeof model;
```



Мы создали **зеркальный тип** без импорта данных.



Альтернативы через маппинг

```
type Mirror<T> = {
  [K in keyof T]: T[K];
};
```

Это простое копирование типа — полезно для разделения типов и модификаций.



Пример для DTO/API

```
// domain/User.ts
export interface User {
  id: string;
  name: string;
  passwordHash: string;
}
```



```
// api/UserDTO.ts
import type { User } from '../domain/User';

// Зеркалим User без поля passwordHash
export type UserDTO = Omit<User, 'passwordHash'>;
```

🧩 Когда это полезно?

- При построении **слоистой архитектуры** (UI ⇌ API ⇌ бизнес-логика ⇌ база)
- При **модульных разделениях**, когда один слой знает о типах, но не об импортах
- В **библиотеках**, когда нужно предоставлять типы, но скрыть реализацию

✅ Советы

- Используй `import type` для явного импорта только типов
- Старайся **выносить типы в отдельные модули**
- Зеркаль через `typeof`, `Pick`, `Omit`, `Mapped Types`, чтобы не тянуть реализации
- Не бойся дублировать интерфейсы, если это снижает связность

Аугментацию модулей для улучшения типов

Аугментация модулей (module augmentation) в TypeScript — это способ **расширить или изменить типы, уже определённые в существующем модуле**, не редактируя сам модуль напрямую.

Это особенно полезно при:

- Добавлении типов к сторонним библиотекам (`express`, `react`, и т.д.)
- Улучшении типов в слабо типизированных модулях
- Расширении интерфейсов, объявленных в другом месте

🔧 Пример: Расширение типов `express.Request`

Допустим, вы хотите добавить в `Request` новое свойство:


```
// types/express.d.ts
import "express";

declare module "express" {
  interface Request {
    user?: { id: string; role: string };
  }
}
```

Теперь во всём проекте вы можете безопасно писать:

```
app.get("/profile", (req, res) => {
  console.log(req.user?.id); // ✅ типизировано!
});
```



Важно:

1. **Файл должен быть модулем** (иметь хотя бы один `import` / `export`)
2. Имя модуля в `declare module "..."` должно точно совпадать с именем импортируемого пакета



Пример с собственным модулем

```
// original.ts
export interface Config {
  debug: boolean;
}
```

```
// augment.ts
import "../original";

declare module "../original" {
  interface Config {
    port?: number;
  }
}
```

Теперь `Config` включает и `debug` , и `port` .

✂ Частый паттерн: расширение глобальных интерфейсов

```
declare global {  
  interface Window {  
    analytics: any;  
  }  
}
```

Тут `declare global` используется для расширения глобального пространства, без модуля.

Где применяется:

- Расширение `Jest`, `Cypress`, `Express`, `Next.js` API
- Добавление метаданных в `Request`, `Window`, `Error`
- Улучшение сторонних типов, если они неполные

Советы:

- Делай это в `.d.ts` файлах
- Используй `declare module` только для уже существующих модулей
- Не путай с **мерджингом деклараций** — он работает в пределах одного пространства имён (например, интерфейсы)

Отдавать предпочтение функциональности ECMAScript перед TypeScript

Когда говорят «Отдавать предпочтение функциональности ECMAScript перед TypeScript», имеют в виду:

Сначала использовать стандартные возможности JavaScript (ECMAScript), если они уже решают вашу задачу, и только потом — возможности TypeScript.

Почему?

TypeScript — это **надстройка над ECMAScript**. Он добавляет типизацию и другие удобства, но при компиляции превращается в обычный JavaScript.

Поэтому:

- Все, что можно выразить средствами **языка (ES)** — предпочтительно и **проще**.
- **Меньше TypeScript — проще сопровождать**.
- Если можно использовать стандарт — не нужно изобретать типовую надстройку.

Примеры

✅ Используйте `Array.prototype.map` вместо собственных утилит с типами:

Плохо (через сложные обобщения):

```
function mapArray<T, U>(arr: T[], fn: (value: T) => U): U[] {  
  return arr.map(fn);  
}
```


Лучше:

```
const names = users.map(u => u.name); // ✔ это уже типизировано стандартной библиотекой
```

✔ Используйте `Record` или `Map` вместо своей типовой структуры:

Плохо:

```
type MyDict = { [key: string]: number };
```

Лучше:

```
const scores = new Map<string, number>(); // ✔ стандартная структура
```

✔ Используйте `as const` вместо избыточных enum:

```
const roles = ['admin', 'editor', 'guest'] as const;  
type Role = typeof roles[number];
```

Вместо:

```
enum Role {  
  Admin = 'admin',  
  Editor = 'editor',  
  Guest = 'guest'  
}
```



Общая идея

- ECMAScript = поведение
- TypeScript = типы

Не нужно решать **поведение** через **типы**, если это уже есть в языке.

💬 Когда стоит использовать TypeScript-функциональность?

- Когда ECMAScript **не может гарантировать корректность** (например, проверка типов при компиляции)
- Когда ты **создаешь библиотеку** и тебе важно типовое API
- Когда нужна **дополнительная защита от ошибок**, которую JavaScript не даёт

✅ Вывод

Старайся **писать валидный, современный JavaScript**, используя возможности ES6+ (модули, `const`, `class`, `async/await`, `Array.prototype.*`, и т.д.).

Используй TypeScript **для усиления типовой безопасности, а не вместо логики**.

Source Map для отладки TypeScript

Source Map (карты исходников) позволяют отлаживать TypeScript-код **как будто это JavaScript**, но с возможностью видеть **оригинальный TypeScript-код** в отладчике (например, в Chrome DevTools, VS Code и других инструментах).

🧩 Что такое Source Map?

Когда TypeScript компилируется в JavaScript, он создаёт `.js`-файлы. Но отлаживать такие файлы неудобно — в них нет типов, исходных строк и часто всё минифицировано.

Source Map (`.map` файл) — это специальный файл, который связывает скомпилированный JavaScript с исходным TypeScript-файлом. Он сообщает отладчику:

"Эта строка в `.js` соответствует вот этой строке в `.ts`."

⚙️ Как включить Source Map

В `tsconfig.json`:


```
{
  "compilerOptions": {
    "sourceMap": true
  }
}
```

После компиляции TypeScript:

- Появится файл `example.js`
- И файл `example.js.map`
- Внутри `example.js` будет строка:

```
//# sourceMappingURL=example.js.map
```

Как использовать Source Map в браузере (например, Chrome DevTools)

1. Включи флаг `sourceMap` в `tsconfig.json`
2. Скомпилируй проект: `tsc`
3. Запусти приложение в браузере
4. Открой **DevTools** → **Sources**
5. Найди исходные `.ts`-файлы
6. Поставь точку останова (breakpoint) — и отладка пойдёт по TypeScript-строкам

Пример

```
// src/example.ts
function greet(name: string) {
  console.log(`Hello, ${name}`);
}
greet("Alice");
```

Компиляция с `sourceMap: true` даст:

- `example.js`
- `example.js.map`

В браузере ты увидишь `example.ts` и сможешь ставить брейкпоинты в оригинальной строке `console.log(...)`.

🛡️ Полезно для:

- Отладки в **Node.js** (с `--enable-source-maps` или `ts-node --inspect`)
- Разработки **фронтенда** (через Webpack/Vite/Parcel)
- Работа в **VS Code** — можно прямо шагать по `ts` -коду при запуске

🔥 Частые проблемы

Проблема	Решение
<code>.ts</code> не видно в DevTools	Убедись, что <code>sourceMap: true</code> , и <code>.map</code> подключён
Source map не найден	Проверь, что путь в <code>//# sourceMappingURL=</code> корректный
Ошибка при запуске в Node	Добавь <code>--enable-source-maps</code> или используй <code>ts-node</code>
Webpack не показывает исходник	Проверь <code>devtool: 'source-map'</code> в конфиге

✅ Вывод

Использование Source Map — это **базовая практика**, если ты хочешь:

- Отлаживать TypeScript так же легко, как JS
- Не теряться между скомпилированным и исходным кодом
- Упростить себе жизнь при разработке

Реконструкция типов на стадии выполнения

Реконструкция типов на стадии выполнения (runtime type reconstruction) в TypeScript — **не тривиальная задача**, потому что TypeScript **удаляет типы при компиляции**. Типы существуют **только во время разработки** и исчезают при запуске.

Но ты всё же можешь *реконструировать или проверить типы в рантайме* с помощью нескольких подходов:

Главный принцип:

TypeScript не предоставляет runtime-типов. Чтобы «реконструировать» тип на стадии выполнения — нужно использовать:

- Явные описания схем
- Библиотеки валидации
- Метаинформацию (например, через декораторы)

Способы реконструкции типов

1. Использование библиотек валидации + типизация

Пример с `zod`:

```
import { z } from 'zod';

const UserSchema = z.object({
  id: z.string(),
  age: z.number().int(),
});

type User = z.infer<typeof UserSchema>; //  тип User получается из runtime-схемы

const input = JSON.parse(someJSON);
const user = UserSchema.parse(input); // Runtime-проверка + TypeScript-типизация
```

Здесь схема задаёт **runtime**-поведение, и TypeScript «реконструирует» тип через `z.infer`.

2. Ручная проверка структуры объектов

```
function isUser(obj: any): obj is { name: string; age: number } {
  return typeof obj?.name === 'string' && typeof obj?.age === 'number';
}
```

Ты можешь использовать это в рантайме, чтобы "восстановить" тип:


```
if (isUser(something)) {
  // TypeScript теперь знает, что something — это User
  console.log(something.name);
}
```

3. 🧩 Использование декораторов + Reflect Metadata

(подходит для классов и DI)

```
import 'reflect-metadata';

class User {
  constructor(
    public name: string,
    public age: number
  ) {}
}

function logParamTypes(target: any) {
  const types = Reflect.getMetadata('design:paramtypes', target);
  console.log(types); // [String, Number]
}

@logParamTypes
class Example {
  constructor(name: string, age: number) {}
}
```

Это позволяет **считывать типы параметров конструктора на этапе выполнения**, если ты используешь `emitDecoratorMetadata`.

🔄 Идея: тип → схема → валидация → тип

С TypeScript это можно сделать двухсторонне:

Способ	Стадия	Что даёт
<code>zod</code> , <code>io-ts</code> , <code>yup</code> , <code>runtypes</code>	Runtime	Схема + проверка
<code>z.infer<typeof schema></code>	Compile-time	Типы на основе схем
Проверка вручную (<code>isX()</code>)	Runtime	Локальная проверка и narrowing



Когда нужно реконструировать тип?

- При приёме данных **из JSON / API / файла**
- Для построения **валидаторов, форм, UI**
- В DSL или системах плагинов
- В фреймворках с декораторами (`nestjs` , `class-validator`)



Рекомендуемые библиотеки

Библиотека	Особенности
<code>zod</code>	Простая, безопасная, популярная
<code>io-ts</code>	Мощная, но сложнее
<code>runtypes</code>	Простой синтаксис
<code>superstruct</code>	Отлично подходит для UI-валидации
<code>class-validator</code> + <code>class-transformer</code>	Для классов (часто в <code>nestjs</code>)

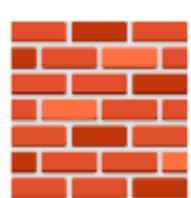


Вывод

*TypeScript не может сам по себе «реконструировать» тип в runtime, но ты можешь **синхронизировать runtime-валидацию и типы** с помощью библиотек или вручную через *guards*, *схемы*, или декораторы.*

Иерархия DOM (Document Object Model)

Разобраться с **иерархией DOM в контексте TypeScript** — значит научиться работать с DOM-элементами так, чтобы **использовать преимущества строгой типизации**: получать автодополнение, ловить ошибки на этапе компиляции и делать код понятнее и безопаснее.



Что такое DOM в контексте TypeScript?

- DOM — это объектная модель HTML-документа.
- TypeScript **знает типы DOM-элементов** благодаря встроенным типам (`lib.dom.d.ts`).

- Это позволяет использовать строго типизированные свойства и методы DOM-элементов.

Как получить доступ к DOM-элементам

♦ Способы доступа:

```
// Явный тип (вручную)
const inputEl: HTMLInputElement | null = document.querySelector("input");

// Или с явным приведение
const inputEl = document.querySelector("input") as HTMLInputElement;
```

♦ Проверка на null:

```
if (inputEl) {
  inputEl.value = "Hello, world!";
}
```

♦ Или через non-null assertion (используй осторожно!):

```
(document.querySelector("input") as HTMLInputElement).value = "Hello";
```

Как работает иерархия DOM в типах

В TypeScript DOM представлен как иерархия интерфейсов:

```
EventTarget
├─ Node
│   └─ Element
│       └─ HTMLElement
│           └─ HTMLInputElement, HTMLDivElement, etc.
```

Пример:

```
const el = document.querySelector("input");

if (el instanceof HTMLInputElement) {
```



```
    el.value = "text"; // доступно, т.к. TypeScript знает: это input
  }
```

🔧 Работа с детьми, родителями, и соседями

```
const el = document.querySelector("ul");

if (el) {
  for (const li of el.children) {
    console.log((li as HTMLLIElement).textContent);
  }

  const parent = el.parentElement; // HTMLElement | null
  const next = el.nextElementSibling; // Element | null
  const firstChild = el.firstChild; // Element | null
}
```

🔄 Пример: обход DOM-дерева с типами

```
function walkTree(node: Element, depth = 0): void {
  console.log("  ".repeat(depth) + node.tagName);

  for (const child of node.children) {
    walkTree(child, depth + 1);
  }
}

const root = document.body;
walkTree(root);
```

✅ Типы элементов

HTML-тег	ТипScript тип
<div>	HTMLDivElement
	HTMLSpanElement
<input>	HTMLInputElement
<form>	HTMLFormElement

Любой тег

HTMLElement или Element

⚠️ Что важно:

- Проверь **null**, особенно после `querySelector`
- Используй конкретные типы, например `HTMLInputElement`, а не `any`
- Уточняй тип через `instanceof` или `as`, если нужно
- Пользуйся подсказками редактора — они основаны на типах DOM

💡 Пример: безопасное взаимодействие с формой

```
const form = document.querySelector("form");
const input = document.querySelector("input[name=username]");

if (form instanceof HTMLFormElement && input instanceof HTMLInputElement) {
  form.addEventListener("submit", (e) => {
    e.preventDefault();
    console.log("Username:", input.value);
  });
}
```



Заключение

В контексте TypeScript иерархия DOM — это не только дерево элементов на странице, но и **иерархия интерфейсов**, которая позволяет тебе:

- обращаться к HTML-элементам типобезопасно;
- использовать строгую проверку;
- сокращать количество багов и получать автодополнение;
- легче читать и поддерживать код.

Точная модель среды выполнения

Создание **точной модели среды выполнения (runtime environment)** в контексте TypeScript — это процесс описания и проверки того, **что реально происходит во время исполнения** программы, особенно при взаимодействии с внешними данными, API, браузером

или файловой системой. TypeScript работает на этапе компиляции, но не гарантирует, что значения в рантайме соответствуют типам. Поэтому нужно **согласовать типы и поведение рантайма**.

Зачем это нужно?

- Убедиться, что данные, полученные из внешних источников, действительно соответствуют ожидаемым типам.
 - Защититься от багов, вызванных расхождением между типами и реальными значениями.
 - Сделать поведение приложения более предсказуемым.
-

Шаги по созданию точной модели среды выполнения

1. ****Опиши ожидаемую структуру данных в типах****

```
type User = {  
  id: number;  
  name: string;  
  email?: string;  
};
```

2. ****Проверь данные на соответствие типу (валидация)****

TypeScript *не проверяет* соответствие типу на этапе исполнения, поэтому нужно использовать:

Либо собственную проверку:

```
function isUser(data: unknown): data is User {  
  return typeof data === "object" &&  
    data !== null &&  
    typeof (data as any).id === "number" &&  
    typeof (data as any).name === "string";  
}
```

Либо использовать библиотеку валидации с выводом типов:

- `zod`
- `io-ts`
- `runtypes`

Пример с Zod:

```
import { z } from "zod";

const UserSchema = z.object({
  id: z.number(),
  name: z.string(),
  email: z.string().optional(),
});

type User = z.infer<typeof UserSchema>;

const userData = JSON.parse(apiResponse);

const parsed = UserSchema.safeParse(userData);

if (parsed.success) {
  const user: User = parsed.data;
  console.log(user.name);
} else {
  console.error("Invalid user data", parsed.error);
}
```

3. ****Моделируй API/систему строго****

Создавай типы, которые отражают реальные состояния среды выполнения, а не желаемые.

- Не делай поля обязательными, если они могут отсутствовать.
- Учитывай `null`, `undefined`, ошибки.

```
type FetchResult<T> =
  | { status: "ok"; data: T }
  | { status: "error"; error: string };
```

4. ****Сопровождай типы runtime-валидацией****

Если ты используешь `as Type` без проверки, то вводишь **ложную уверенность**. Лучше сделать проверку.

5. ****Не полагайся на типы для сторонних данных****

TypeScript не знает, что пришло из API, базы или `localStorage`. Доверяй — но проверяй.

6. ****Учитывай платформенные особенности****

Типы могут отличаться между:

- Node.js и браузером
- SSR и CSR
- Разными API-браузерами (например, `navigator` , `File` , `Clipboard`)

Создай условные типы или абстракции, если поведение разное.

Краткий чеклист: точная модель среды

- ☒ Описаны ожидаемые типы (`type` / `interface`)
- ☒ Есть проверка данных в рантайме (ручная или через `zod` , `io-ts`)
- ☒ Все внешние источники данных валидируются
- ☒ Типы моделируют **реальные состояния**, включая ошибки
- ☒ Есть тесты на невалидные входные данные

Если коротко

***Точная модель среды выполнения** — это когда твои типы в TypeScript реально отражают, что может быть в рантайме, и ты проверяешь это на практике.*

Проверка типов и юнит-тестирование

Связь между **проверкой типов** и **юнит-тестированием** — в их общей цели: 🔍 **поймать ошибки до выполнения кода**, но делают они это по-разному, **дополняя, а не заменяя друг друга**.

В чём разница?

Критерий	Проверка типов (TypeScript)	Юнит-тестирование
Что проверяет	Статическую корректность типов	Поведение кода при разных входных данных
Когда срабатывает	На этапе компиляции	На этапе выполнения

Что ловит	Ошибки типов, несовместимость структур	Ошибки логики, баги, неверные результаты
Гарантирует ли работу?	❌ Нет (не проверяет поведение)	✅ Да (проверяет конкретные случаи)

🔗 Как они связаны?

1. ****Типы → Меньше необходимости в тестах на формы данных****

Типы проверяют структуру и сигнатуры функций. Пример:

```
type User = { name: string; age: number };
```

Если ты передашь `{ name: "Alice", age: "25" }`, TypeScript тут же скажет: ❌ `"25"` не `number`.

→ Это устраняет целый класс багов, которые раньше ловились только через тесты.

2. ****Типы помогают писать тесты****

Типы дают тебе информацию о:

- возможных значениях входов
- вариантах поведения (например, `Result<Ok, Err>`)
- исключениях и `null`-состояниях

→ Ты точно знаешь, какие случаи надо покрыть.

3. ****Типы ≠ поведение****

Вот где нужны юнит-тесты.

```
function isAdult(user: User): boolean {  
  return user.age > 18; // логическая ошибка, должно быть >=  
}
```

Тип `User` может быть корректным, но поведение — неправильным. Только юнит-тест его поймает.

4. **Типы могут "подсказывать" тест-кейсы**

```
type Result = "success" | "error";

function handle(result: Result) { ... }
```

→ Ты знаешь, что нужно написать два теста: один для `"success"`, один для `"error"`.

🔧 Лучшее использование — вместе

```
type LoginResponse =
  | { status: "ok"; token: string }
  | { status: "error"; message: string };
```

- ✓ Тип гарантирует, что `status` — только `"ok"` или `"error"`
- ✓ Юнит-тесты проверяют:
 - что при `status: "ok"` мы сохраняем токен
 - что при `"error"` показываем сообщение

📌 Вывод

TypeScript ≠ тесты.

- Типы — это **проверка формы** данных, сигнатур и совместимости.
- Тесты — это **проверка поведения** и логики.

Они **дополняют друг друга**:

- TypeScript снижает количество нужных тестов,
- но не заменяет их.

Быстродействие компилятора

📦 **TypeScript-компилятор (tsc)** может тормозить на больших проектах. Чтобы **уделить внимание быстродействию компилятора**, нужно понимать, **что замедляет сборку**, и как это контролировать.

Кратко: как ускорить компиляцию TypeScript

1. Разделяй проект на модули (project references)
2. Отключи ненужные фичи в `tsconfig.json`
3. Избегай тяжёлых типов
4. Используй `tsc --incremental` и `--watch`
5. Следи за структурными типами
6. Профилируй производительность (используй `--diagnostics`)

1. Разделение на проекты (`Project References`)

Для больших монорепозиториях или приложений:

- Делай несколько `tsconfig.json`
- Каждому под-проекту — свой сборочный кэш

```
// tsconfig.json
{
  "files": [],
  "references": [{ "path": "../packages/core" }, { "path": "../packages/ui" } ]
}
```

🕒 Позволяет собирать только изменившиеся части проекта.

2. Настрой `tsconfig.json` для скорости

Опция	Эффект
<code>"skipLibCheck": true</code>	 Пропускает проверку типов в <code>node_modules</code>
<code>"noEmit": true</code>	 Только проверка типов, без генерации JS
<code>"incremental": true</code>	 Кэширует промежуточные результаты
<code>"isolatedModules": true</code>	⚡ Требуется для <code>swc</code> , <code>vite</code> , <code>next</code> и пр.

```
{
  "compilerOptions": {
    "skipLibCheck": true,
    "incremental": true,
    "noEmit": true,
```



```
    "strict": true
  }
}
```

🧠 3. Избегай сложных и дорогих для проверки типов

TypeScript может «зависать» на:

- Больших **рекурсивных типах**
- Использовании **условных типов** внутри **union**’ов
- **Map-like** структур, моделируемых через `Record<string, T>`
- Очень глубоко вложенных `Partial<Deep<...<T>>>`

🔧 Советы:

- Упростить типы
- Избегать чрезмерной генерации типов (в `node_modules`, `graphql-codegen`, `zod`, `io-ts`, и пр.)
- Проверять время с помощью `--extendedDiagnostics`

👁️ 4. Используй флаг `--watch` и `--incremental`

Они сильно ускоряют локальную разработку, потому что:

- Кэшируют уже проверенные файлы
- Проверяют только изменившиеся

```
tsc --watch
```

📊 5. Включи `--diagnostics` для анализа

```
tsc --diagnostics
```

Ты получишь:

```
Files:          980
Lines:        10593
```


Nodes:	68456
Identifiers:	21345
Symbols:	56235
Types:	78291
Memory used:	188122 KB
I/O read:	0.07s
I/O write:	0.02s
Parse time:	0.18s
Bind time:	0.12s
Check time:	1.52s
Emit time:	0.00s
Total time:	1.82s

🕒 Это поможет найти узкие места.

🧹 6. Удали неиспользуемые декларации

Иногда `*.d.ts` или генерируемые типы в `node_modules` увеличивают нагрузку. Можно:

- Удалить неиспользуемые пакеты
- Использовать `@ts-ignore` осознанно
- Отключить типы для ненужных зависимостей через `types: []`

🧰 Инструменты для быстрогодействия

Инструмент	Назначение
<code>[tsc --watch]</code>	Инкрементальная проверка
<code>[ts-patch]</code>	Патчи для TypeScript (например, babel-like плагины)
<code>[esbuild , swc]</code>	Быстрая трансляция, если не нужен type-check

✅ Вывод

TypeScript-компилятор тормозит, если:

- много непроверенных библиотек,
- тяжелые типы,
- плохо структурированный `tsconfig`.

Что делать:

- Разделить проект (references)
- Включить `--incremental`, `--skipLibCheck`
- Упростить сложные типы
- Использовать `--diagnostics` для анализа

Использовать современный JavaScript

Писать **современный JavaScript** — значит использовать язык эффективно, понятно и безопасно, **в духе последних стандартов**. Ниже — компактное руководство.

1. Используй ES6+ синтаксис

✓ **let / const** вместо `var`:

```
const name = "Alice"; // неизменяемое
let age = 30;          // изменяемое
```

✓ **Стрелочные функции:**

```
const greet = (name) => `Hello, ${name}`;
```

✓ **Деструктуризация:**

```
const user = { name: "Bob", age: 25 };
const { name, age } = user;
```

✓ **Шаблонные строки:**

```
const message = `Hi, ${name}!`;
```

✓ **Модули (ESM):**

```
// greet.js
export const greet = (name) => `Hi, ${name}`;
```



```
// app.js
import { greet } from './greet.js';
```

2. Работай с асинхронностью через `async/await`

✓ Вместо вложенных `then`:

```
async function fetchUser(id) {
  const res = await fetch(`/api/user/${id}`);
  const user = await res.json();
  return user;
}
```

3. Используй современные коллекции

- `Map`, `Set`, `WeakMap`, `WeakSet` — для сложных структур
- `Array.prototype.map`, `filter`, `reduce` — для работы с массивами

4. Пиши декларативно

✓ Вместо императивных циклов:

```
const names = users.map(u => u.name).filter(n => n.startsWith("A"));
```

5. Придерживайся хорошего стиля

- Используй [Prettier](#) и [ESLint](#)
- Именуй переменные понятно (`user`, `isLoggedIn`, `fetchData`)
- Разделяй логику по функциям и модулям

6. Используй новые фишки стандарта (ECMAScript)

- `??` — нулевое слияние (`a ?? b`)
- `?.` — опциональная цепочка (`obj?.user?.name`)
- `Object.fromEntries()` и `Object.entries()`
- `Array.flat()` и `flatMap()`
- `Promise.allSettled()` и `Promise.any()`

7. Избегай старого и небезопасного

- ❌ `var`, `==`, `function` в стиле ES5
- ❌ Глобальные переменные
- ❌ Неиспользуемый код

8. Автоматизируй сборку и трансформации

- Используй [Babel](#) или [SWC](#) — если нужен старый браузер
- Используй [Vite](#), [ESBuild](#) или [Rollup](#) для сборки

9. Покрывай код тестами

- Пиши юнит-тесты с [Jest](#) или [Vitest](#)
- Используй [Playwright](#) или [Cypress](#) для E2E

10. Пиши с оглядкой на TypeScript

Даже если пишешь чистый JS, можешь:

- Добавлять JSDoc-комментарии
- Включить `checkJs` в `tsconfig.json`
- Получить автодополнение и проверку типов

✨ Хочешь краткую памятку? Вот:

```
// Современный стиль
const greet = ({ name }) => `Hi, ${name ?? "guest"}`;

const result = await fetch(url).then(r => r.json()).catch(console.error);
```



```
const usersMap = new Map(users.map(u => [u.id, u]));
```

ECMAScript-модули (ESM)

Модули позволяют **разделять код по файлам** и делать его **переиспользуемым, изолированным и читабельным**.

✓ Как экспортировать

```
// math.js
export const add = (a, b) => a + b;
export const PI = 3.14;
```

✓ Как импортировать

```
// app.js
import { add, PI } from './math.js';

console.log(add(2, 3), PI);
```

✓ Экспорт по умолчанию

```
// logger.js
export default function log(msg) {
  console.log(`[LOG]: ${msg}`);
}
```

```
// app.js
import log from './logger.js';
log('Hello!');
```

 **Важно:** В `package.json` указывай `"type": "module"` — если хочешь использовать ESM в Node.js. Или используй расширение `.mjs`.

Классы вместо прототипов

Раньше JavaScript использовал прототипное наследование напрямую:

```
function Person(name) {  
  this.name = name;  
}  
Person.prototype.greet = function () {  
  return `Hi, I'm ${this.name}`;  
};
```

✓ Современный синтаксис: `class`

```
class Person {  
  constructor(name) {  
    this.name = name;  
  }  
  
  greet() {  
    return `Hi, I'm ${this.name}`;  
  }  
}
```

Наследование:

```
class Student extends Person {  
  constructor(name, grade) {  
    super(name);  
    this.grade = grade;  
  }  
  
  greet() {  
    return `${super.greet()} and I'm in grade ${this.grade}`;  
  }  
}
```

Почему это важно?

- ✓ **Классы** читаются проще, чем работа с `prototype`.
 - ✓ **Модули** позволяют избегать глобальных переменных и улучшают масштабируемость.
 - ✓ Поддерживаются во всех современных браузерах и в Node.js.
-

Было (устаревшее)	Стало (современное)
<code>require(...)</code> / <code>module.exports</code>	<code>import</code> / <code>export</code>
Функции-конструкторы	<code>class</code> и <code>extends</code>
Глобальные переменные	Модули с локальным пространством
Ручное подключение файлов	Модули автоматически по <code>import</code>

@ts-check и JSDoc в TypeScript

Если ты хочешь **писать JavaScript**, но **пользоваться преимуществами TypeScript** (например, автодополнением и проверкой типов) **без полной миграции в .ts-файлы**, то комбинация `@ts-check` и **JSDoc** — идеальный инструмент для экспериментов и постепенного внедрения.

✓ Что такое `@ts-check`?

Это директива, которую ты пишешь в начале `.js` файла:

```
// @ts-check
```

Она включает **проверку типов** TypeScript прямо в JavaScript-файле.

✓ Что такое JSDoc?

Это специальные комментарии, которые позволяют **аннотировать типы** в JavaScript.

Пример:

```
// @ts-check

/**
 * Суммирует два числа
 * @param {number} a
 * @param {number} b
 * @returns {number}
```



```
*/  
function add(a, b) {  
  return a + b;  
}
```

Теперь редактор (VSCode, WebStorm) покажет автодополнение, ошибки и документацию — как будто ты пишешь на TypeScript.

Примеры аннотаций с JSDoc

Типизация переменной

```
/** @type {string} */  
let name = "Alice";
```

Массивы

```
/** @type {number[]} */  
const scores = [10, 20, 30];
```

Объекты

```
/** @type {{ x: number, y: number }} */  
const point = { x: 5, y: 10 };
```

Обобщения (дженерики)

```
/** @type {Array<string>} */  
const names = ['Alice', 'Bob'];
```

Как включить поддержку `@ts-check` в проекте

1. В редакторе (например, VSCode) всё начнёт работать из коробки.
2. В `jsconfig.json` или `tsconfig.json` можно явно включить:


```
{
  "compilerOptions": {
    "checkJs": true
  },
  "include": ["src/**/*.js"]
}
```

Рекомендации

- Используй `@ts-check` в каждом `.js`-файле, где хочешь проверку.
- Используй JSDoc-теги: `@param`, `@returns`, `@typedef`, `@type`, `@template`
- Подходит для **прототипирования, миграции, или библиотек без TS**.

Пример с `@typedef`

```
// @ts-check

/**
 * @typedef {Object} User
 * @property {string} name
 * @property {number} age
 */

/** @type {User} */
const user = {
  name: "Alice",
  age: 30,
};
```

Когда это полезно

- Пробуешь TypeScript без миграции
- Работаешь в JS, но хочешь автодополнение и проверку
- Делаешь демо, эксперименты, код-песочницы
- Улучшаешь типобезопасность в старом JS-коде

allowJs для совмещения TypeScript и JavaScript

Опция `allowJs` в TypeScript позволяет **использовать JavaScript-файлы (`.js`) внутри проекта на TypeScript**. Это особенно полезно при:

- ✔ поэтапной миграции с JavaScript на TypeScript
- ✔ работе со сторонними библиотеками или устаревшим кодом
- ✔ создании проектов, где TS и JS сосуществуют

✔ Как включить `allowJs`

В `tsconfig.json`:

```
{
  "compilerOptions": {
    "allowJs": true,
    "checkJs": true,      // (опционально) включает проверку типов в .js-файлах
    "outDir": "./dist",   // укажи куда собирать
    "target": "ES2020",   // пример — на твой выбор
    "module": "ESNext"
  },
  "include": ["src/**/*.ts"]
}
```

🧩 Разница между `allowJs` и `checkJs`

Опция	Что делает
<code>allowJs</code>	Позволяет компилятору читать <code>.js</code> файлы
<code>checkJs</code>	Включает проверку типов в <code>.js</code>

Ты можешь использовать их вместе или по отдельности:

- ✔ `allowJs: true` + `checkJs: false` → просто разрешаешь `.js`, без проверок
- ✔ `allowJs: true` + `checkJs: true` → и разрешаешь, и проверяешь

🧪 Пример смешанного кода

math.js (JavaScript с JSDoc)

```
// @ts-check

/**
 * @param {number} x
 * @param {number} y
 */
export function add(x, y) {
  return x + y;
}
```

main.ts (TypeScript)

```
import { add } from './math.js';

console.log(add(2, 3));
```

TypeScript понимает типы из JSDoc и даже проверяет ошибки в `.js`, если `checkJs: true`.

 Дополнительные советы

- Убедись, что `allowJs` используется с `include`, чтобы не компилировать сторонние `.js` (например, в `node_modules`)
- Лучше **не писать новый код в .js**, если уже используешь TypeScript — но `allowJs` отлично помогает при **миграции**
- Если не хочешь, чтобы `.js` компилировались, добавь `"exclude": ["**/*.js"]`, но оставь `allowJs: true` для чтения и анализа

 TL;DR

Хочешь...	Делай это
Использовать <code>.js</code> в проекте TypeScript	<code>"allowJs": true</code>
Проверять типы в <code>.js</code>	<code>"checkJs": true</code>
Ускорить миграцию с JS на TS	Комбинируй <code>allowJs</code> , JSDoc и <code>@ts-check</code>
Постепенно переписывать <code>.js</code> в <code>.ts</code>	Начинай с JSDoc и <code>checkJs</code>

Конвертируйте модуль за модулем вверх по графу зависимостей

Фраза «**Конвертируйте модуль за модулем вверх по графу зависимостей**» означает подход к **постепенной миграции проекта с JavaScript на TypeScript**, в котором ты начинаешь переводить модули **от "листьев" к "корню"** в **графе зависимостей**.



Что такое граф зависимостей?

Это способ представить, **как модули зависят друг от друга**:

- Если `A` импортирует `B`, то `A` зависит от `B`.
- Значит, в графе есть стрелка: `A → B`.

Пример:

```
app.ts
├─ service.ts
│   ├─ utils/math.js
│   └─ utils/logger.js
```

Граф зависимостей:

```
app → service → math, logger
```



Что значит «вверх»?

Начни миграцию **с самых нижних модулей**, у которых **нет или почти нет зависимостей** — они внизу графа. Это проще, потому что:

- они изолированы,
- их можно легко типизировать,
- не требуют сначала переписывать всё, что их использует.

Потом продвигайся выше — к тем модулям, которые **используют уже типизированные**.



Почему это работает?

- Ты **не ломаешь весь проект сразу**.
- Модули, которые ты уже переписал в `.ts`, предоставляют **более точные типы** наверх.

- Это помогает TypeScript **лучше понимать остальной код** и находить ошибки.

✓ Пример пошаговой миграции

1. Начни с **утилит** — `math.js`, `logger.js` → переименуй в `.ts`, добавь типы.
2. Перейди к модулям, которые используют их — `service.ts`.
3. Наконец, перейди к верхнему уровню — `app.ts`.

🔄 Итог

"Конвертировать вверх по графу" = Сначала типизируй модули, от которых все зависят, а потом те, которые используют уже типизированные.

Не считайте миграцию завершённой, пока не включите `noImplicitAny`

«Не считайте миграцию завершённой, пока не включите `noImplicitAny`» — важное правило при переходе на TypeScript.

Что такое `noImplicitAny`?

Это опция компилятора TypeScript (`tsconfig.json`), которая запрещает **неявное присвоение типа** `any`.

То есть:

- Если TypeScript не может вывести точный тип для переменной, параметра или возвращаемого значения функции —
- и ты **не указал тип явно**,
- компилятор выдаст ошибку вместо того, чтобы подставить `any` автоматически.

Почему это важно?

- `any` — «чёрная дыра» для типизации, которая отключает проверку для этой части кода.

- Включив `noImplicitAny`, ты **заставляешь себя или команду явно прописывать типы и следить за точной типизацией**.
- Это помогает найти потенциальные баги, которые могли бы «спрятаться» из-за `any`.

Как выглядит ошибка без `noImplicitAny`

```
function greet(name) { // name: any (неявно)
  return `Hello, ${name.toUpperCase()}`; // может упасть, если name не строка
}
```

При включенном `noImplicitAny` TS скажет:

Parameter 'name' implicitly has an 'any' type.

Когда включать `noImplicitAny`?

- Только после того, как **основная миграция сделана**, и большая часть кода уже типизирована.
- Иначе ты получишь слишком много ошибок, и миграция станет мучительной.

Итого

До включения <code>noImplicitAny</code>	После включения
Много неявных <code>any</code> — ошибки не видны	Явное объявление всех типов
Легко написать некорректный код	Код становится безопаснее и понятнее

Вывод: не останавливай миграцию, пока не заставишь компилятор требовать явных типов для всего — это гарантия качественного, безопасного кода.



Справочные материалы

Базовые типы

Вот основные:

Тип	Пример значения	Описание
<code>string</code>	<code>"привет"</code>	Строка
<code>number</code>	<code>42</code> , <code>3.14</code>	Число (любое, нет разделения int/float)
<code>boolean</code>	<code>true</code> , <code>false</code>	Логическое значение
<code>null</code>	<code>null</code>	Ничего
<code>undefined</code>	<code>undefined</code>	Не определено
<code>symbol</code>	<code>Symbol("id")</code>	Уникальный идентификатор (редко)
<code>bigint</code>	<code>123n</code>	Большое целое число

✓ Примеры:

```
let name: string = "Alice";
let age: number = 30;
let isAdmin: boolean = true;
let nothing: null = null;
let notSet: undefined = undefined;
let id: symbol = Symbol("id");
let big: bigint = 1000000000000000000000000n;
```



Особенности

1. ``string`` и ``String`` — не одно и то же


```
let a: string = "hello"; // ✅ правильно
let b: String = new String("hi"); // ⚠️ объект-обёртка — избегай!
```

👉 **Используй строчные типы** (`string`, `number`, `boolean`), а не объектные (`String`, `Number`, ...).

2. `null` и `undefined` — по умолчанию запрещены, если не указаны явно:

```
let name: string = null; // ❌ ошибка, если strictNullChecks включён

// Правильно:
let name: string | null = null;
```

3. Литеральные типы

```
let dir: "left" | "right" = "left"; // можно только "left" или "right"
```

🧠 TypeScript добавляет смысл

*JavaScript — динамический, а TypeScript позволяет добавить **аннотации типов**, чтобы защититься от ошибок на этапе компиляции.*

Пример без TypeScript:

```
function greet(name) {
  return "Hi, " + name.toUpperCase();
}
```

С TypeScript:

```
function greet(name: string): string {
  return "Hi, " + name.toUpperCase();
}
```



```
}
```

Резюме

Тип	Когда использовать
<code>string</code>	Для строк
<code>number</code>	Для любых чисел
<code>boolean</code>	Истина/ложь
<code>null</code> / <code>undefined</code>	Если явно нужно отсутствие значения
<code>symbol</code>	Для уникальных ключей
<code>bigint</code>	Для очень больших чисел

Дженерики

Дженерики (Generics) в TypeScript — это мощный инструмент, позволяющий создавать **повторно используемый, гибкий и типобезопасный код**. Они позволяют описывать функции, классы и интерфейсы, которые работают с разными типами данных, сохраняя при этом строгую типизацию.

◆ Зачем нужны дженерики?

Допустим, ты хочешь написать функцию, которая просто возвращает переданный ей аргумент:

```
function identity(arg: any): any {  
  return arg;  
}
```

Это работает, но теряется информация о типе: на вход был `string`, но TypeScript уже этого не знает.

Решение: использовать **дженерики**!


```
function identity<T>(arg: T): T {  
  return arg;  
}
```

Теперь тип `T` — это **параметр типа**, он определяется во время вызова функции.

♦ Примеры использования

1. Функция с дженериком

```
function identity<T>(arg: T): T {  
  return arg;  
}  
  
let output1 = identity<string>("Привет");  
let output2 = identity<number>(42);  
let output3 = identity(true); // TypeScript сам выведет T = boolean
```

2. Работа с массивами

```
function logArrayLength<T>(arr: T[]): void {  
  console.log(arr.length);  
}
```

3. Ограничения (`extends`)

Можно ограничить, какие типы допустимы:

```
function getLength<T extends { length: number }>(arg: T): number {  
  return arg.length;  
}  
  
getLength("строка"); // OK  
getLength([1, 2, 3]); // OK  
getLength(123); // ❌ Ошибка: у number нет length
```

4. Дженерики в интерфейсах


```
interface Box<T> {
  value: T;
}

let stringBox: Box<string> = { value: "hello" };
let numberBox: Box<number> = { value: 123 };
```

5. Дженерики в классах

```
class DataHolder<T> {
  constructor(public data: T) {}

  get(): T {
    return this.data;
  }
}

const numberHolder = new DataHolder<number>(42);
```

6. Дженерики с несколькими параметрами

```
function merge<T, U>(obj1: T, obj2: U): T & U {
  return { ...obj1, ...obj2 };
}

const combined = merge({ name: "Alice" }, { age: 30 });
// combined имеет тип: { name: string, age: number }
```

Если ты хочешь описать **обобщённую (generic)** функцию в TypeScript, у которой:

- вход имеет тип `T`,
- выход — тип `P`,

то тебе нужен **generic с двумя параметрами**: `T` и `P`.



Синтаксис:

```
function yourFunc<T, P>(input: T): P {
  // ...
}
```



```
}
```

Здесь `T` — тип входа, `P` — тип выхода, а функция возвращает `P`.

Пример 1: преобразование объекта

```
function transform<T, P>(input: T, converter: (x: T) => P): P {
  return converter(input);
}

// Использование:
const result = transform({ name: "Alice" }, x => x.name.length);
// T = { name: string }, P = number
```

✓ TypeScript автоматически выводит `T` и `P`.

Пример 2: асинхронная функция

```
async function asyncTransform<T, P>(value: T, handler: (v: T) => Promise<P>): Promise<P> {
  return await handler(value);
}
```

Пример 3: тип функции

Если ты описываешь **тип** (например, для переменной или пропса), делай так:

```
type Mapper<T, P> = (input: T) => P;

const stringLength: Mapper<string, number> = str => str.length;
```

А если `P` зависит от `T`?

Тогда можно использовать `infer` или обобщения внутри типа `P<T>`, например:


```
type Output<T> = T extends string ? number : boolean;

function convert<T>(input: T): Output<T> {
  // Пример для иллюстрации – реализация требует type guard'ов
  return typeof input === "string" ? input.length : true as Output<T>;
}
```

Резюме

Что нужно	Как записать
Generic вход и выход	<code>function fn<T, P>(arg: T): P</code>
Тип функции	<code>type Fn<T, P> = (x: T) => P</code>
Зависимый выход	<code>type Output<T> = ...</code> + <code>Output<T></code>

♦ Итог

Дженерики позволяют:

- Писать универсальный код без потери типизации
- Повышать читаемость и повторное использование
- Сохранять строгую типовую безопасность

Специальные типы

специальными типами в TypeScript — они нужны для случаев, когда обычные базовые типы не подходят, или для улучшения безопасности и контроля кода.

Основные специальные типы:

Тип	Описание	Когда использовать
<code>any</code>	Любой тип, отключает проверку типов	Когда тип неизвестен или нужна свобода

<code>unknown</code>	Неизвестный тип, безопаснее, чем <code>any</code>	Когда есть данные, но не знаешь тип
<code>never</code>	Тип, который никогда не возникает	Для функций, которые не возвращают (бросают ошибку или бесконечно работают)
<code>void</code>	Отсутствие значения (обычно для функций)	Для функций, которые ничего не возвращают

Подробнее с примерами

1. ``any``

```
let a: any = 123;
a = "привет";      // Можно всё
a.foo.bar();       // Ошибок TypeScript нет, но может быть runtime ошибка
```

 Используй `any` осторожно — теряешь преимущества типизации.

2. ``unknown``

```
let u: unknown = 5;
u = "hello";

// Нельзя просто так использовать:
u.toUpperCase(); // ❌ Ошибка

if (typeof u === "string") {
  u.toUpperCase(); // ✅ ОК, проверили тип
}
```

`unknown` — безопаснее `any`, заставляет проверять тип перед использованием.

3. ``never``

```
function fail(msg: string): never {
  throw new Error(msg);
}

function infiniteLoop(): never {
```



```
while(true) {}  
}
```

`never` означает: **эта функция никогда не вернёт управление.**

4. `void`

```
function log(msg: string): void {  
  console.log(msg);  
  // ничего не возвращаем  
}
```

`void` — тип для функций, которые не возвращают значение (или возвращают `undefined`).

Когда что использовать

Тип	Основное назначение
<code>any</code>	Быстрая "затычка", когда тип неизвестен
<code>unknown</code>	Безопасное хранение "любой" информации
<code>never</code>	Обозначение невозможных состояний
<code>void</code>	Функции без возвращаемого значения